QF5204 Numerical Methods in Quantitative Finance

AY2025/26 Sem2 By Zhao Peiduo

Lecture 1

Numerical Methods in the Financial Industry Numerical methods play a critical role in modeling, analyzing, and solving complex problems in finance, especially when analytical solutions are infeasible. They are widely applied in risk management, derivative pricing, portfolio optimization, and market simulation.

Key Applications Option pricing and hedging, risk management, portfolio optimization, fixed-income analytics, algorithmic trading.

What Is a Numerical Method A numerical method focuses on obtaining approximate solutions to mathematical problems when exact, closed-form solutions are unavailable or impractical.

Main Components

- Theoretical development of numerical schemes
- Practical implementation as efficient and reliable algorithms

How Numerical Methods Work Simplify complex problems to tractable approximations and solve them numerically. These methods rely on tools from linear algebra, real analysis, and functional analysis. A central focus is understanding and controlling error, including error magnitude, analytical bounds, efficiency, and computational cost.

Core Considerations

- **Accuracy:** ensuring results are precise and reliable
- **Computational cost:** balancing efficiency with time and resource constraints

Python Toolbox

- Packages: numpy, scipy, matplotlib, pandas, statsmodels, quandl, matplotlib
- Techniques: Reshaping, reference vs copy, j for complex number, indexing and slicing, upcasting, broadcasting, vectorization, lambda expressions.
- Practices: Modularization and organization, error handling, computational cost estimation

Errors in Numerical Methods Numerical methods produce approximate solutions when analytical solutions are unavailable, making error analysis central to reliability.

Types of Errors

- **Modeling error:** mismatch between physical reality and simplified mathematical model
- **Truncation error:** approximation of infinite processes by finite ones
- **Round-off error:** finite precision limitations in computer arithmetic

Approximation Pipeline

- Problem $\to$ mathematical model (modeling error)
- Model $\to$ numerical method (truncation error)
- Method $\to$ computation (rounding error)

Numerical Error Measures

- **True error:** $E_t = \text{true value} - \text{approximation}$
- **Relative error:** $\dfrac{E_t}{\text{true value}}$
- **Relative error percentage:** $\dfrac{E_t}{\text{true value}} \times 100\%$

Binary Number Representation Computers use binary representations composed of bits $\{0, 1\}$

- Positive integer:

$$d_n d_{n-1} \cdots d_0 = \sum_{i=0}^{n} d_i 2^i$$

- Fractional number:

$$d_{-1} d_{-2} \cdots d_{-m} = \sum_{i=1}^{m} d_{-i} 2^{-i}$$

Integer Ranges (32-bit) Unsigned: $[0, 2^{32} - 1]$; Signed: $[-2^{31}, 2^{31} - 1]$

Fixed-Point Representation

- Binary point is fixed at a predefined position
- Defined by width w and binary point position b
- Denoted as $\text{fixed}\langle w, b\rangle$; Integer type: $\text{fixed}\langle 32, 0\rangle$
- Limited range $\Rightarrow$ overflow error when exceeded

Fixed-Point Value Formula

For $\text{fixed}\langle w, b\rangle$ with bits $d_{w-1} \cdots d_0$:

$$x = \begin{cases} \displaystyle\sum_{k=0}^{w-2} d_k 2^{k-b}, & \text{if } d_{w-1} = 0 \quad \text{(positive real number)}, \\ & \text{e.g., } 0111_2 \text{ in } \text{fixed}\langle 4, 1\rangle : \ x = 2 + 1 + 0.5 = 3.5, \\ \displaystyle -2^{w-1-b} + \sum_{k=0}^{w-2} d_k 2^{k-b}, & \text{if } d_{w-1} = 1 \quad \text{(negative real number)}, \\ & \text{e.g., } 1011_2 \text{ in } \text{fixed}\langle 4, 1\rangle : \ x = -4 + 1 + 0.5 = -2.5 \end{cases}$$

Floating-Point Representation (IEEE 754) Binary scientific notation

- Structure: sign bit, exponent bits, significand bits
- Value:

$$(-1)^s (1 + \text{significand}) 2^{\text{exponent} - \text{bias}} \text{(e.g.} 1.01 \times 2^{-1}, \ 1{,}01 \text{ Significand; -1 Exponent)}$$

Floating-Point Ranges 32-bit float: $\approx 10^{-38}$ to 10^{38}; 64-bit float: $\approx 10^{-308}$ to 10^{308}

Numerical Errors Definitions

- True error: $E_t = \text{True value} - \text{Approximation}$
- Relative error: $E_r = \dfrac{|E_t|}{|\text{True value}|}$
- Relative error percentage: $\%\ \text{error} = E_r \times 100\%$

Number Representation and Range

- **Binary representation:** integers use bits $d_n \cdots d_0$ with value $\sum_{k=0}^{n} d_k 2^k$.
- **32-bit integer ranges:** unsigned $[0, 2^{32} - 1]$, signed $[-2^{31}, 2^{31} - 1]$.
- **Fixed-point:** type $\text{fixed}\langle w, b\rangle$ has width w and binary point at position b; limited range $\Rightarrow$ overflow possible.
- **Floating-point:** scientific notation in base 2 (IEEE 754); wide dynamic range. Typical ranges: 32-bit float $\approx 10^{-38}$ to 10^{38}, 64-bit float $\approx 10^{-308}$ to 10^{308}.

Rounding Error Effects

- Errors accumulate gradually across computations
- Subtracting nearly equal numbers causes catastrophic cancellation
- Adding numbers of very different magnitudes causes loss of precision

Machine Epsilon

- Machine epsilon ε is the smallest number such that $1 + \varepsilon \neq 1$ in machine arithmetic.
- It quantifies round-off effects and depends on hardware/precision.

Truncation Error (Taylor-based) Truncation error comes from dropping higher-order terms.

$$f(x + h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \cdots$$

$$f(x) = f(a) + f'(a)(x - a) + \cdots + \frac{f^{(n)}(a)}{n!}(x - a)^n + R_n(x), \qquad R_n(x) = \frac{f^{(n+1)}(c)}{(n + 1)!}(x - a)^{n+1}.$$

Local vs. Global Truncation Error

- **Local truncation error:** error introduced in one step; for first-order Taylor,

$$f(x + h) = f(x) + hf'(x) + O(h^2), \quad \Rightarrow \text{local error } = O(h^2).$$

- A method has order p if local truncation error is $O(h^{p+1})$.
- **Global truncation error:** accumulation over K steps with $\Delta h = h/K$:

$$e_K \approx K \cdot O(\Delta h^2) = \frac{h}{\Delta h} O(\Delta h^2) = O(h\Delta h).$$

- Convergent methods satisfy global error $\to 0$ as $\Delta h \to 0$.

Convergence and Rate

- $\{x_k\}$ converges to x^* if $\lim_{k\to\infty} x_k = x^*$.
- With error $e_k = |x_k - x^*|$, order p if

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^p} = c_0.$$

Common Convergence Behaviors

- **Linear ($p = 1$):** $\|x_{k+1} - x^*\| \le c_0 \|x_k - x^*\|, \ 0 < c_0 < 1$.
- **Sub-linear:** Logarithmic Convergence: $\|x_k - x^*\| = O(1/\log k)$ or Polynomial Convergence: $O(1/k^p)$.
- **Super-linear:** $\dfrac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \to 0$.
- **Quadratic ($p = 2$):** $\dfrac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} = c_0$

Tolerance and Stopping Criteria

- True error requires knowing the true value:

$$E_t = \text{True value} - \text{Approximation}.$$

- In practice, the true value is unknown, so we use tolerance-based tests to decide when an iterate is "close enough".

Typical tests (tolerance ε_0) Choice of test depends on the application (scaling, units, noise sensitivity).

- **Absolute change:** $|x_{k+1} - x_k| \le \varepsilon_0$.
- **Relative change:** $\dfrac{|x_{k+1} - x_k|}{|x_k|} \le \varepsilon_0$.

Lecture 2

The Time Value of Money

- An amount x invested at interest rate r grows to $x(1 + r) = x + xr$, where x is the principal and xr is the interest.
- With continuous compounding for n years, the value becomes xe^{rn}.
- A *fixed-income security* provides a predetermined stream of cash flows.
- *Annuities*: pay a fixed amount A at the end of every period indefinitely.
- *Bonds*: pay a face (par) value at a specified maturity time.

Introduction to Forward Contracts

- A *derivative* is a security whose payoff depends on another variable.
- A *forward contract* fixes today the price of an asset delivered at future time T.
- Two times are fixed: 0 (today) and T (delivery). $F(0, T)$ denotes the forward price agreed upon today.
- *Long position*: agree to buy at T for $F(0, T)$.
- *Short position*: agree to sell at T for $F(0, T)$.

No-Arbitrage Condition in Forward Pricing

- Arbitrage arises if the forward price deviates from its fair theoretical value.
- *Cash-and-carry arbitrage* (forward overpriced): borrow at rate r, buy spot asset at S, short the forward, profit $F - Se^{rT}$.
- *Reverse cash-and-carry* (forward underpriced): short the asset, invest proceeds, go long the forward, profit $Se^{rT} - F$.
- No-arbitrage implies the cost-of-carry formula $F = Se^{rT}$.

Dynamic (Tree-Based) Models

- Discrete-time price dynamics are commonly modeled using trees.
- Asset values evolve multiplicatively: $S \to uS$ (up) or dS (down).
- Constants u and d are fixed multipliers.
- Let p be the probability of an up move and $1 - p$ of a down move.

Binomial Tree Parameters

- Define yearly expected growth rate and volatility:

$$\mu = \mathbb{E}[\ln(S(1)/S(0))], \quad \sigma = \sqrt{\mathrm{Var}(\ln(S(1)/S(0)))}.$$

- Choose parameters: Let Δt be the time step (typically a day or a week). We model the log-price increment over one time step Δt by a binomial move:

$$\log \frac{S_{t+\Delta t}}{S_t} = \begin{cases} \log u, & \text{with probability } p, \\ \log d, & \text{with probability } 1-p. \end{cases}$$

Choose symmetric up/down factors

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}} \quad \Rightarrow \quad \log u = \sigma\sqrt{\Delta t}, \quad \log d = -\sigma\sqrt{\Delta t}.$$

The expected log-return is then

$$\mathbb{E}\left[\log \frac{S_{t+\Delta t}}{S_t}\right] = p \log u + (1-p)\log d = (2p-1)\sigma\sqrt{\Delta t}.$$

To match the drift μ of the continuous-time log-price,

$$\mathbb{E}\left[\log \frac{S_{t+\Delta t}}{S_t}\right] \approx \mu\,\Delta t \Rightarrow (2p-1)\sigma\sqrt{\Delta t} = \mu\,\Delta t.$$

Solving for p yields

$$p = \frac{1}{2} + \frac{1}{2}\frac{\mu}{\sigma}\sqrt{\Delta t}.$$

General Discrete-Time Models

- Consider the multiplicative model

$$S(k+1) = u(k)S(k),$$

where $\{u(k)\}$ are independent random variables.
- Taking logarithms,

$$\ln S(k+1) = \ln S(k) + \ln u(k).$$

- If $X \sim \mathcal{N}(\mu, \sigma^2)$, then $Y = e^X$ is lognormally distributed:

$$Y \sim LN(\mu, \sigma^2), \quad \text{and} \quad \mathbb{E}[Y] = e^{\mu+\sigma^2/2}.$$

- In particular, in a lognormal multiplicative model with $u(k) \sim LN(\nu, \sigma^2)$, the log-returns satisfy

$$\mathbb{E}\left[\ln \frac{S(k+1)}{S(k)}\right] = \mathbb{E}[\ln u(k)] = \nu \quad \mathrm{Var}\left[\ln \frac{S(k+1)}{S(k)}\right] = \mathrm{Var}[\ln u(k)] = \sigma^2.$$

Random Walks and Wiener Process

- Discrete stochastic process:

$$z(t_{k+1}) = z(t_k) + \epsilon(t_k)\sqrt{\Delta t}, \quad \epsilon(t_k) \sim \mathcal{N}(0,1).$$

- As $\Delta t \to 0$, $z(t)$ converges to the Wiener process:

$$dz(t) = \epsilon(t)\sqrt{dt}.$$

- Properties of Brownian motion: $z(0) = 0$, continuous paths,

$$z(t) - z(s) \sim \mathcal{N}(0, t-s), \quad t > s.$$

Ito Process and Geometric Brownian Motion

- Wiener process with drift:

$$dx(t) = a\,dt + b\,dz(t), \quad a, b \in \mathbb{R}.$$

- Integral form:

$$x(t) = x(0) + at + bz(t).$$

- General Ito process:

$$dx(t) = a(t)\,dt + b(t)\,dz(t).$$

- Geometric Brownian motion (GBM) for stock prices:

$$dS(t) = \mu S(t)\,dt + \sigma S(t)\,dz(t).$$

Ito's Lemma

- Let $x(t)$ follow an Ito process and $G(x,t)$ be a smooth function.
- Taylor expansion (ignoring higher-order terms):

$$\Delta G = \frac{\partial G}{\partial x}\Delta x + \frac{\partial G}{\partial t}\Delta t + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}(\Delta x)^2.$$

- With $\Delta x = a\Delta t + b\epsilon\sqrt{\Delta t}$, $\epsilon \sim \mathcal{N}(0,1)$ and $\mathbb{E}[\epsilon^2] = 1$,
- Taking limits yields Ito's lemma:

$$\Delta G = \frac{\partial G}{\partial x}\Delta x + \frac{\partial G}{\partial t}\Delta t + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\epsilon^2\Delta t \quad \Rightarrow \quad \Delta G = \frac{\partial G}{\partial x}\Delta x + \frac{\partial G}{\partial t}\Delta t + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\Delta t$$

Ito's Lemma for Stock Prices

- Taking limits, for a function $G(x,t)$:

$$dG = \frac{\partial G}{\partial x}\,dx + \frac{\partial G}{\partial t}\,dt + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\,dt.$$

- Substituting $dx = a\,dt + b\,dz$, we obtain Ito's lemma:

$$dG = \left(\frac{\partial G}{\partial x}a + \frac{\partial G}{\partial t} + \frac{1}{2}\frac{\partial^2 G}{\partial x^2}b^2\right)dt + \frac{\partial G}{\partial x}b\,dz.$$

- Application to the stock price process:

$$dS = \mu S\,dt + \sigma S\,dz.$$

- For a function $G(S,t)$:

$$dG = \left(\frac{\partial G}{\partial S}\mu S + \frac{\partial G}{\partial t} + \frac{1}{2}\frac{\partial^2 G}{\partial S^2}\sigma^2 S^2\right)dt + \frac{\partial G}{\partial S}\sigma S\,dz.$$

Options

- An option is a financial contract whose value depends on an underlying asset.
- The holder has the right, but not the obligation, to trade at a fixed price.
- Strike price: K, maturity time: T.
- Call option: right to buy at time T for price K.
- Put option: right to sell at time T for price K.
- Payoffs at maturity:

$$\text{Call: } \max(S_T - K, 0), \qquad \text{Put: } \max(K - S_T, 0).$$

- Derivative payoff is a function of the underlying: $X = F(S_T)$.

Option Strategies

- Option strategies combine basic options to create desired payoffs.
- Strangle: buy a put at K_1 and a call at $K_2 > K_1$.
- Straddle: special case of strangle with $K_1 = K_2$.
- Bull spread: buy call at K_1, sell call at $K_2 > K_1$.
- Bear spread: sell put at K_1, buy put at $K_2 > K_1$.

Option Moneyness

- Moneyness describes the relation between stock price S and strike K.
- In-the-Money (ITM): option has positive intrinsic value if exercised now.
- At-the-Money (ATM): $S \approx K$.
- Out-of-the-Money (OTM): option has zero intrinsic value if exercsised.

Financial Derivatives

- A financial derivative is a contract whose value depends on an underlying asset (stocks, bonds, commodities).
- Types of derivatives:
  - Vanilla options: standardized, simple payoff structures.
  - Exotic options: complex features and conditions.
- Linear products include forwards, futures, and swaps.
- **Law of One Price**: identical assets must have the same price in frictionless markets.

European Options and Black–Scholes Model

- European options may only be exercised at expiry.
- Priced analytically using the Black–Scholes model.
- Assumptions:
  - No arbitrage opportunities.
  - Constant risk-free interest rate.
  - Log-normal distribution of underlying prices.
  - No dividends.
  - Frictionless markets.

Pricing Exotic Options

- Exotic options have complex or path-dependent payoffs.
- Examples:
  - American options: exercisable anytime before expiry.
  - Barrier options: activated when price hits a barrier.
  - Asian options: payoff depends on average price.
- Analytical pricing may fail due to complexity.
- Numerical approaches:
  - Binomial tree models.
  - Monte Carlo simulation.
  - Finite difference methods.

Binomial Tree Model

- Discrete-time model for option pricing.
- Asset price moves up or down at each step.
- Proposed by Cox, Ross, and Rubinstein (1979).
- Advantages: simple, applicable to European and American options.

One-Step Binomial Model

- Time interval $[0, T]$ with two possible states at T.
- Initial stock price: S_0.
- Up state: $S_u = uS_0$, with probability p.
- Down state: $S_d = dS_0$, with probability $1 - p$.
- Risk-free interest rate: r.

Pricing by Replication

- Portfolio consists of δ shares of stock and cash B.
- Initial value:
$$V_0 = \delta S_0 + B.$$
- Replication conditions:
$$\delta S_u + e^{rT} B = V_u, \quad \delta S_d + e^{rT} B = V_d.$$

Option Payoffs

- Call option:
$$V_u = \max(S_u - K, 0), \quad V_d = \max(S_d - K, 0).$$
- Put option:
$$V_u = \max(K - S_u, 0), \quad V_d = \max(K - S_d, 0).$$

Solving the Linear System

- Hedge ratio:
$$\delta = \frac{V_u - V_d}{S_u - S_d}.$$
- Cash position:
$$B = e^{-rT}(V_u - \delta S_u).$$
- Option price:
$$V_0 = e^{-rT}(q_u V_u + q_d V_d),$$
where
$$q_u = \frac{S_0 e^{rT} - S_d}{S_u - S_d}, \quad q_d = \frac{S_u - S_0 e^{rT}}{S_u - S_d}.$$

Implementation

- Inputs:
  - Stock price S_0, interest rate r.
  - Up/down factors u, d.
  - Option type, strike K, maturity T.
- Output: option price.
- One-step binomial pricing demonstrated for call and put.
- In practice, u and d are chosen using volatility σ.

Is the Binomial Tree Model Still Useful?

- The binomial tree is a simplified, step-by-step framework for understanding option price evolution.
- It provides a discretized approximation of the continuous Black–Scholes model.
- It avoids explicitly fixing S_u and S_d by linking them to volatility σ.
- Stock prices are viewed as following a continuous stochastic process (GBM).

Intuition Behind Black–Scholes

- Stock prices evolve continuously rather than via discrete up/down jumps.
- Two driving components:
  - Drift: average rate of return.
  - Randomness: market uncertainty (volatility).

Black–Scholes Model

- Under the risk-neutral measure, the stock price satisfies
$$dS_t = S_t(r\,dt + \sigma\,dW_t), \qquad \frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t.$$
- Parameters:
  - r: risk-free interest rate,
  - σ: volatility,
  - W_t: standard Brownian motion.
- Assumptions: continuous trading, constant r and σ.

Solution of the Stock Price SDE

- Applying Itô's lemma to $f(S_t) = \ln S_t$:
$$d\ln S_t = \left(r - \tfrac{1}{2}\sigma^2\right)dt + \sigma\,dW_t.$$
- Hence,
$$S_t = S_0 \exp\left(\left(r - \tfrac{1}{2}\sigma^2\right)t + \sigma W_t\right), \quad W_t \sim \mathcal{N}(0, t).$$

Black–Scholes Option Pricing

- Call option price at time 0:
$$C(S_0, K, T) = e^{-rT}\mathbb{E}_Q[(S_T - K)_+].$$
- Defining
$$d_\pm = \frac{\ln(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}},$$
the closed-form solution is
$$C(S_0, K, T) = S_0 N(d_+) - Ke^{-rT}N(d_-).$$
- Corresponding put price:
$$P(S_0, K, T) = Ke^{-rT}N(-d_-) - S_0 N(-d_+).$$

Binomial vs. Black–Scholes at Maturity

- Binomial tree: two-point discrete distribution for S_T vs Black–Scholes: continuous lognormal distribution.
- Both models share the same mean:
$$\mathbb{E}[S_T] = S_0 e^{rT}.$$
- Variances differ, but converge as the tree is refined.

Matching the Variance of Two

- One-step binomial model:
$$S_u = uS_0, \quad S_d = dS_0, \quad u > 1,\ 0 < d < 1.$$
- Variance of binomial tree:
$$\mathrm{Var}(S_T) = S_0^2\left(pu^2 + (1-p)d^2\right) - S_0^2 e^{2rT}.$$
- Variance under Black–Scholes:
$$\mathrm{Var}(S_T) = S_0^2 e^{2rT}\left(e^{\sigma^2 T} - 1\right).$$
- Matching variances requires
$$e^{2rT + \sigma^2 T} = pu^2 + (1-p)d^2.$$
- CRR assumption:
$$d = \frac{1}{u}.$$

Risk-Neutral Probability

- No-arbitrage condition:
$$pu + (1-p)d = e^{rT}.$$
- Risk-neutral probability:
$$p = \frac{e^{rT} - 1/u}{u - 1/u}.$$

Solving for u

- Substitution gives:
$$e^{2rT + \sigma^2 T} = e^{rT}\left(u + \frac{1}{u}\right) - 1.$$
- Quadratic equation:
$$u^2 - \left(e^{rT + \sigma^2 T} + e^{-rT}\right)u + 1 = 0.$$
- Roots:
$$u = \frac{b + \sqrt{b^2 - 4}}{2}, \quad d = \frac{b - \sqrt{b^2 - 4}}{2},$$
where $b = e^{rT + \sigma^2 T} + e^{-rT}$.

Bringing It Closer

- One-step variance matching does not fully replicate Black–Scholes.
- Binomial model approximates a continuous stochastic process.
- Accuracy improves by increasing time steps.

Multi-Step Binomial Tree

- Divide $[0, T]$ into N steps with $\Delta t = T/N$.
- State probabilities:
$$p(S_{N,i}) = \binom{N}{i} p^i (1-p)^{N-i}.$$
- Recombining tree ensures polynomial growth of nodes.

Moments of S_T

- First moment:
$$\mathbb{E}[S_T \mid S_0] = S_0(pu + (1-p)d)^N.$$
- Second moment:
$$\mathbb{E}[S_T^2 \mid S_0] = S_0^2(pu^2 + (1-p)d^2)^N.$$

N-Step Matching with Black–Scholes

- Risk-neutral probability:
$$p = \frac{e^{r\Delta t} - d}{u - d}.$$
- Mean matching:
$$\mathbb{E}[S_T \mid S_0] = S_0 e^{rT}.$$
- Variance matching:
$$u = \frac{b + \sqrt{b^2 - 4}}{2e^{r\Delta t}}, \quad d = \frac{1}{u}, \quad \text{where} \quad b = e^{(2r + \sigma^2)\Delta t} + 1.$$
- As $N \to \infty$, S_T converges to a log-normal distribution.

Lecture 3

What's the Next Step?

- European calls/puts and digitals are not the only tradable products. Other products of interest include European options with generic payoff, American options, Barrier options. etc.
- Many products have no closed-form solutions, and Binomial tree pricer can handle these cases.
- Goal: refine implementation to be easier to maintain and extend.

American Options

- At each node, compute the continuation value via conditional expectation.
- American options allow early exercise at any node.
- Two quantities at each node:
  - Intrinsic value:
$$\max(S-K,0)\ \text{(Call)}, \qquad \max(K-S,0)\ \text{(Put)}.$$
  - Continuation value: expected discounted future payoff.
- Decision rule:
$$V(t) = \max(\text{Intrinsic Value},\ \text{Continuation Value}).$$

Optimal Exercise Strategy

- Continue holding if continuation value exceeds intrinsic value.
- Exercise immediately if intrinsic value exceeds continuation value.
- At every node, maximize the option value.

Refining Code Structure

- Modify European pricer `n_step_binomial_tree`.
- At each iteration: continuation value (no exercise), payoff value (immediate exercise)
- Avoid copy–paste; improve extensibility.
- Allow flexible payoff definitions.

First-Step Extension

- Encapsulate tradable instrument parameters.
- European options:
  - expiry T, strike K, payoff type
- American options:
  - European features
  - early exercise capability

Tree Algorithm (American Option)

- Initialize terminal payoff.
- Backward induction for $k = N-1, \dots, 0$:
  - Compute discounted continuation value.
  - Compute option value from payoff and continuation.
- Step 5: tree pricer.
- Step 6: product-specific payoff rule.

Call Spread

- Strategy: buy and sell calls with different strikes.
- Bull call spread:
  - Buy call at lower strike K_{low}.
  - Sell call at higher strike K_{high}.
- Risk: limited to total premium paid.
- Reward: capped at $K_{\text{high}} - K_{\text{low}}$ minus cost.

Barrier Options

- Payoff depends on whether the underlying hits a barrier.
- Defined by:
  - one or two barriers (up/down),
  - monitoring window (usually $[0,T]$).
- Knock-In (KI): option becomes active only if barrier is hit.
- Knock-Out (KO): option expires if barrier is hit.
- Underlying payoff:
  - European payoff, or
  - fixed rebate (touch option).

Barrier Options Example (1): Up-and-In Call

- Strike price: $K = 100$
- Barrier level: $B = 120$
- Option premium: \$6

Payoff Scenarios

- Barrier touched, $S_T = 130$:
$$\text{Payoff} = 30, \quad \text{Net Profit} = 24$$
- Barrier not touched, $S_T = 130$:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -6$$
- Barrier touched, $S_T = 95$:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -6$$

Motivation

- Lower premium than vanilla call due to conditional activation.
- Useful when exposure is desired only after breaching a price level.

Barrier Options Example (2): Up-and-Out Call

- Strike price: $K = 100$
- Barrier level: $B = 120$
- Option premium: \$8

Payoff Outcomes

- Barrier not touched, $S_T = 110$:
$$\text{Payoff} = 10, \quad \text{Net Profit} = 2$$
- Barrier touched at any time:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -8$$
- $S_T < 100$:
$$\text{Payoff} = 0, \quad \text{Net Profit} = -8$$

Motivation

- Cheaper than vanilla call since payoff is eliminated if barrier is hit.
- Suitable for moderate upside views below the barrier.

CRR Binomial Tree Recap

- Constraint: $u = \frac{1}{d}$.
- Second moment matching:
$$e^{(2r+\sigma^2)\Delta t} = pu^2 + (1-p)d^2.$$
- First moment matching:
$$p = \frac{e^{r\Delta t} - d}{u - d}.$$

Pricing Knock-Out Barrier Options

- At each node, check barrier condition.
- If barrier is triggered:
$$V = 0.$$
- Otherwise, continuation value:
$$V = e^{-r\Delta t}(pV_{\text{up}} + (1-p)V_{\text{down}}).$$
- Terminal payoff (if not knocked out):
$$\max(S-K,0)\ \text{(Call)}, \qquad \max(K-S,0)\ \text{(Put)}.$$

Alternative Binomial Tree Models

Jarrow–Rudd Model

- Equal probabilities (not risk-neutral): $p = \frac{1}{2}$.
- Parameters:
$$u = e^{(r-\frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}}, \quad d = e^{(r-\frac{1}{2}\sigma^2)\Delta t - \sigma\sqrt{\Delta t}}.$$

Tian Binomial Model

- Matches first three moments.
- Parameters:
$$p = \frac{e^{r\Delta t} - d}{u - d},$$
$$u = e^{(r-\frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}}, \quad d = e^{(r-\frac{1}{2}\sigma^2)\Delta t - \sigma\sqrt{\Delta t}}.$$

Options with Continuous Dividend Yield

- Continuous dividend yield at rate q:
$$1 \text{ unit at } t=0 \ \longrightarrow\ e^{qT} \text{ units at } T.$$
- Replicating portfolio at T:
$$\delta S_T e^{qT} + e^{rT}(V_0 - \delta S_0).$$
- Replication equations:
$$\begin{cases} \delta S_u e^{qT} + e^{rT}(V_0 - \delta S_0) = V_u, \\ \delta S_d e^{qT} + e^{rT}(V_0 - \delta S_0) = V_d. \end{cases}$$
- Solution:
$$\delta = e^{-qT}\frac{V_u - V_d}{S_u - S_d}, \qquad V_0 = e^{-rT}(pV_u + (1-p)V_d),$$
$$p = \frac{S_0 e^{(r-q)T} - S_d}{S_u - S_d}.$$

Black–Scholes and CRR Tree Models

- Without dividends:
$$\frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t.$$
- With continuous dividend yield q:
$$\frac{dS_t}{S_t} = (r-q)\,dt + \sigma\,dW_t.$$
- Second moment matching:
$$\mathbb{E}[S_T^2] = e^{2(r-q)T + \sigma^2 T} = pu^2 + (1-p)d^2.$$
- CRR constraints ($u = 1/d$):
$$u = \frac{b + \sqrt{b^2-4}}{2}, \quad d = \frac{b - \sqrt{b^2-4}}{2},$$
$$b = e^{(r-q)T + \sigma^2 T} + e^{-(r-q)T}.$$

Path-Dependent Options in Trees

- American options: assume not exercised prior to current node.
- Barrier Knock-Out (KO): assume not knocked out yet.
- Other examples:
  - Knock-In (KI): activates only if barrier is hit,
  - Asian options: depend on average price history.

Barrier Options: KO vs KI

- **KO**: option becomes worthless if barrier is breached.
- **KI**: option becomes active only after barrier is breached.
- KO tree: track validity only.
- KI tree: track two states:
  - Barrier not triggered,
  - Barrier already triggered.

KIKO Parity

- KI and KO exhaust all barrier scenarios.
- Pricing identity:
$$\text{KI} = \text{Underlying payoff} - \text{KO}.$$
- Ensures arbitrage-free pricing and numerical stability.

Pricing Knock-In Options on Trees

- Two states at node (i,j):
$$V_{i,j}[0] \text{ (not triggered)}, \quad V_{i,j}[1] \text{ (triggered)}.$$
- Transition:
$$V_{i,j}[0] = \begin{cases} pV_{i+1,j}[0] + (1-p)V_{i+1,j+1}[0], & \text{no hit}, \\ pV_{i+1,j}[1] + (1-p)V_{i+1,j+1}[1], & \text{hit}, \end{cases}$$
$$V_{i,j}[1] = pV_{i+1,j}[1] + (1-p)V_{i+1,j+1}[1].$$

ValueAtNode Function

- Node value represented as a vector:
$$V_{i,j} = \begin{pmatrix} V_{i,j}[0] \\ V_{i,j}[1] \end{pmatrix} = f(\mathbb{E}^{\mathbb{Q}}[V_{i+1,j}]).$$
- Same logic as standard trees, but with vector-valued continuation.

Asian Options with Trees

- Payoff depends on average price $\bar{S}$:
$$\bar{S} = \frac{1}{n}\sum_{i=1}^{n} S_{T_i}.$$
- Asian call payoff:
$$\max(\bar{S} - K, 0).$$
- Decomposition at final fixing:
$$\bar{S} = \frac{1}{n}((n-1)A + S_{T_n}), \quad \tilde{K} = nK - (n-1)A.$$
- Auxiliary state: accumulated average A.
- Tree algorithm:
  - Discretize possible A values,
  - Backward induction for each A_i,
  - Node values returned as vectors.

Handling Intermediate and Fixing Dates (Asian Options)

Between Fixing Dates (T_{i-1}, T_i)

- The accumulated average A does not change,
- Node values equal continuation values.

At Fixing Dates T_i

The accumulated average is updated using the stock price S:
$$\widehat{A} = \frac{A \cdot i + S}{i+1}.$$

Interpolation of the Accumulated Average

Since $\widehat{A}$ may not lie on the grid $\{A_1, A_2, \ldots, A_m\}$, we apply linear interpolation.
Find j such that $\widehat{A} \in [A_j, A_{j+1}]$, and compute
$$V(\widehat{A}) = \frac{\widehat{A} - A_j}{A_{j+1} - A_j} V_{j+1} + \frac{A_{j+1} - \widehat{A}}{A_{j+1} - A_j} V_j,$$
where V_j, V_{j+1} are continuation values.

Additive Binomial Tree

- Use log-price as state variable: $X = \ln S$.
- Define increments:
$$u_x = \ln u, \qquad d_x = \ln d.$$

One-step dynamics (in X)
$$X_u = X_0 + \ln u = X_0 + u_x, \qquad X_d = X_0 + \ln d = X_0 + d_x,$$
with probabilities p and $1-p$ over one step Δt.

Calibration by matching moments

First moment

Black–Scholes solution for X:
$$X = X_0 + \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma W_{\Delta t}.$$
$$\mathbb{E}[X] = X_0 + \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t.$$
In the binomial model,
$$\mathbb{E}[X] = X_0 + pu_x + (1-p)d_x = X_0 + \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t. \tag{10}$$
Equivalently,
$$pu_x + (1-p)d_x = \left(r - \tfrac{1}{2}\sigma^2\right)\Delta t \implies p = \frac{\left(r - \frac{1}{2}\sigma^2\right)\Delta t - d_x}{u_x - d_x}.$$

Second moment
$$\mathbb{E}[X^2] = p(X_0 + u_x)^2 + (1-p)(X_0 + d_x)^2$$
Expanding:
$$\mathbb{E}[X^2] = X_0^2 + 2pX_0u_x + 2(1-p)X_0d_x + pu_x^2 + (1-p)d_x^2$$
Group terms:
$$\mathbb{E}[X^2] = X_0^2 + 2X_0(pu_x + (1-p)d_x) + pu_x^2 + (1-p)d_x^2. \tag{15}$$
Black–Scholes second moment:
$$X_0^2 + 2X_0\left(r - \tfrac{1}{2}\sigma^2\right)\Delta t + \left(r - \tfrac{1}{2}\sigma^2\right)^2\Delta t^2 + \sigma^2\,\mathbb{E}[W_{\Delta t}^2], \tag{16}$$
with
$$\mathbb{E}[W_{\Delta t}^2] = \Delta t.$$
Matching yields:
$$pu_x^2 + (1-p)d_x^2 - \left(r - \tfrac{1}{2}\sigma^2\right)^2\Delta t^2 = \sigma^2\Delta t. \tag{17}$$

CRR assumption
$$u_x = -d_x.$$

Convenient parameterization

Let
$$u_x = -d_x = \Delta x.$$
By CRR and (17):
$$\Delta x^2 = \sigma^2\Delta t + \left((r - \tfrac{1}{2}\sigma^2)\Delta t\right)^2,$$
so
$$\Delta x = \sqrt{\sigma^2\Delta t + \left((r - \tfrac{1}{2}\sigma^2)\Delta t\right)^2} \approx \sigma\sqrt{\Delta t} \qquad \text{when } \Delta t \to 0.$$
Also,
$$p = \frac{\left(r - \frac{1}{2}\sigma^2\right)\Delta t - d_x}{u_x - d_x} = \frac{1}{2} + \frac{r - \sigma^2/2}{2\sigma}\sqrt{\Delta t}.$$

Multi-Dimensional Binomial Method

Spread Option

A spread option gives payoff
$$[S_1(T) - S_2(T)]_+.$$
Assume both assets follow Black–Scholes with correlated Brownian motions:
$$\frac{dS_1}{S_1} = (r - q_1)dt + \sigma_1\,dW_1 \qquad \frac{dS_2}{S_2} = (r - q_2)dt + \sigma_2\,dW_2$$
and correlation
$$\rho\,dt = dW_1\,dW_2. \tag{24}$$

Log-state dynamics

Using the additive tree, define
$$x_1 = \ln S_1, \qquad x_2 = \ln S_2.$$
By Itô's lemma:
$$\begin{cases} dx_1 = \left(r - q_1 - \frac{1}{2}\sigma_1^2\right)dt + \sigma_1\,dW_1, \\ dx_2 = \left(r - q_2 - \frac{1}{2}\sigma_2^2\right)dt + \sigma_2\,dW_2. \end{cases} \tag{25}$$

Four-branch step (equal up/down jump sizes)

Choose equal up and down jump sizes:
$$\Delta x_1 = u_{x_1}, \qquad \Delta x_2 = u_{x_2}.$$
Each step has four outcomes with probabilities $p_{uu}, p_{ud}, p_{du}, p_{dd}$:
$$\begin{aligned} p_{uu}&: \quad (x_1 + \Delta x_1,\ x_2 + \Delta x_2), \\ p_{ud}&: \quad (x_1 + \Delta x_1,\ x_2 - \Delta x_2), \\ p_{du}&: \quad (x_1 - \Delta x_1,\ x_2 + \Delta x_2), \\ p_{dd}&: \quad (x_1 - \Delta x_1,\ x_2 - \Delta x_2). \end{aligned}$$

Calibration

Denote
$$v_1 = r - q_1 - \tfrac{1}{2}\sigma_1^2, \qquad v_2 = r - q_2 - \tfrac{1}{2}\sigma_2^2.$$
Then
$$\begin{cases} dx_1 = v_1\,dt + \sigma_1\,dW_1, \\ dx_2 = v_2\,dt + \sigma_2\,dW_2. \end{cases}$$
So
$$\mathbb{E}[dx_1] = v_1\Delta t, \qquad \mathbb{E}[dx_2] = v_2\Delta t,$$
$$\mathrm{Var}[dx_1] = \sigma_1^2\Delta t, \qquad \mathrm{Var}[dx_2] = \sigma_2^2\Delta t, \qquad \mathrm{Cov}[dx_1, dx_2] = \rho\sigma_1\sigma_2\Delta t.$$
In the binomial model,
$$\mathbb{E}[dx_1] = (p_{uu} + p_{ud})\Delta x_1 - (p_{du} + p_{dd})\Delta x_1,$$
$$\mathrm{Var}[dx_1] = (p_{uu} + p_{ud})\Delta x_1^2 + (p_{du} + p_{dd})\Delta x_1^2 - \left(\mathbb{E}[dx_1]\right)^2 \approx (p_{uu} + p_{ud})\Delta x_1^2 + (p_{du} + p_{dd})\Delta x_1^2 \quad (\Delta t \text{ is very small}),$$
$$\mathrm{Cov}[dx_1, dx_2] = (p_{uu} - p_{ud} - p_{du} + p_{dd})\Delta x_1\Delta x_2.$$

Matching system

$$\begin{cases} \mathbb{E}[dx_1] = (p_{uu} + p_{ud})\Delta x_1 - (p_{du} + p_{dd})\Delta x_1 = v_1 \Delta t, \\ \mathbb{E}[dx_2] = (p_{uu} + p_{du})\Delta x_2 - (p_{ud} + p_{dd})\Delta x_2 = v_2 \Delta t, \\ \mathrm{Var}[dx_1] = (p_{uu} + p_{ud})\Delta x_1^2 + (p_{du} + p_{dd})\Delta x_1^2 = \sigma_1^2 \Delta t + (v_1 \Delta t)^2 \approx \sigma_1^2 \Delta t, \\ \mathrm{Var}[dx_2] = (p_{uu} + p_{du})\Delta x_2^2 + (p_{ud} + p_{dd})\Delta x_2^2 = \sigma_2^2 \Delta t + (v_2 \Delta t)^2 \approx \sigma_2^2 \Delta t, \\ (p_{uu} - p_{ud} - p_{du} + p_{dd})\Delta x_1 \Delta x_2 = \rho \sigma_1 \sigma_2 \Delta t, \\ p_{uu} + p_{ud} + p_{du} + p_{dd} = 1. \end{cases}$$

Solution (model parameters)

$$\Delta x_1 = \sigma_1 \sqrt{\Delta t}, \qquad \Delta x_2 = \sigma_2 \sqrt{\Delta t},$$

$$\begin{cases} p_{uu} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 + \Delta x_2 v_1 \Delta t + \Delta x_1 v_2 \Delta t + \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}, \\ p_{ud} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 + \Delta x_2 v_1 \Delta t - \Delta x_1 v_2 \Delta t - \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}, \\ p_{du} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 - \Delta x_2 v_1 \Delta t + \Delta x_1 v_2 \Delta t - \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}, \\ p_{dd} = \dfrac{1}{4} \dfrac{\Delta x_1 \Delta x_2 - \Delta x_2 v_1 \Delta t - \Delta x_1 v_2 \Delta t + \rho \sigma_1 \sigma_2 \Delta t}{\Delta x_1 \Delta x_2}. \end{cases}$$

Multi-dimensional binomial tree: node definition and pricing
For each node (k, i, j):
- k: time step (starting at $t = 0$),
- i: number of downward moves of asset 1 up to time k,
- j: number of downward moves of asset 2 up to time k.

For asset 1, upward moves are $k - i$, so cumulative change:

$$(k - i)\Delta x_1 - i\Delta x_1 = (k - 2i)\Delta x_1,$$

and similarly for asset 2: $(k - 2j)\Delta x_2$.
Thus prices at node (k, i, j) are

$$S_{1,k,i,j} = S_1 e^{(k-2i)\Delta x_1}, \qquad S_{2,k,i,j} = S_2 e^{(k-2j)\Delta x_2}.$$

Terminal payoff:

$$V = \max(0, S_1 - S_2).$$

Backward induction:

$$V_{k,i,j} = e^{-r\Delta t}\left[p_{uu} V_{k+1,i,j} + p_{ud} V_{k+1,i,j+1} + p_{du} V_{k+1,i+1,j} + p_{dd} V_{k+1,i+1,j+1}\right].$$

Analytic solution for spread option (Margrabe formula)
Price via change of numeraire; express payoff in units of S_2:

$$\left[\frac{S_1(T)}{S_2(T)} - 1\right]_+.$$

Under the dollar measure, the volatility of $\frac{S_1(T)}{S_2(T)}$ is

$$\sigma = \sqrt{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}.$$

Trinomial Tree Model
- The trinomial tree is an extension of the binomial tree model.
- At each time step, the underlying asset has three possible movements:
  - Upward movement: price multiplies by $u > 1$ with probability p_u,
  - Middle movement: price remains unchanged with probability p_m,
  - Downward movement: price multiplies by $d < 1$ with probability p_d.
- Probabilities satisfy:

$$p_u + p_m + p_d = 1.$$

Calibration of the Trinomial Tree
To calibrate parameters (u, d, p_u, p_m, p_d), we match moments of the discrete model to the continuous-time model.
- First moment (mean):

$$e^{\mu\Delta t} = p_u u + p_m + p_d d.$$

- Second moment:

$$e^{2\mu\Delta t + \sigma^2 \Delta t} = p_u u^2 + p_m + p_d d^2.$$

- Symmetry assumption:

$$u = \frac{1}{d}.$$

Underdetermined System
- There are 5 unknowns and only 4 equations.
- This leads to infinitely many solutions.
- To resolve this, introduce a tunable parameter λ controlling jump size.

Choice of Jump Sizes

$$u = e^{\lambda\sigma\sqrt{\Delta t}}, \qquad d = e^{-\lambda\sigma\sqrt{\Delta t}}.$$

Resulting Probabilities

$$p_u = \frac{1}{2\lambda^2} + \frac{\mu - \frac{1}{2}\sigma^2}{2\lambda\sigma}\sqrt{\Delta t},$$
$$p_d = \frac{1}{2\lambda^2} - \frac{\mu - \frac{1}{2}\sigma^2}{2\lambda\sigma}\sqrt{\Delta t},$$
$$p_m = 1 - p_u - p_d.$$

Example Choice
- Let $\lambda = \sqrt{3}$. Then:

$$u = e^{\sqrt{3\Delta t}}, \qquad d = e^{-\sqrt{3\Delta t}}.$$

- Corresponding probabilities:

$$p_u = \frac{1}{6} + \frac{\mu - \frac{1}{2}\sigma^2}{\sqrt{12}\sigma}\sqrt{\Delta t}, \quad p_d = \frac{1}{6} - \frac{\mu - \frac{1}{2}\sigma^2}{\sqrt{12}\sigma}\sqrt{\Delta t}, \quad p_m = \frac{2}{3}.$$

Remarks
- Trinomial trees often provide faster convergence than binomial trees.
- As $\Delta t \to 0$, the trinomial tree converges to the Black–Scholes model.

Lecture 4

From Constant to Implied Volatility
- **Black–Scholes assumption: constant volatility**

$$\frac{dS}{S} = (r - q)\,dt + \sigma\,dW$$

  - r: risk-free rate, q: dividend yield, σ: constant volatility.
  - This simplifies the mathematics and allows for a closed-form solution.
  - The volatility parameter σ must be specified by the user.
- **Market reality: volatility is not constant**
  - Volatility changes over time (volatility clustering).
  - Options with different strikes and maturities trade at different volatilities.
- **Implied volatility (IV): reverse of Black–Scholes**
  - Observe the market price of the option.
  - Solve for the σ that makes the Black–Scholes price match the market price.

Implied Volatility: Definition and Properties
- IV is obtained by **inverting** an option pricing model (e.g., Black–Scholes):
  - Input: observed market option price
  - Output: volatility that reproduces this price in the model
- **Key properties**
  - IV is model-derived, not directly observed.
  - IV is forward-looking (unlike historical volatility).
  - IV is not constant in practice.
- IV varies across:
  - Strike (moneyness), Maturity, Time (sentiment and events).
- **Interpretation:** market replaces constant-σ with IV to better reflect expectations.

Implied Volatility: Practical Meaning and Uses
- **Link to Black–Scholes**
  - Black–Scholes gives option price as a function of volatility.
  - Market practice reverses this relation to obtain implied volatility.
  - IV is the volatility input that makes the BS price match the market price.
  - Hence BS is a quoting and calibration tool, not only a pricing formula.
- **Why practitioners work with IV instead of price**
  - Option prices move mechanically with the spot price.
  - IV is more stable and comparable across contracts.
  - IV provides a normalized scale across strikes and maturities.
- **How traders use IV**
  - Evaluate whether an option is relatively expensive/cheap (vs. model).
  - Trade volatility directly (low-IV buy / high-IV sell strategies).
  - Compute Greeks and manage hedging risk consistently.
- **Result:** options are often quoted and managed in implied-volatility space.

Low vs. High Implied Volatility
- Options with higher IV are priced higher due to greater uncertainty.

Implied Volatility Surface
- In practice, vanilla options (standard calls/puts) are highly liquid and actively traded.
- Market participants agree on prices for common maturities and strikes, forming an **implied volatility surface**.
- Market makers must handle requests across a wide range of strikes and maturities.
- Directly managing option prices is impractical because:
  - Prices depend dynamically on the spot price, which fluctuates in real time.
  - Instead, participants quote IV, e.g. "105 call, 3-month, at 32% vol" rather than "price = 2.73".
- The IV surface allows traders to:
  - Interpolate/extrapolate prices for non-standard options,
  - Adjust for market conditions without recalibrating the entire pricing model.

Aspect	Low IV	High IV
Market expectation	Small moves	Large moves
Sentiment	Calm / Bullish	Fearful / Bearish
Risk	Lower	Higher
Option price	Lower	Higher

Black–Scholes Model: A Handy Tool for Option Pricing

- Directly managing option prices is challenging due to dependence on the underlying spot price.
- Black–Scholes provides a mathematical framework to compute option prices from key parameters:

$$V_{\text{call}}(K, S, \sigma, T, r) = S\,N(d_+) - e^{-rT} K\,N(d_-)$$

where

$$d_\pm = \frac{\ln\left(\frac{S}{K}\right) + (r-q)T \pm \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}.$$

- with S current asset price, K strike, σ volatility, T time to maturity, r risk-free rate, q dividend yield, and $N(\cdot)$ the standard normal CDF.

Implied Volatility as an Implicit Function

- View Black–Scholes as a function from volatility σ to call price V:

$$f_{\{S,r,q,K,T\}}(\sigma) = V.$$

- Implied volatility is obtained by inverting this function:

$$\sigma = f^{-1}(V).$$

- **Key property:** the call price is monotone in σ:
  - When σ increases, V increases (higher volatility raises time value).
  - This holds as long as $T \geq 0$ (time remaining before expiration).
- However, f^{-1} has no closed form $\Rightarrow$ IV is an **implicit** function of σ.
- We therefore resort to a **root finding algorithm** to back out σ from market prices.

Root Finding: Solving $f(x) = 0$

- Given a function f, the goal is to determine the value(s) of x such that

$$f(x) = 0.$$

- This can arise in one-dimensional or multi-dimensional (N-dimensional) settings.
- **One-dimensional problem** ($f : \mathbb{R} \to \mathbb{R}$)
  - Easier to solve due to well-developed robust numerical algorithms.
  - Typically requires only function evaluations or derivatives to iteratively converge.
- **N-dimensional problem** ($f : \mathbb{R}^N \to \mathbb{R}$)
  - More challenging due to complex function landscapes.
  - Structure, curvature, and sensitivity often matter for efficient convergence.
- In implied volatility computation, we mainly deal with the **one-dimensional** case.

Bracketing the root: Ensuring the root is contained

Most fast root finding algorithms require an initial interval (bracket) that contains the root.
A bracket is an interval $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs:

$$f(a) \cdot f(b) \leq 0.$$

This ensures that a root exists within the interval, provided that f is continuous.

How to Bracket the Root?

- Start with an initial interval that is likely to contain the root.
- If the initial interval does not satisfy $f(a) \cdot f(b) \leq 0$, adjust it by:
  - Expanding outward to search for a sign change,
  - Shrinking inward, if the function exhibits a known behavior in a smaller range.
- Once a valid bracket is found, iterative methods can be applied to refine the root estimate.

Root Bracketing Algorithm

Algorithm 1: RootBracketing(f, a, b, maxIter, factor)

- For $k = 1$ to maxIter:
- If $f(a)f(b) < 0$, return (a, b).
- If $|f(a)| < |f(b)|$, update

$$a \leftarrow a + \text{factor}\,(a - b).$$

- Else update

$$b \leftarrow b + \text{factor}\,(b - a).$$

If factor > 0, the algorithm searches outward.
If $-1 <$ factor < 0, the algorithm searches inward.

Root finding algorithms

Now we are equipped with a root bracketing algorithm.

Root finding

Given a function f, find the x such that

$$f(x) = 0.$$

Commonly used root finding algorithms:

- Bisection method
- Secant method
- False position method
- Brent's method
- Newton method

Bisection Method

Start with $[a, b]$ that brackets the root, cut it by half, then see which half contains the root.

Algorithm 2: rfbisect(f, a, b, tol)
Require: $f(a)f(b) < 0$, f is continuous, tolerance tol.

- while $(b - a)/2 >$ tol do
- $c \leftarrow \dfrac{a+b}{2}$
- if $f(c) = 0$ then return c
- else if $f(a)f(c) < 0$ then $b \leftarrow c$
- else $a \leftarrow c$

Bisection Method: Pro and Con

Pros

- Simple and easy to implement
- Only requires function evaluations, no derivatives needed
- Theoretically guaranteed to converge
- Robustness: if the interval contains more than one root, bisection will find one of them

Cons

- Slow convergence with only linear rate

$$\epsilon_n = |c_n - r| \leq \frac{b_n - a_n}{2}, \qquad c_n = \frac{a_n + b_n}{2}$$

Considering worst case:

$$\epsilon_{n+1} = \frac{1}{2}\epsilon_n.$$

Secant method

The secant method is motivated by the need for a faster alternative to bisection by utilizing the slope information of the function, yet does not require derivatives like Newton's method.
Consider a first-order Taylor expansion around a root x:

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n).$$

Since $f(x) = 0$, we have

$$0 \approx f(x_n) + f'(x_n)(x - x_n) \quad \Rightarrow \quad x \approx x_n - \frac{f(x_n)}{f'(x_n)}.$$

Replace the derivative by the first-order difference:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

Secant Method Iteration

The iterative scheme is:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Algorithm 3: rfsecant(f, x_1, x_2, tol)
Require: f is continuous, tolerance tol.

Secant Method: Pro and Con

Pros

- Open bracket method: does not require a bracket containing the root
- Faster convergence than bisection
- Superlinear convergence rate with order ≈ 1.62 (Golden Ratio)
- Works very well for nearly linear function

Cons

- Does not always converge, especially if:
  - The function has discontinuities or steep curvature
  - The denominator $f(x_{n+1}) - f(x_n)$ approaches zero
- Can fail without a proper stopping criterion
- Solution: implement a maximum iteration limit (maxIter)

False Position Method

The **false position method** (regula falsi) is a combination of the **bisection** and **secant** methods.

- Start with an interval $[a, b]$ that brackets the root.
- Draw a secant line through $(a, f(a))$ and $(b, f(b))$.
- Find its x-intercept at c.
- Update a or b to maintain the bracketed root.

Algorithm 4: `rffalsi`(f, a, b, tol)

- Require: $f(a)f(b) < 0$, f continuous
- While $b - a >$ tol:
- Compute

$$c \leftarrow \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

- If $f(c) = 0$, return c.
- Else if $f(a)f(c) < 0$, set $b \leftarrow c$.
- Else set $a \leftarrow c$.

False Position Method: Pro and Con

Pros

- More robust than the secant method (maintains a bracket).
- Guaranteed to find a root if one exists.
- Often faster than bisection.

Cons

- Can converge slowly in certain cases.
- May behave similarly to bisection if one endpoint remains fixed.

Root Finding Algorithms: Considerations

Numerical methods are sensitive to:

- Initial values and assumptions,
- Problem characteristics,
- Tolerance settings,
- Round-off and truncation errors.

Before applying a root-finding algorithm, the problem must be analyzed carefully.

Brent's Method

The bisection method is robust but converges linearly. Secant and false-position methods converge faster but may fail. Brent's method is an adaptive hybrid strategy:

- Uses secant-like steps when they perform well,
- Falls back to bisection when needed,
- Ensures robustness while maintaining fast convergence.

Brent's method combines bisection, secant, inverse quadratic interpolation.

Inverse Quadratic Interpolation

This method improves convergence by fitting a quadratic function through the last three iteration points.

- Given three points
$$(x_0, f(x_0)),\ (x_1, f(x_1)),\ (x_2, f(x_2)),$$
fit
$$q(y) = ay^2 + by + c.$$
- Solve for the next estimate by taking
$$x = q(0).$$

Newton Method

Newton's method starts with one point instead of two.

The iteration scheme is:
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

At each iteration, it updates using the intercept of the tangent line with the x-axis.

Convergence Rate of Newton Method

Newton's method achieves **quadratic convergence** under suitable conditions.

Define
$$g(x) = x - \frac{f(x)}{f'(x)}.$$

Then Newton iteration is
$$x_{n+1} = g(x_n).$$

Define the error $e_n = x_n - r$. By Taylor expansion:
$$g(x_n) = g(r + e_n) = g(r) + g'(r)e_n + O(e_n^2).$$

Note that
$$g(r) = r, \qquad g'(r) = \frac{f(r)f''(r)}{f'(r)^2} = 0.$$

Hence
$$e_{n+1} = \frac{g''(\eta)}{2} e_n^2.$$

Newton Method: Potential Issues: Newton's method requires a good initial guess. Poor initialization may lead to divergence. The remedy is to mix Newton's method with bisection for stability.

Newton Method in Multi-Dimension Space

Newton's method can be extended to solve a system of non-linear equations:
$$f(x) = 0 \quad \to \quad \{f_i(x) = 0,\ 1 \le i \le N\}, \qquad x \in \mathbb{R}^N.$$

Update Rule in Multiple Dimensions

The update rule in multiple dimensions is given by:
$$x_{n+1} = x_n - J^{-1}(x_n)f(x_n).$$

- x_n is the current approximation of the root.
- $J(x)$ is the **Jacobian matrix**, containing first-order partial derivatives:
$$J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

Computational Challenges

Evaluating and inverting $J(x)$ can be computationally expensive.

Instead of directly inverting $J(x)$, one typically solves the linear system:
$$J(x_n)\Delta x = -f(x_n), \qquad \Delta x = x_{n+1} - x_n.$$

Convergence Issues

Newton's method in high dimensions can face convergence difficulties due to:

- Complex nonlinear landscapes,
- Sensitivity to initialization,
- Expensive Jacobian computations.

Root Finding vs. Optimization in Quantitative Finance

In quantitative finance, we rarely encounter direct N-dimensional root-finding problems:
$$f(x) = 0, \qquad x \in \mathbb{R}^N.$$

This is because financial models typically do not require solving general nonlinear systems of equations without additional structure.

Optimization is More Common Instead, we frequently deal with:

- Minimization or maximization problems, where the goal is to optimize an objective function.
- Finding stationary points where the gradient satisfies:
$$\nabla g(x) = 0,$$

which is mathematically equivalent to solving a root-finding problem for the gradient. Financial models impose constraints on gradients, making them predictable and solvable with structured numerical techniques.

Lecture 5

Implied Volatility and Surface Construction

Converting Option Prices to Implied Volatility

We can now compute the implied volatility corresponding to a given option price.

In practice, options are often quoted using implied volatility instead of price because:

- implied volatility is less sensitive to changes than spot price,
- implied volatility is changing less frequently than spot price.

Discrete Data from Market Observations

Assume we have a discrete set of liquid maturities, such as 1W, 2W, 1M, 2M, 6M, 1Y, etc.

For each maturity (or pillar), market data provides a set of strike vs. option price tuples:
$$(k_i, C_i), \qquad i = 1, 2, \ldots, N.$$

Using the root-finding algorithm, we convert these into strike vs. implied volatility tuples:
$$(k_i, \sigma_i), \qquad i = 1, 2, \ldots, N.$$

Constructing the Implied Volatility Surface

To quote or price vanilla options at any arbitrary strike and maturity, we need to construct an implied volatility surface. It requires:

- **Interpolation**: estimating implied volatilities between known market points,
- **Extrapolation**: extending the volatility surface beyond observed strikes or maturities.

A well-constructed volatility surface allows traders and risk managers to:

- Price options consistently across different strikes and expirations,
- Identify market trends and arbitrage opportunities,
- Calibrate pricing models for hedging and risk management.

Volatility Smile Interpolation

Volatility Smile

The function $k \mapsto \sigma$ for a fixed maturity describes how implied volatility changes with strike price.

This function is often referred to as the **volatility smile** due to its typical convex shape.

Understanding the Pattern

Higher implied volatility for deep in-the-money (ITM) and out-of-the-money (OTM) strikes:

- Options far from the current stock price tend to have higher implied volatility.
- Deep ITM and OTM options have lower liquidity and are more sensitive to large price swings.
- Market participants demand higher volatility premiums to compensate for uncertainty.

Lower implied volatility near at-the-money (ATM):

- ATM options typically exhibit the lowest implied volatility.
- These options are the most liquid and actively traded.
- Their implied volatility is stabilized by market forces.

Starting with Linear Interpolation

Given sample points (x_i, y_i), $i \in [0, 1, \ldots, N]$, linear interpolation constructs a piecewise linear function $L(x)$ of the form
$$L(x) = L_i(x), \qquad x \in [x_i, x_{i+1}], \quad i = 0, \ldots, N-1.$$

Such that:

- Each L_i is a linear polynomial,
- $L_0(x_0) = y_0$, $L_{N-1}(x_N) = y_N$,
- $L_i(x_{i+1}) = L_{i+1}(x_{i+1}) = y_{i+1}$.

The linear function is uniquely determined by:
$$L_i(x_i) = y_i, \qquad L_i(x_{i+1}) = y_{i+1}.$$

Let $h_i = x_{i+1} - x_i$, then for $x \in [x_i, x_{i+1}]$:
$$L_i(x) = y_{i+1}\frac{x - x_i}{h_i} + y_i\frac{x_{i+1} - x}{h_i}.$$

Cubic Spline Interpolation

Given sample points (x_i, y_i), cubic spline interpolation constructs a C^2 piecewise cubic polynomial $S(x)$ of the form
$$S(x) = S_i(x), \qquad x \in [x_i, x_{i+1}], \quad i = 0, \ldots, N-1.$$

Such that:

- Each S_i is a cubic polynomial,
- $S_0(x_0) = y_0$, $S_{N-1}(x_N) = y_N$,
- $S_i(x_{i+1}) = S_{i+1}(x_{i+1}) = y_{i+1}$,
- $S_i'(x_{i+1}) = S_{i+1}'(x_{i+1})$,
- $S_i''(x_{i+1}) = S_{i+1}''(x_{i+1})$.

Assume the second derivatives at each node are:
$$S''(x_j) = M_j, \qquad 0 \le j \le N.$$

Then for each S_i:
$$S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}, \quad S_i''(x_i) = M_i, \quad S_i''(x_{i+1}) = M_{i+1}.$$

The four conditions determine a unique cubic polynomial. Let $h_i = x_{i+1} - x_i$:
$$S_i(x) = \frac{M_{i+1}}{6h_i}(x - x_i)^3 + \frac{M_i}{6h_i}(x_{i+1} - x)^3 + \left(y_i - \frac{M_i h_i^2}{6}\right)\frac{x_{i+1} - x}{h_i} + \left(y_{i+1} - \frac{M_{i+1}h_i^2}{6}\right)\frac{x - x_i}{h_i}.$$

Recall S has continuous first derivatives:
$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}).$$

Setting this yields the standard tridiagonal system:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_i M_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right), \qquad 1 \le i \le N-2.$$

Additional boundary conditions:

- Natural spline:

$$M_0 = 0, \qquad M_{N+1} = 0.$$

- Or impose boundary values on first derivatives:

$$S_0'(x_0) = p_0, \qquad S_{N-1}'(x_N) = p_1.$$

Cubic Spline Interpolation in our Application

We may impose clamped boundary conditions by setting y_0' and y_{N-1}' to 0. This ensures flattened volatility beyond the boundary.

Interpolation can be applied to either variance σ^2 or volatility σ.

No-Arbitrage Considerations

- Interpolated implied volatilities are not guaranteed to be arbitrage-free.
- This can be problematic in pegged currency markets where strict rules prevent arbitrage.

Time Interpolation in Implied Volatility

At each maturity, we have a volatility smile capturing the variation of implied volatility across strikes.

When evaluating options expiring between pillar dates, we must interpolate implied volatility across time to obtain a smooth volatility surface.

A common choice is **linear interpolation on total variance** σ^2 along the strike line. This ensures that variance increases with time, which is necessary to preserve calendar arbitrage constraints.

- For options with the same strike, longer maturity implies higher option value.
- Interpolating variance guarantees variance remains increasing, provided variance at the next pillar is higher than at the previous one.

Mathematical Formulation

We interpolate the total variance linearly in time:

$$v_t(K) = \frac{t_{i+1} - t}{t_{i+1} - t_i} v_{t_i}(K) + \frac{t - t_i}{t_{i+1} - t_i} v_{t_{i+1}}(K), \qquad v_t(K) = \sigma_t^2(K)\, t.$$

Implementation of Implied Volatility Surface

1. Interpolation Across Strike (Fixed Maturity)

For each maturity t_i:

- Market provides implied volatility at discrete strikes.
- Construct a smooth volatility smile using **cubic spline interpolation**:

$$\sigma_{t_i}(K) = S_{t_i}(K).$$

Purpose: smooth variation across strike and avoid unrealistic jumps in volatility.

2. Interpolation Across Time (Fixed Strike)

For a given strike K, volatility must be interpolated across maturities. We interpolate **total variance**, not volatility:

$$v(t, K) = \sigma(t, K)^2\, t.$$

Reason: total variance should increase with maturity to prevent calendar arbitrage.

Summary: Computing Implied Volatility at Strike K and Maturity t

- Step 1: For each market maturity t_i, construct cubic spline $S_{t_i}(K)$.
- Step 2: Locate surrounding maturities $t_i \le t \le t_{i+1}$.
- Step 3: Evaluate spline at strike K:

$$\sigma_{t_i}(K) = S_{t_i}(K), \qquad \sigma_{t_{i+1}}(K) = S_{t_{i+1}}(K).$$

- Step 4: Convert to total variance:

$$v_{t_i}(K) = \sigma_{t_i}(K)^2 t_i, \qquad v_{t_{i+1}}(K) = \sigma_{t_{i+1}}(K)^2 t_{i+1}.$$

- Step 5: Linear interpolation in time:

$$v_t(K) = \frac{t_{i+1} - t}{t_{i+1} - t_i} v_{t_i}(K) + \frac{t - t_i}{t_{i+1} - t_i} v_{t_{i+1}}(K).$$

- Step 6: Convert back to volatility:

$$\sigma_t(K) = \sqrt{\frac{v_t(K)}{t}}.$$

Market Variations in Volatility Quotes

- Some asset classes (commodities, equities, rates) provide option price quotes across strikes and maturities.
- Other markets, such as FX options, do not directly quote a full volatility surface.

Since FX markets do not provide implied volatilities for all strikes, an extra step is needed to map market quotes into surface inputs.

What are FX Options?

FX options give the right (but not obligation) to exchange a specified amount of one currency for another at a pre-agreed strike rate on or before maturity.

Key Features of FX Options

- Two interest rates: domestic and foreign.
- Underlying asset: exchange rate (currency pair).
- Premium payment typically in domestic currency.

How Option is Quoted in FX Options

In FX options, traders often summarize the volatility smile by quoting:

- **ATM volatility**: implied volatility of an option whose strike equals the forward exchange rate.
- **Risk Reversal (RR)**: measures skew of the implied volatility smile.
- **Butterfly (BF)**: measures smile curvature (fat-tailedness).

Quoting via ATM, RR, BF provides a compact way to negotiate, price, and hedge FX options.

FX Market Conventions: Spot

- FX spot rate: exchange rate at which two currencies can be exchanged immediately.
- In practice, FX spot transactions settle in 1 or 2 days ($T+1$ or $T+2$).
- This delay is called the **spot-lag**.

For simplicity, we ignore the spot-lag in this course.

FX Market Conventions: Moneyness

In FX options, the strike price K is expressed as an exchange rate.

Moneyness is determined by comparing strike K to forward price F:

$$F = Se^{(r_d - r_f)T},$$

where S is spot price and T is maturity.

- At-the-Money (ATM): $K = F$.
- In-the-Money (ITM): Call $K < F$, Put $K > F$.
- Out-of-the-Money (OTM): Call $K > F$, Put $K < F$.

FX Market Conventions: ATMVOL

ATMVOL refers to the implied volatility of an ATM option in FX markets.

Two common definitions:

- ATM Forward (ATM-FWD)
- ATM Delta-Neutral Straddle (ATM-DNS)

ATM Forward strike is set to the forward price:

$$K_{ATM} = F = Se^{\int_0^t (r_d(s) - r_f(s))ds}.$$

Since strike matches forward price, call and put have the same price (put-call parity).

FX Market Conventions: Delta

Delta measures sensitivity of an option price to changes in the underlying asset.

- Call options: Delta ranges from 0 to $+1$.
- Put options: Delta ranges from 0 to -1.

Key principle: an option can never react more strongly than owning the asset itself.

Delta for Black-Scholes Price (CALL)

The Black–Scholes price for a European FX call option is:

$$Call_{BS} = Se^{-r_f T} N(d_+) - Ke^{-r_d T} N(d_-),$$

with

$$d_\pm = \frac{\ln(F/K) \pm \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}.$$

where:

- r_f: foreign risk-free rate,
- r_d: domestic risk-free rate,
- $F = Se^{(r_d - r_f)T}$: forward exchange rate.

The call delta is:

$$\Delta_{call}(\sigma, K, T, r_d, r_f) = \frac{\partial Call_{BS}}{\partial S} = e^{-r_f T} N(d_+).$$

Delta (PUT)

The Black–Scholes price for a European FX put option is:

$$Put_{BS} = Ke^{-r_d T} N(-d_-) - Se^{-r_f T} N(-d_+).$$

The put delta is:

$$\Delta_{put}(\sigma, K, T, r_d, r_f) = \frac{\partial Put_{BS}}{\partial S} = -e^{-r_f T} N(-d_+).$$

Forward delta (derivative w.r.t. forward price F) is:

$$\Delta_{call} = N(d_+), \qquad \Delta_{put} = -N(-d_+).$$

ATM Delta-Neutral Straddle (ATM-DNS)

A straddle consists of:

- Buying one call and one put,
- Same strike K and maturity T.

The ATM-DNS strike is defined by zero net delta:

$$\Delta_{call}(K_{DNS}) + \Delta_{put}(K_{DNS}) = 0.$$

Solving the delta-neutral condition gives:

$$K_{DNS} = Fe^{\frac{1}{2}\sigma_{ATM}^2 T}.$$

FX Market Convention — Our Use Case

In practice, FX markets often use:

- ATM forward convention,
- Forward delta instead of spot delta.

Forward delta ranges:

$$\Delta_{call} = N(d_+), \qquad \Delta_{put} = -N(-d_+).$$

Using Delta
Delta is fundamental in FX options trading:
- First risk to hedge,
- Standardized measure of moneyness.

FX options are quoted by delta levels Δ, e.g.

$$25\Delta \text{ Call}.$$

The strike K must be found by solving:

$$0.25 = \Delta_{call} = N(d_+), \qquad d_+ = \frac{\ln(F/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}.$$

Market Strangle
A market strangle is:
- Buy one OTM call (higher strike),
- Buy one OTM put (lower strike).

It hedges large price moves in either direction.
Market Strangle Volatility Spread
In FX, strangles are quoted as a volatility spread relative to ATM:

$$\sigma^{\delta}_{ms}.$$

The OTM volatilities satisfy:

$$\frac{\sigma_c + \sigma_p}{2} = \sigma_{ATM} + \sigma^{\delta}_{ms},$$

where:
- σ_c: implied vol of OTM call,
- σ_p: implied vol of OTM put,
- σ_{ATM}: ATM implied vol.

Pricing the Strangle
Under Black–Scholes:

$$\text{Strangle price} = C(K_c, \sigma_c) + P(K_p, \sigma_p).$$

Market strangles constrain the curvature of the volatility smile:
- Expensive strangles $\rightarrow$ stronger convexity,
- Cheap strangles $\rightarrow$ flatter smile.

The strikes are determined by delta conditions:

$$\Delta_{call}(K_c, \sigma_{ATM} + \sigma_{\Delta_{ms}}) = \Delta,$$

$$\Delta_{put}(K_p, \sigma_{ATM} + \sigma_{\Delta_{ms}}) = -\Delta.$$

Link Between Market Strangle and Volatility Smile
- In a market strangle, traders rely on implied volatility to generate profits.
- The implied volatility curve follows a volatility smile:
  - Out-of-the-money (OTM) options have higher implied volatility,
  - Hence OTM calls and puts are more expensive.
- If the volatility smile is steep, strangles become more costly.
- If the smile is flatter, strangles are cheaper but profit potential is smaller.

Market Role of Strangles
- Constrains the volatility smile and surface shape.
- Measures smile convexity:

$$\text{greater strangle spread} \;\Rightarrow\; \text{more convex smile}.$$

-
Smile Strangle and Risk Reversal
Smile Strangle (Butterfly)
The smile strangle BF_Δ (butterfly spread) measures convexity:

$$BF_\Delta = \frac{1}{2}\left(\sigma_{\Delta_c} + \sigma_{\Delta_p}\right) - \sigma_{ATM}.$$

- Δ_c: delta level of the call option
- Δ_p: delta level of the put option
- Example: 25Δ smile strangle averages implied vols of 25Δ call/put minus ATM.

Risk Reversal
The risk reversal RR_Δ measures asymmetry (skewness):

$$RR_\Delta = \sigma_{\Delta_c} - \sigma_{\Delta_p}.$$

- Shows difference between implied vol of OTM calls and OTM puts.
- If $RR_\Delta > 0$, calls are more expensive (bullish skew).
- If $RR_\Delta < 0$, puts are more expensive (bearish skew).

The FX Volatility Marking Problems
In FX markets, implied volatility is quoted at specific delta levels rather than fixed strikes.
Given ATM volatility σ_{ATM}, smile strangles, and risk reversals, we solve:

$$\begin{cases} BF_\Delta = \frac{1}{2}(\sigma_{\Delta_c} + \sigma_{\Delta_p}) - \sigma_{ATM}, \\ RR_\Delta = \sigma_{\Delta_c} - \sigma_{\Delta_p}. \end{cases}$$

This yields:

$$\sigma_{\Delta_c} = \sigma_{ATM} + BF_\Delta + \frac{1}{2}RR_\Delta, \qquad \sigma_{\Delta_p} = \sigma_{ATM} + BF_\Delta - \frac{1}{2}RR_\Delta.$$

Converting Delta Quotes to Strike-Based Inputs
Market quotes are given as (Δ_i, σ_i) pairs, but we need strike-volatility pairs (K_i, σ_i).
Convert using:

$$\Delta_{call}(K_c, \sigma) = \Delta, \qquad \Delta_{put}(K_p, \sigma) = -\Delta.$$

Then K_i is found by solving these equations via root-finding.
Market Data Considerations
- Risk reversal RR is directly quoted.
- Smile strangle BF is not explicitly quoted: market provides only *market strangle* data.
- Challenge: construct the full implied volatility surface from ATM, RR, and strangles.

Market Strangle, Risk Reversal, and Surface Construction
- Risk reversal is observable in the raw market.
- Smile strangle is not directly observable; only market strangles are given.
- Key steps in volatility surface construction:
  - Collect bid-ask spreads for strangles and risk reversals across maturities.
  - Interpolate missing smile strangle data using spline or arbitrage-free methods.
  - Apply advanced volatility models to fit observed quotes.
  - Ensure no-arbitrage consistency with risk-neutral pricing.

How About Diffusion Models?
So far, we built an implied volatility surface to match vanilla option quotes.
However, Black–Scholes assumes constant volatility and cannot reproduce the full smile across strikes and maturities.
Need for an Advanced Model
- We are model-less: no full stochastic process for the underlying.
- Exotic and path-dependent options require richer models.
- Any advanced model must calibrate to the implied volatility surface.

Local Volatility Model
The local volatility model is a natural extension of Black–Scholes:
- Volatility depends on both time and underlying price.
- Designed to fit the implied volatility surface exactly.
- Provides a powerful framework for pricing exotic options.

Lecture 6

Black-Scholes Model and Implied Volatility
- The Black-Scholes formula determines option prices based on a model that assumes constant volatility.
- However, market prices indicate that implied volatility varies across different strikes and maturities, leading to the concept of the implied volatility smile and implied volatility surface.
- This inconsistency suggests that we need refinements such as stochastic volatility models or local volatility models to better match observed market behavior.
- The implied volatility surface provides a more intuitive representation of market expectations, helping traders and analysts better assess option pricing dynamics.

Black-Scholes Term Structure Model
- Extension of Black-Scholes with time-dependent volatility:

$$\frac{dS}{S} = \mu(t)dt + \sigma(t)dW$$

- Distinction between instantaneous volatility $\sigma(t)$ and implied volatility $\sigma_{iv}(K,T)$ inferred from market prices
- $\sigma_{iv}(K,T)$ reflects market consensus across strikes K and maturities T.
- Local volatility $\sigma(S,t)$ directly drives the stochastic process.
- These coincide only when volatility is constant.
- Connection Through Total Variance

$$\int_0^T \sigma^2(u)\,du$$

$$\sigma^2_{iv}(T)T = \int_0^T \sigma^2(u)\,du$$

- Implied volatility summarizes cumulative instantaneous variance.
- Recovering $\sigma(S,t)$ from $\sigma_{iv}(K,T)$ is an inverse problem.
- Assume no volatility smile (no strike dependence), but ATM implied volatility varies with maturity.
- Market quotes $\sigma_{iv}(T_i)$ at discrete maturities.
- Piecewise-constant instantaneous volatility:

$$\sigma(t) = \sigma_i, \quad t \in [T_i, T_{i+1}]$$

$$\sigma^2_{iv}(T_{i+1})T_{i+1} = \sigma^2_{iv}(T_i)T_i + \sigma^2_i(T_{i+1} - T_i)$$

$$\sigma^2_i = \frac{\sigma^2_{ATM}(T_{i+1})T_{i+1} - \sigma^2_{ATM}(T_i)T_i}{T_{i+1} - T_i}$$

- Instantaneous variance equals slope of total variance.
- Guarantees match to ATM term structure.

Dependency on Strike Prices
- Strike dependence is more complex than maturity dependence.
- Goal: construct a diffusion consistent with full implied volatility surface.
- Approaches:
  - Stochastic volatility (Heston, SABR)
  - Jump diffusion
- Trade-offs:
  - Additional risk factors
  - Numerical complexity
  - Residual calibration errors

Local Volatility Model

$$\frac{dS}{S} = \mu(t)dt + \sigma(S,t)dW$$

- Volatility depends on both price and time.
- Exactly matches implied volatility surface.
- The way to compute $\sigma(S,t)$ and the uniqueness of solution is addressed by Dupire's Model.

Dupire's Equation: Extracting Local Volatility

$$\sigma^2(S,t) = \frac{\frac{\partial}{\partial T}\left(\sigma_{iv}^2 K^2 T\right)}{1 + K\frac{\partial}{\partial K}\left(\sigma_{iv}^2 K^2 T\right)}$$

- Requires differentiating the implied volatility surface.
- Numerator: time sensitivity of total variance.
- Denominator: strike sensitivity adjustment.
- Produces a local volatility surface consistent with market prices.

Local Volatility and Arbitrage

- Dupire formula:

$$\sigma^2(K,T) = \frac{2\,\frac{\partial C(K,T)}{\partial T}}{K^2\,\frac{\partial^2 C(K,T)}{\partial K^2}}$$

- Denominator(Breeden–Litzenberger result):

$$K^2\,\frac{\partial^2 C(K,T)}{\partial K^2} = K^2 p(K,T)$$

- Well-posedness requires:

$$p(K,T) \geq 0$$

No-Arbitrage Conditions on Call Prices

- Positivity:

$$C(K,T) \geq 0$$

- Monotonicity in strike:

$$\frac{\partial C}{\partial K} \leq 0$$

- Convexity in strike (butterfly arbitrage-free):

$$\frac{\partial^2 C}{\partial K^2} \geq 0$$

- Monotonicity in maturity (calendar arbitrage-free):

$$\frac{\partial C}{\partial T} \geq 0$$

Implications for Local Volatility

- Numerator $\partial C/\partial T$ must be non-negative.
- Denominator $K^2\partial^2 C/\partial K^2$ must be positive.
- Violations lead to:
  - Negative local variance
  - Explosive or undefined diffusion
- Hence: implied volatility surface must be arbitrage-free.

From Call Prices to Implied Volatility

- Market provides implied vol surface $\sigma_{iv}(K,T)$.
- Convert to call prices via Black–Scholes:

$$C(K,T) = BS(S_0, K, T, \sigma_{iv}(K,T))$$

- Differentiate numerically to compute:

$$\frac{\partial C}{\partial T}, \qquad \frac{\partial^2 C}{\partial K^2}$$

- Plug into Dupire formula.

Practical Implementation Steps

1. Construct arbitrage-free implied volatility surface.
2. Convert to smooth call price surface.
3. Compute derivatives w.r.t. K and T.
4. Apply Dupire formula.
5. Obtain local volatility surface $\sigma(S,t)$.

Numerical Challenges

- Differentiation amplifies noise.
- Sparse market data.
- Need stable interpolation across:
  - Strike dimension
  - Maturity dimension
- Often use:
  - Spline smoothing
  - Parametric volatility models
  - Arbitrage-free parameterizations

Economic Interpretation

- Local volatility matches today's smile exactly.
- It encodes market-implied distribution of future prices.
- However, future smile dynamics are deterministic.

Limitations of Local Volatility

- Does not reproduce realistic smile dynamics.
- Forward-start options often mispriced.
- Volatility-of-volatility effects absent.
- Motivates stochastic volatility extensions.

Big Picture

- Fokker–Planck links SDEs to density evolution.
- Breeden–Litzenberger links density to option prices.
- Dupire links option prices to local volatility.
- Full chain: Market Smile $\Rightarrow$ $C(K,T)$ $\Rightarrow$ $p(K,T)$ $\Rightarrow$ $\sigma(S,t)$

Convexity Condition of Call Prices

- For any no-arbitrage call price function, convexity in strike is required:

$$\frac{\partial^2 C(K,T)}{\partial K^2} = p(K,T) \geq 0$$

- If convexity is violated at some K:

$$p(K,T) = \frac{\partial^2 C(K,T)}{\partial K^2} < 0$$

- Negative density implies arbitrage.
- This is called strike (butterfly) arbitrage.
- In practice, non-convexity must be corrected.

How Local Volatility Handles Arbitrage Issues

- Local volatility:

$$\sigma(K,T) = \sqrt{\frac{2\,\frac{\partial C(K,T)}{\partial T}}{K^2\,\frac{\partial^2 C(K,T)}{\partial K^2}}}$$

- If $\frac{\partial^2 C}{\partial K^2} < 0$, σ becomes complex.
- Practical fix: floor the denominator ($\varepsilon > 0$ small threshold).

$$K^2 \max\left(\frac{\partial^2 C}{\partial K^2}, \varepsilon\right)$$

Role of the Numerator

- Numerator: $2\,\frac{\partial C(K,T)}{\partial T}$ (dual theta).
- Expected: $\frac{\partial C(K,T)}{\partial T} \geq 0$
- Violation $\Rightarrow$ calendar arbitrage.
- Numerator often floored: $2\max\left(\frac{\partial C(K,T)}{\partial T}, \varepsilon\right)$

Local Volatility – General Case (Non-zero Drift)

$$\sigma(K,T) = \sqrt{\frac{2\left[\frac{\partial C}{\partial T} + (r_d - r_f)K\frac{\partial C}{\partial K} + r_f C\right]}{K^2\,\frac{\partial^2 C}{\partial K^2}}}$$

- Only numerator changes vs driftless case.
- Depends on domestic r_d and foreign r_f rates (FX case).

Derivation Outline

- SDE:

$$dS_t = (r_d - r_f)S_t dt + \sigma(S_t,t)S_t dW_t$$

- Fokker-Planck equation:

$$\frac{\partial p}{\partial t} = -\frac{\partial}{\partial s}[(r_d - r_f)sp] + \frac{1}{2}\frac{\partial^2}{\partial s^2}[s^2\sigma^2 p]$$

- Using $p(K,T) = \frac{\partial^2 C}{\partial K^2}$,
- Integrate and solve for $\sigma(K,T)$.

Local Volatility in Terms of Implied Volatility

- Since FX markets quote implied vols, rewrite formula using $\sigma_{iv}(K,T)$.

$$\sigma(K,T) = \sqrt{\frac{\sigma_{iv}^2 + 2T\sigma_{iv}\frac{\partial\sigma_{iv}}{\partial T} + 2(r_d - r_f)KT\sigma_{iv}\frac{\partial\sigma_{iv}}{\partial K}}{\left(1 + Kd_1\sqrt{T}\frac{\partial\sigma_{iv}}{\partial K}\right)^2 + K^2 T\sigma_{iv}\left(\frac{\partial^2\sigma_{iv}}{\partial K^2} - d_1\sqrt{T}\left(\frac{\partial\sigma_{iv}}{\partial K}\right)^2\right)}}$$

$$d_1 = \frac{\ln(F/K) + \frac{1}{2}\sigma_{iv}^2 T}{\sigma_{iv}\sqrt{T}}$$

Finite Difference Approximation of Derivatives

First-order derivatives

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \qquad f'(x) \approx \frac{f(x) - f(x-h)}{h} \qquad f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Second-order derivative

$$f''(x) \approx \frac{f(x+h) + f(x-h) - 2f(x)}{h^2}$$

- Large $h \Rightarrow$ truncation error.
- Small $h \Rightarrow$ round-off error.

What Does the Local Volatility Surface Look Like?

- Jumps at pillars mainly due to discontinuities of $\partial\sigma_{iv}/\partial T$.
- Smooth along strike due to C^2 spline interpolation.

Lecture 7

Martingality of Discounted Derivative Price

- Key idea: Risk-neutral pricing determines the fair value of derivatives.
- Under the risk-neutral measure $\mathbb{Q}$, the discounted derivative price is a martingale.
- This means the expected discounted future price equals the current discounted price.

$$\frac{V_t}{B_t} = \mathbb{E}_{\mathbb{Q}}\left[\frac{V_T}{B_T}\right]$$

- $\mathbb{Q}$ = risk-neutral probability measure.
- B_t = value of the risk-free investment (money market account).
- V_t = price of the derivative at time t.
- The result holds whether B_t is deterministic or stochastic.

Why Do We Discount B_t?

- If interest rates are constant:

$$B_t = e^{rt}$$

- Money today is worth more than money in the future due to interest rates.
- Discounting adjusts prices for the time value of money.

Martingale Property

- A martingale is a process whose conditional expectation equals its current value.
- Ensures absence of arbitrage opportunities.

Implication for Derivative Pricing

- Pricing derivatives reduces to computing expectations.
- The fair value equals the expected discounted payoff under $\mathbb{Q}$.

$$\frac{V_t}{B_t} = \mathbb{E}_{\mathbb{Q}}[h(X)]$$

- X represents the random factors affecting the payoff.
- $h(X)$ is the payoff function of the derivative.

Monte Carlo Simulation for Expectations

- Monte Carlo methods approximate expectations numerically.
- Useful when:
  - Analytical solutions do not exist.
  - The underlying process is complex.
  - The problem involves many random variables.
- Handles exotic and path-dependent derivatives.
- Scales well in high-dimensional problems.

Law of Large Numbers (LLN)

- Let $X_1, X_2, \ldots, X_n$ be i.i.d. random variables.

$$\mathbb{E}[X_i] = m, \qquad \mathrm{Var}(X_i) = \sigma^2 < \infty$$

- Consider

$$\mu = \mathbb{E}[h(X)]$$

- Define the sample mean estimator:

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} h(X_i)$$

- LLN states:

$$P(|\hat{\mu}_n - \mu| \leq \epsilon) \to 1 \quad \text{as } n \to \infty$$

LLN and Monte Carlo

- The sample average approximates the true expectation.
- The estimator is unbiased:

$$\mathbb{E}[\hat{\mu}_n] = \frac{1}{n}\mathbb{E}\left[\sum_{i=1}^{n} h(X_i)\right] = \mu$$

- Monte Carlo procedure:
  - Generate many independent samples.
  - Compute the payoff for each sample.
  - Average the results.

Monte Carlo Algorithm

1. Initialize $s = 0$.
2. For $i = 1, \ldots, n$:
   - Generate X_i.
   - Compute $h_i = h(X_i)$.
   - Update $s = s + h_i$.
3. Return

$$\hat{\mu}_n = \frac{s}{n}$$

Monte Carlo as an Integration Method

- Many integrals can be written as expectations.

$$\int_0^1 f(x)\,dx = \mathbb{E}[f(x)], \quad x \sim U(0,1)$$

- For a general interval $[a, b]$:

$$\mathbb{E}[f(x)] = \int_a^b f(x)p(x)dx = \frac{1}{b-a}\int_a^b f(x)dx$$

$$\int_a^b f(x)dx = (b-a)\mathbb{E}[f(x)], \quad x \sim U(a,b)$$

- Monte Carlo approximation:

$$\int_a^b f(x)dx \approx (b-a)\frac{1}{N}\sum_{i=1}^{N} f(x_i)$$

- Works well for high-dimensional integrals.
- Effective when analytical integration is difficult.

Estimating Probabilities with Monte Carlo

- A probability can be expressed as an expectation:

$$P(X \in A) = \mathbb{E}[I_A(X)]$$

- Indicator function:

$$I_A(X) = \begin{cases} 1 & X \in A \\ 0 & \text{otherwise} \end{cases}$$

- Monte Carlo estimator:

$$P(X \in A) \approx \frac{1}{n}\sum_{i=1}^{n} I_A(X_i)$$

Example: Estimating π

- Consider a unit circle inside a 2×2 square.
- A random point (x, y) lies inside the circle if:

$$x^2 + y^2 \leq 1$$

- The probability of falling inside the circle is: $P = \frac{\pi}{4}$
- Estimate π via simulation:

$$\pi \approx 4\frac{\text{points inside circle}}{\text{total points}}$$

- Monte Carlo works well for complex probability problems.
- Easily scales to high-dimensional settings.

How Large is the Error? — Central Limit Theorem

- Consider i.i.d. variates $\xi_i = h(X_i)$ with

$$\mathbb{E}[\xi_i] = \mu, \qquad \mathbb{V}[\xi_i] = \sigma^2$$

- Define the sample mean

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} h(X_i)$$

- As n increases, the normalized deviation

$$e_n := \frac{\hat{\mu}_n - \mu}{\sigma/\sqrt{n}}$$

- Converges in distribution to the standard normal distribution

$$e_n \Rightarrow \mathcal{N}(0,1)$$

- Thus the sample mean becomes approximately normally distributed regardless of the original distribution.

Error Estimation for Monte Carlo

- By the CLT:

$$\hat{\mu}_n \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$$

- The uncertainty of $\hat{\mu}_n$ is measured by its standard deviation

$$\frac{\sigma}{\sqrt{n}}$$

- Since σ is usually unknown, we estimate it using the sample variance

$$\hat{\sigma}_n = \sqrt{\frac{1}{n}\sum_{i=1}^{n} h_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n} h_i\right)^2}$$

- The Monte Carlo standard error becomes

$$\epsilon_n = \frac{\hat{\sigma}_n}{\sqrt{n}}$$

Monte Carlo Convergence

- Monte Carlo error decreases at rate

$$O\left(\frac{1}{\sqrt{n}}\right)$$

- To reduce the error by a factor of 10, the number of samples must increase by a factor of 100.
- The standard error is the standard deviation of the estimator $\hat{\mu}_n$.
- For a normal distribution $\mathcal{N}(\mu, \sigma^2)$:
  - 68.27% of values lie within 1σ
  - 95% within 2σ
  - 99.7% within 3σ

Monte Carlo Confidence Interval

- Consider the normalized estimator

$$Z_n = \frac{\hat{\mu}_n - \mu}{\epsilon_n}$$

- Approximately follows

$$Z_n \approx \mathcal{N}(0, 1)$$

- A two-sided confidence interval with level $1 - \delta$ satisfies

$$P\left(-N^{-1}(1 - \delta/2) \leq Z_n \leq N^{-1}(1 - \delta/2)\right) = 1 - \delta$$

- Rearranging gives the confidence interval

$$\left[\hat{\mu}_n - N^{-1}(1 - \delta/2)\epsilon_n,\ \hat{\mu}_n + N^{-1}(1 - \delta/2)\epsilon_n\right]$$

Confidence Interval Interpretation

- The interval covers the true value μ with probability $1 - \delta$.
- The interval is random because it depends on $\hat{\mu}_n$ and ϵ_n.
- When ϵ_n is estimated using $\hat{\sigma}_n/\sqrt{n}$, the distribution becomes Student's t.

Rule of Thumb

- For $\delta = 0.05$:

$$N^{-1}(1 - \delta/2) \approx 1.96$$

- Thus a 95% confidence interval is

$$\hat{\mu}_n \pm 1.96\,\epsilon_n$$

- Other common levels:
  - 90%: 1.645
  - 99%: 2.576

Generation of Random Process X_i

- The only non-trivial part of Monte Carlo simulation is generating X_i.
- In the expectation $\mathbb{E}[h(X_i)]$:
  - h is the payoff function.
  - X_i represents the underlying asset state.
- The distribution of X_i may not be known in closed form.
- Under the Black–Scholes model:

$$\frac{dS}{S} = (r - q)dt + \sigma dW_t$$

- Under the local volatility model:

$$\frac{dS}{S} = (r - q)dt + \sigma(S, t)dW_t$$

Random Number Generators

- Random numbers from non-uniform distributions can be generated from uniform samples.
- Common methods:
  - CLT approximation
  - Box–Muller transform
  - Inverse transform sampling
- Computers are deterministic, so they produce pseudo-random numbers.
- True randomness requires external sources.
- Pseudo-random numbers are sufficient for most Monte Carlo applications.

Desired Properties of Random Numbers in QF

- Samples should mimic i.i.d. draws from the target distribution.
- Pseudo-random numbers work well for Monte Carlo integration.
- Reproducibility is important for debugging and validation.
- No PRNG is perfect, so multiple generators may be used for validation.

PRNG: General Principle

1. **State initialization:** internal seed variables define the generator state.
2. **State transition:** a deterministic recurrence updates the state.
3. **Number generation:** a transformation maps the state to a uniform random variable $U(0, 1)$.

- PRNGs generate sequences that appear random while remaining deterministic and reproducible.

PRNG: Middle-Square Method

- One of the earliest pseudo-random number algorithms (von Neumann).
- State variable: x_i
- Iteration rule: square the number and extract the middle digits.
- Example with seed $x_0 = 0.9876$

$$x_0 = 0.9876 \quad x_0^2 = 0.97535376 \quad x_1 = 0.5353 \quad x_1^2 = 0.28654609 \quad x_2 = 0.6546$$

Middle-Square Method: Issue

- A major drawback of the Middle-Square Method is its tendency to enter short periodic cycles or collapse to zero.
- This makes it unreliable for generating long sequences of random numbers.
- Example 1: With seed 0.9876, the sequence starts repeating from x_{46}, producing only four distinct values before cycling:

$$0.96 \to 0.16 \to 0.56 \to 0.36$$

- Example 2: With seed 0.5649, the sequence collapses to zero at x_{84} and remains stuck thereafter.

Congruential Generators

- Congruential generators update the state variable using the recurrence

$$m_{n+1} = (am_n + c) \bmod M$$

- Values are scaled to $[0, 1]$ by

$$x_n = \frac{m_n}{M - 1}$$

- To avoid collapse to a fixed point:
  - a and M must be co-prime
  - $c \neq 0$ prevents absorption at zero
- When $c = 0$, the generator becomes a **linear congruential generator (LCG)**.
- In this case the sequence remains non-trivial provided
  - a and M are co-prime
  - $m_0 \neq 0$
- These conditions ensure a long cycle before repetition.

Congruential Generators: Example

- For small M, the generator quickly enters a periodic cycle.
- Example:

$$M = 11, \quad a = 3, \quad c = 0, \quad m_0 = 3$$

- Generated sequence (period 5):

$$9, 5, 4, 1, 3, 9, 5, 4, 1, \ldots$$

- The maximum possible period is $M - 1$, but this is not guaranteed.
- With sufficiently large M, repetitions become negligible for practical use.
- Choice of parameters is critical.
- Example early IBM generator:

$$a = 65539, \quad M = 2^{31}, \quad m_0 = 1$$

- Later found to have poor statistical properties.
- Improved parameters:

$$a = 5^{17}, \quad M = 2^{40}, \quad m_0 = 1$$

- Achieves period 2^{38}.

Congruential Generators: Extensions

1. **Rand0: Minimal Standard Generator**
   - LCG with

$$a = 16807, \quad M = 2^{31} - 1$$

   - Provides a long period and reasonable statistical properties.
2. **Rand1: Shuffled Enhancement**
   - Improves Rand0 using a shuffle table.
   - Previously generated values are stored and reused to reduce correlation.
   - Example:
     - Generated numbers: $17, 42, 8, 91, 35, 60, 14, \ldots$
     - Shuffle table initialized with: $[17, 42, 8, 91]$
     - New value 35 selects entry 2: output 42, replace by 35
     - New value 60 selects entry 4: output 91, replace by 60
     - New value 14 selects entry 1: output 17, replace by 14

Congruential Generators: Extensions (Rand2)

- Combines two independent LCGs to extend the period.

$$Y_n = (X_n + Z_n) \bmod M$$

- Helps avoid short cycles and improves distribution properties.
- Example sequences:

$$X_n : 12, 45, 31, 8, 27, \ldots$$

$$Z_n : 7, 19, 40, 22, 11, \ldots$$

$$Y_n = (X_n + Z_n) \bmod 50$$

$$Y_1 = 19, \quad Y_2 = 14, \quad Y_3 = 21, \quad Y_4 = 30$$

- Combined sequence:

$$19, 14, 21, 30, 38, \ldots$$

Mersenne Twister (Matsumoto & Nishimura, 1998)

- Designed to eliminate flaws of earlier generators.
- Provides extremely long period:

$$2^{19937} - 1$$

- Uses a twisting transformation to mix bits and improve randomness.

$$x_{k+n} = x_{k+m} \oplus ((x_k^{upper} \parallel x_{k+1}^{lower})A)$$

- $\oplus$ denotes bitwise XOR.
- $\parallel$ denotes concatenation of bits.
- A is a fixed transformation matrix.

Mersenne Twister (MT19937) and Equidistribution

- Equidistribution means generated vectors fill the k-dimensional space evenly.

$$(u_1, u_2), (u_3, u_4), \ldots$$

- In general: $(u_i, u_{i+1}, \ldots, u_{i+k-1})$

- MT19937 achieves 623-dimensional equidistribution.
- Much better than LCGs which typically achieve only a few dimensions.

LCGs are Weak in High Dimensions

- Successive k-dimensional points generated by LCGs lie on parallel hyperplanes.
- Marsaglia's theorem:

$$\text{At most } (k!)^{1/k} \text{ hyperplanes}$$

- Example:
  - $k = 3$: about 2.3 planes
  - $k = 6$: about 2.99 planes
- Thus patterns emerge in higher dimensions.
- This is undesirable for Monte Carlo simulation.

Uniform to Normal via Central Limit Theorem

- Transform uniform samples $U(0,1)$ into normal variables.
- Using CLT:

$$\frac{\hat{\mu}_n - \mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0,1)$$

- Variance of uniform distribution:

$$\int_0^1 (x-0.5)^2 dx = \frac{1}{12}$$

- Generate 12 uniform samples u_i and compute

$$m_k = \sum_{i=1}^{12} u_i - 6$$

- The sum of uniform variables approximates a normal distribution.
- Subtracting 6 centers the distribution around zero.

Uniform To Normal Distribution: Box–Muller Method

- The CLT-based method for generating normal variables lacks precision, especially in the tails.
- The Box–Muller method is a simple and efficient algorithm to transform uniform random variables into normally distributed variables.
- It uses a property of the bivariate normal distribution.
- If Z_1, Z_2 are independent standard normal variables, then

$$R = \sqrt{Z_1^2 + Z_2^2}$$

- follows an exponential distribution with mean 2:

$$P(R \le r^2) = 1 - e^{-r^2/2}$$

- Conditional on $R = r$, the point (Z_1, Z_2) is uniformly distributed on the circle of radius r.
- The joint PDF of (Z_1, Z_2) is

$$f_{Z_1,Z_2}(z_1, z_2) = \frac{1}{2\pi} e^{-\frac{1}{2}(z_1^2+z_2^2)}$$

Derivations

- Represent the variables in polar coordinates

$$Z_1 = R\cos\theta, \qquad Z_2 = R\sin\theta$$

- The Jacobian of the transformation is R.
- The joint PDF of (R, θ) becomes

$$f_{R,\theta} = \frac{1}{2\pi} e^{-r^2/2} r, \qquad 0 \le r < \infty, \quad 0 \le \theta \le 2\pi$$

- Thus
  - θ is uniform on $[0, 2\pi]$
  - $f_R(r) = re^{-r^2/2}$

Probability Integral Transform

- If $U \sim U(0,1)$ and F_X is the CDF of X, then

$$X = F_X^{-1}(U)$$

- produces the random variable with distribution F_X.

Derivation

- If $U \sim U(0,1)$

$$P(U \le y) = y$$

- Using the exponential CDF

$$1 - e^{-r^2/2} = U$$

- solving gives

$$r = \sqrt{-2\ln U}$$

Box–Muller Algorithm

1. Generate $U_1, U_2 \sim U(0,1)$
2. $R \leftarrow -2\log(U_1)$
3. $V \leftarrow 2\pi U_2$
4. Compute

$$Z_1 = \sqrt{R}\cos V, \qquad Z_2 = \sqrt{R}\sin V$$

Acceptance–Rejection Method

- A general method to sample from a target distribution $f(x)$ using an easier proposal distribution $g(x)$.
- Suppose

$$f(x) \le c\, g(x)$$

- Procedure:
  - Sample x from $g(x)$
  - Accept with probability $\frac{f(x)}{cg(x)}$
  - Otherwise reject and repeat
- Practically: draw $u \sim U(0,1)$ and accept if

$$u \le \frac{f(x)}{cg(x)}$$

Why Acceptance–Rejection Works

- Let Y denote the accepted samples.
- For any set A

$$P(Y \in A) = \frac{P(x \in A, u \le f(x)/cg(x))}{P(u \le f(x)/cg(x))}$$

- The denominator equals

$$P\left(u \le \frac{f(x)}{cg(x)}\right) = \int \frac{f(x)}{cg(x)} g(x)dx = \frac{1}{c}$$

- Therefore

$$P(Y \in A) = \int_A f(x)\, dx$$

- Thus Y has density $f(x)$.
- Acceptance rate is $1/c$.

Acceptance–Rejection Algorithm

1. repeat
2. Generate $x \sim g(x)$
3. Generate $u \sim U(0,1)$
4. until $u \le f(x)/cg(x)$
5. return x

Polar Rejection Method

- A modified version of Box–Muller that avoids computing sin and cos.
- Uses acceptance–rejection to generate points uniformly inside the unit disk.

1. repeat
2. generate $u_1, u_2 \sim U(0,1)$
3. transform

$$u_1 \leftarrow 2u_1 - 1, \qquad u_2 \leftarrow 2u_2 - 1$$

1. compute

$$x = u_1^2 + u_2^2$$

1. until $x \le 1$
2. compute

$$y = \sqrt{-2\ln x}$$

$$Z_1 = y\frac{u_1}{\sqrt{x}}, \qquad Z_2 = y\frac{u_2}{\sqrt{x}}$$

- Faster than Box–Muller despite the acceptance step.
- Acceptance probability is $\pi/4$.

Lecture 8

Inverse Transformation Method

- A fundamental approach for generating random samples from a target distribution transforms uniform random variables using the inverse of the cumulative distribution function (CDF).

Given a random variable X with CDF:

$$P(X \le x) = CDF(x)$$

The inverse transformation method sets

$$X = CDF^{-1}(u), \quad u \sim U(0,1)$$

- To generate a sample x, draw $u \sim U(0,1)$ and compute

$$x = CDF^{-1}(u)$$

- This method is used, for example, to generate exponential variables in the Box–Muller method.

Inverse Transformation Method: Derivation

- Express the probability for X in terms of U:

$$P(X \le x) = P(CDF^{-1}(U) \le x) = P(U \le CDF(x))$$

- For $U \sim U(0,1)$ the CDF is

$$P(U \le u) = u, \quad u \in [0,1]$$

Therefore

$$P(X \le x) = P(U \le CDF(x)) = CDF(x)$$

Inverse Transformation Method — Considerations

- The inverse CDF is not always one-to-one.
- Situations may arise where:
  - many-to-one mappings occur (jumps in CDF)
  - one-to-many mappings occur (flat CDF)
- In such cases predefined conventions are required when applying inverse transforms.

Inverse Transform Method for Normal Distribution

- The normal CDF has no closed-form inverse.

Typical procedure:

- Use an analytical approximation for $N^{-1}(x)$ as an initial estimate.
- Apply a root-finding algorithm to refine the solution.

Efficiency considerations:

- This can be computationally expensive due to iterative methods.
- Alternatives such as the **Box–Muller transform** are commonly used.

Advantages:

- The normal CDF $N(x)$ is strictly increasing, preserving monotonicity.
- Only one uniform random number is required per normal sample.

Pricing European Options with Monte Carlo

Under the Black–Scholes model:

$$\frac{dS_t}{S_t} = r dt + \sigma dW_t$$

Since S_T has a closed-form solution:

$$S_T = S_0 e^{(r-q-\frac{1}{2}\sigma^2)T + \sigma W_T}$$

- Only the terminal value W_T needs to be simulated.
- This avoids simulating the entire Brownian path.

Properties of Brownian motion:

- $W_0 = 0$
- $W_t \sim \mathcal{N}(0, t)$

Thus

$$W_T = \sqrt{T} Z, \quad Z \sim \mathcal{N}(0, 1)$$

Models without Closed-Form Solutions

- Black–Scholes is an exception; most SDEs lack closed-form solutions.

Example: local volatility model

$$dS = S(r-q)dt + S\sigma(S,t)dW_t$$

- Since $\sigma(S,t)$ depends on the state, no analytical solution exists.

General SDE:

$$dX(t) = a(X(t))dt + b(X(t))dW(t)$$

Integrating:

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} a(X(u))du + \int_{t_i}^{t_{i+1}} b(X(u))dW_u$$

- Numerical methods such as Euler–Maruyama are required.

Euler Discretization Scheme

- For time step Δt:

$$\int_t^{t+\Delta t} a(X(u))du = a(X(t))\Delta t + O(\Delta t^2)$$

$$\int_t^{t+\Delta t} b(X(u))dW_u = b(X(t))\Delta W = b(X(t))\sqrt{\Delta t} Z + O(\Delta t)$$

where $Z \sim N(0,1)$.

Step update:

$$X(t+\Delta t) = X(t) + a(X(t))\Delta t + b(X(t))\sqrt{\Delta t} Z$$

- Drift truncation error: $O(\Delta t)$
- Diffusion truncation error: $O(\sqrt{\Delta t})$

Overall convergence order:

$$O(\sqrt{\Delta t})$$

Euler Scheme for Black–Scholes

$$S_{t+\Delta t} = S_t(1 + \mu\Delta t + \sigma\Delta W_t)$$

Example simulation:

- $S_0 = 10$
- $\sigma = 30\%$
- $\mu = 10\%$
- horizon = 5 years

Change of State Variable

- The state variable is the variable tracked during Monte Carlo simulation.
- It does not need to be the asset price itself.

Using log-transformed state variable:

$$X_t = \log S_t$$

Ito expansion gives

$$dX_t = \left(\mu - \frac{1}{2}\sigma^2\right) dt + \sigma dW_t$$

Euler update:

$$X_{t+\Delta t} = X_t + \left(\mu - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\Delta W_t$$

Reconstruct the price:

$$S_{t+\Delta t} = e^{X_{t+\Delta t}} = S_t e^{(\mu - \frac{1}{2}\sigma^2)\Delta t + \sigma\Delta W_t}$$

- This follows from the linear diffusion of X.

Local Volatility Model Diffusion

Using the same state variable X:

$$dX_t = \left(\mu - \frac{1}{2}\sigma(S,t)^2\right) dt + \sigma(S,t)dW_t$$

Euler discretization:

$$X_{t+\Delta t} = X_t + \left(\mu - \frac{1}{2}\sigma(X,t)^2\right)\Delta t + \sigma(X,t)\Delta W_t$$

- Unlike geometric Brownian motion, no closed-form solution exists.
- Discretization introduces error of order

$$O(\sqrt{\Delta t})$$

Milstein Discretization Scheme

- Higher–order convergence in discretization can be achieved using the Milstein scheme.
- This scheme is particularly popular in Monte Carlo simulations for improving accuracy.
- The goal is to achieve $O(\Delta t)$ convergence. Since the drift term is already $O(\Delta t)$, we only need to refine the diffusion term.
- In the Euler scheme, the diffusion term is approximated using $b(X(t))$ over the period $t \to t+\Delta t$, which introduces truncation error.
- The Milstein scheme corrects this by incorporating an additional term to capture the stochastic integral more accurately.

Milstein Discretization Derivation

Consider the process

$$dX(t) = a(X(t))dt + b(X(t))dW(t)$$

The Euler discretization reads

$$X(t+\Delta t) = X(t) + a(X(t))\Delta t + b(X(t))\Delta W_t$$

- The issue is that this assumes $b(X_t)$ is constant over $[t, t+\Delta t]$.
- For higher accuracy we account for the fact that $b(X_t)$ is also stochastic.

Using Itô's lemma and $(dX)^2 = b^2 dt$:

$$db(X(t)) = \frac{\partial b}{\partial X} dX + \frac{1}{2}\frac{\partial^2 b}{\partial X^2} dX^2$$

$$= b'(a\,dt + b\,dW_t) + \frac{1}{2} b'' b^2 dt$$

$$= (ab' + \tfrac{1}{2} b^2 b'')dt + b(X(t))b'(X(t))dW_t$$

Approximating the Diffusion Term

$$b(X(u)) = b(X(t)) + \int_t^u b(X(s))b'(X(s))dW_s + \int_t^u c(X(s))ds$$

- The integral $\int_t^u c(X(s))ds$ is of order $O(\Delta t)$ and can be omitted.

Thus

$$b(X(u)) \approx b(X(t)) + \int_t^u b(X(s))b'(X(s))dW_s$$

Approximating the stochastic integral:

$$\int_t^u b(X(u))b'(X(u))dW_u = b(X(t))b'(X(t))(W_u - W_t) + O(u-t)$$

Hence

$$b(X(u)) = b(X(t)) + b(X(t))b'(X(t))(W_u - W_t) + O(u-t)$$

Milstein Correction

Recall

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} a(X(u))du + \int_{t_i}^{t_{i+1}} b(X(u))dW_u$$

Then

$$\int_{t_i}^{t_{i+1}} b(X(u))dW_u = b(X(t))(W_{t+\Delta t} - W_t) + \frac{1}{2} b(X(t))b'(X(t))((W_{t+\Delta t} - W_t)^2 - \Delta t)$$

Equivalently,

$$\int_{t_i}^{t_{i+1}} b(X(u))dW_u = b(X(t))\sqrt{\Delta t}Z + \frac{1}{2}b(X(t))b'(X(t))\Delta t(Z^2-1)$$

where $Z \sim N(0,1)$.

Itô Isometry

Using Itô's formula for W_u^2:

$$d(W_u^2) = 2W_u dW_u + du$$

Thus

$$W_u dW_u = \frac{1}{2}d(W_u^2) - \frac{1}{2}du$$

Integrating from t to $t+\Delta t$:

$$\int_t^{t+\Delta t} W_u dW_u = \frac{1}{2}(W_{t+\Delta t}^2 - W_t^2) - \frac{1}{2}\Delta t$$

Milstein Scheme for Local Volatility

In a local volatility model

$$b(X,t) = \sigma(X,t)$$

and

$$b'(X) = \frac{\partial \sigma(X,t)}{\partial X}$$

Euler scheme:

$$X_{t+\Delta t} \approx X_t + \left(\mu - \frac{\sigma(X,t)^2}{2}\right)\Delta t + \sigma(X,t)\Delta W_t$$

Milstein correction:

$$\frac{1}{2}\sigma(X,t)\frac{\partial \sigma(X,t)}{\partial X}(\Delta W_t^2 - \Delta t)$$

Final scheme:

$$X_{t+\Delta t} = X_t + \left(\mu - \frac{\sigma(X,t)^2}{2}\right)\Delta t + \sigma(X,t)\Delta W_t + \frac{\sigma(X,t)}{2}\frac{\partial \sigma(X,t)}{\partial X}(\Delta W_t^2 - \Delta t)$$

Example: Milstein with State Variable S

Consider the local volatility SDE

$$dS_t = \mu S_t dt + \sigma(S_t,t)S_t dW_t$$

Milstein scheme:

$$S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma(S_t,t)S_t\Delta W_t + \frac{1}{2}b(S_t)b'(S_t)(\Delta W_t^2 - \Delta t)$$

Here

$$b(S_t) = \sigma(S_t,t)S_t$$

$$b'(S_t) = \sigma(S_t,t) + S_t\frac{\partial \sigma(S_t,t)}{\partial S}$$

Thus

$$\frac{1}{2}\sigma(S_t,t)S_t\left(\sigma(S_t,t) + S_t\frac{\partial \sigma(S_t,t)}{\partial S}\right)(\Delta W_t^2 - \Delta t)$$

If σ is constant (Black–Scholes):

$$S_{t+\Delta t} = S_t\left(1 + \mu\Delta t + \sigma\Delta W_t + \frac{\sigma^2}{2}(\Delta W_t^2 - \Delta t)\right)$$

Monte Carlo Method — Pros and Cons

Pros:

- Easy to implement.
- Highly extensible to complex models and path dependencies.
- Performs well in high-dimensional problems.

Cons:

- Monte Carlo noise requires many simulations for accuracy.
- Slower than PDE or tree methods in low dimensions.
- Limited efficiency for American options (requires LSMC).

Two-Factor Monte Carlo: Correlated Brownian Motions

When simulating multiple factors, correlated Brownian motions must be constructed.

Correlation of increments:

$$\rho = \frac{\mathrm{Cov}(\Delta W_1, \Delta W_2)}{\sqrt{\mathrm{Var}(\Delta W_1)\mathrm{Var}(\Delta W_2)}} = \frac{E(\Delta W_1 \Delta W_2)}{dt}$$

Since

$$E(\Delta W_i) = 0, \quad Var(\Delta W_i) = dt$$

we obtain

$$\rho = \frac{E(\Delta W_1 \Delta W_2)}{dt}$$

Let

$$\Delta W_1 = \xi_1\sqrt{dt}, \qquad \Delta W_2 = \xi_2\sqrt{dt}$$

with

$$E(\xi_1\xi_2) = \rho$$

Correlation of the Increments of Two Brownian Motions

- Represent the correlation matrix as

$$\mathbf{C} = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$$

Random samples for ξ_1 and ξ_2 (to generate correlated Brownian increments):

$$\hat{\xi}_1 = [\hat{\xi}_{1,0}, \hat{\xi}_{1,1}, \ldots, \hat{\xi}_{1,n}]$$

$$\hat{\xi}_2 = [\hat{\xi}_{2,0}, \hat{\xi}_{2,1}, \ldots, \hat{\xi}_{2,n}]$$

In terms of expectations:

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix} = \begin{pmatrix} \hat{\xi}_1\cdot\hat{\xi}_1 & \hat{\xi}_1\cdot\hat{\xi}_2 \\ \hat{\xi}_2\cdot\hat{\xi}_1 & \hat{\xi}_2\cdot\hat{\xi}_2 \end{pmatrix} = \begin{pmatrix} n & n\rho \\ n\rho & n \end{pmatrix} = n\mathbf{C}$$

Correlating Standard Normal Vectors

- For $-1 < \rho < 1$, the matrix $\mathbf{C}$ is invertible.
- Decompose it using Cholesky factorization:

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

We have

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix} = n\mathbf{L}\mathbf{L}^\top$$

Thus

$$\mathbf{L}^{-1}\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix}(\mathbf{L}^\top)^{-1} = n\mathbf{I}_2$$

where $\mathbf{I}_2$ is the 2×2 identity matrix.

Generating Correlated Normals

Let

$$\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix} = \mathbf{L}^{-1}\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}$$

Then ζ_1, ζ_2 are uncorrelated since

$$\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix}\begin{pmatrix} \zeta_1^\top & \zeta_2^\top \end{pmatrix} = n\mathbf{I}$$

Therefore, if we generate independent standard normals ζ_1, ζ_2, we obtain correlated normals by

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix} = \mathbf{L}\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix}$$

Cholesky Decomposition

- If a matrix is symmetric and positive definite, it admits a Cholesky decomposition:

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

where $\mathbf{L}$ is lower triangular.

- For correlation matrices this holds when $-1 < \rho < 1$.

To verify positive definiteness:

$$\mathbf{x}^\top\mathbf{C}\mathbf{x} = \frac{1}{n}\mathbf{x}^\top\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix}\begin{pmatrix} \hat{\xi}_1^\top & \hat{\xi}_2^\top \end{pmatrix}\mathbf{x} = \frac{\|y\|}{n} \geq 0$$

where

$$y = [\hat{\xi}_1, \hat{\xi}_2]\mathbf{x}$$

Thus $\mathbf{C}$ is semi-positive definite and strictly positive definite when $\rho \neq \pm 1$.

Generating Two Correlated Brownian Motions

Procedure:

1. Generate independent standard normals ζ_1, ζ_2.
2. Compute Cholesky decomposition $\mathbf{C} = \mathbf{L}\mathbf{L}^\top$.
3. Generate correlated normals

$$\begin{pmatrix} \hat{\xi}_1 \\ \hat{\xi}_2 \end{pmatrix} = \mathbf{L}\begin{pmatrix} \zeta_1 \\ \zeta_2 \end{pmatrix}$$

- The same procedure extends to m correlated random variables.

Cholesky Decomposition Algorithm
Example for a 3×3 symmetric matrix

$$\mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & C_{13} \\ C_{12} & C_{22} & C_{23} \\ C_{13} & C_{23} & C_{33} \end{pmatrix}$$

Factorization

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

$$\mathbf{L} = \begin{pmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{pmatrix}$$

Algorithm:

1. Compute L_{11} from the top-left element.
2. Move down the first column to compute L_{21}, L_{31}.
3. Proceed diagonally to compute L_{22}.
4. Continue column by column.

Python implementation:

$$L = \texttt{np.linalg.cholesky(C)}$$

Example of Correlated Brownian Motions
- Using the above procedure we can generate Brownian motions with different correlation coefficients ρ.

Lecture 9

Generic Monte Carlo Framework
- Similar to the Binomial Tree framework, the Monte Carlo (MC) framework can be structured in a modular way to provide a generic and flexible approach to pricing financial derivatives.
- Unlike the tree-based approach, Monte Carlo methods offer greater flexibility, particularly in handling path-dependent products, making them a natural choice for pricing exotic options and complex financial instruments.
- The generic MC framework consists of several key components that facilitate pricing:
  - **Market Model:** Defines the financial environment, including interest rates, volatility, and asset dynamics.
  - **Random Number Generator (RNG):** Generates the stochastic inputs required for simulating asset price paths.
  - **Monte Carlo Product:** Represents the specific derivative contract being priced, including payoff structures and contract terms.
  - **Monte Carlo Diffusion Model:** Models the evolution of asset prices based on stochastic differential equations (e.g., Black-Scholes, Heston, or Local Volatility models).
  - **MC Pricer:** The central engine that integrates all components, simulates asset paths, and computes the expected payoff using statistical techniques.

MC Framework: Steps
- **Handling Multi-Asset Trades:**
  - For trades involving multiple underlying assets, the market model must account for the correlation structure between all relevant factors.
- **Generating Correlated Brownian Motions:**
  - The random number generator (RNG) produces independent standard Brownian motions.
  - These motions are then transformed to reflect the correlation matrix of the model factors, ensuring consistency with the market dynamics.
- **Diffusion Model for Asset Price Simulation:**
  - The diffusion model maps each Brownian motion to an underlying asset price process.
  - Each asset has its own diffusion model, which evolves according to the chosen stochastic differential equation (SDE).
- **Simulating Market Realizations:**
  - Each Monte Carlo simulation path represents a possible realization of future market movements.
- **Trade Valuation via Discounted Cash Flows:**
  - The trade (financial product) processes each simulated market realization to compute its discounted cash flows for that scenario.
  - This corresponds to evaluating
    $$h(X_i)$$
    in the expected value formulation
    $$\mathbb{E}[h(X)].$$
  - The final price estimate is obtained by averaging across multiple simulation paths.

Requirements for the Diffusion Model
A diffusion model must implement the following methods:
- `num_factors()` Returns the number of Brownian motions needed:

```
num_factors = model.num_factors()
```

- `get_time_steps(event_dates)` Returns the necessary time steps for diffusion:

```
time_steps = models[asset].get_time_steps(trade.all_dates())
```

- `diffuse(dts, bs)` Returns a function that interpolates the asset price:

```
observables[asset] = model.diffuse(time_steps, correlated_brownians)
```

Trade Payoff and Required Trade Functions
- Once the observable price paths for all relevant assets are simulated, the trade uses these observables to compute the discounted payoff for this simulation path:

```
h = trade.discounted_mc_payoff(fobs)
```

  This function evaluates the present value of the trade given the simulated market scenario.
- In addition to computing payoffs, the trade must also provide essential information for simulation:
  - Assets involved in the payoff:
    `trade.asset_names()`
  - Critical event dates for the trade:
    `trade.all_dates()`

Pricing Spread Option with Correlated Brownian Motions
- Recall that a spread option is the derivative contract whose payoff depends on the difference (or spread) between the prices of two underlying assets.
- A European spread option with maturity T and strike price K has a payoff of the form:
  - For a call option:
    $$\max(S_1(T) - S_2(T) - K, 0)$$
  - For a put option:
    $$\max(K - (S_1(T) - S_2(T)), 0)$$

  where $S_1(T)$ and $S_2(T)$ denote the prices of the two underlying assets at maturity.
- Spread options are more complex than standard (single-asset) options because the payoff depends on two correlated stochastic processes.
- There is no closed-form solution for the general spread option under the Black-Scholes framework.

Variance Reduction Techniques
- When performing Monte Carlo simulations, a range of variance reduction methods can significantly improve the accuracy of estimates with small computational overhead.
- *Antithetic Sampling:* Use negatively correlated (or "opposite") random draws to reduce the variance of the estimator.
- *Variates Recycling:* Reuse random numbers or underlying stochastic processes in multiple calculations to reduce noise.
- *Control Variates:* Incorporate additional known expectations (control variates) to offset random fluctuations in the primary estimator.
- *Stratified Sampling:* Divide the sampling space into distinct "strata" (groups) and sample within each stratum, ensuring more balanced coverage of the distribution.
- *Importance Sampling:* Sample more frequently in regions of higher importance (where the integrand is large) and adjust weights accordingly to improve efficiency.

Antithetic Sampling
- Antithetic sampling exploits the symmetry in Brownian motion to reduce Monte Carlo noise.
- Instead of using independent sample paths, we generate pairs of negatively correlated paths, which helps smooth out fluctuations in the estimator.
- When simulating Brownian motion, we recognize that for any randomly drawn path W_i, its mirror, $-W_i$, has the same probability distribution.
- Instead of using only W_i to compute the payoff $h(W_i)$, we also evaluate $h(-W_i)$.
- The new antithetic estimator is given by:
  $$\hat{h}_i = \frac{1}{2}\big(h(W_i) + h(-W_i)\big)$$
- This leads to the improved Monte Carlo estimate of the expected payoff
  $$\frac{1}{n}\sum_{i=1}^{n} \hat{h}_i.$$
- By averaging over positively and negatively correlated paths, variance is reduced, leading to a more efficient estimator compared to standard Monte Carlo sampling.

Antithetic Sampling
- The variance of the new estimator is:
  $$\mathrm{Var}\Big[\frac{1}{2}\big(h(W_i) + h(-W_i)\big)\Big] = \frac{1}{2}\mathrm{Var}[h(W_i)] + \frac{1}{2}\mathrm{Cov}[h(W_i), h(-W_i)]$$
- Variance Comparison with Standard Monte Carlo
  - To properly assess the variance reduction, we compare it against the variance obtained from a standard Monte Carlo estimator using twice the number of paths,
    $$\frac{1}{2}\mathrm{Var}[h(W_i)].$$
  - If the covariance term satisfies
    $$\mathrm{Cov}[h(W_i), h(-W_i)] < 0,$$
    then the total variance is reduced, making antithetic sampling more efficient than independent sampling.
- This holds whenever $h(W)$ is monotonic with respect to W
  - If f is increasing, then for a large positive Z is large and $f(-Z)$ is small.
  - For a large negative Z, $f(Z)$ is small and similarly, $f(-Z)$ is large.
  - So when one term is above its mean, the other tends to be below its mean.
- Payoff: Antithetic sampling is most effective when the payoff is **directional** with respect to the Brownian motion. Let
  $$S_T = S_0 \exp\left(\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}\,Z\right),$$
  and pair the two samples Z and $-Z$.

- The method reduces variance when
$$\mathrm{Cov}\big(f(S_T(Z)), f(S_T(-Z))\big) < 0.$$
- Why it works well for asymmetric payoffs
  - For a monotone payoff such as a call,
$$f(S) = (S - K)_+,$$
if Z is large, then $S_T(Z)$ is large and the payoff is large, while $S_T(-Z)$ is small and the payoff is small. Hence the two paired payoffs tend to move in **opposite directions**.
  - More generally, antithetic sampling works well when the payoff reacts **asymmetrically** to the underlying shock.

Why it is less effective for symmetric payoffs
- For a symmetric payoff such as a straddle,
$$f(S) = |S - K|,$$
a large positive shock and a large negative shock can both produce a large payoff.
- Therefore, under the antithetic pair Z and $-Z$, it is no longer true that one payoff is typically high while the other is low.
- As a result, the covariance is often only weakly negative, or close to zero, so the variance reduction is much smaller.
- General Rule:
  - Antithetic sampling tends to favor **monotone, directional, or asymmetric payoffs**.
  - For symmetric payoffs such as a straddle, it usually helps much less.
  - In most such cases, it does not significantly worsen the estimator, but the gain in variance reduction is limited.

Variates Recycling for Greek estimation
- In financial derivatives pricing, Greeks (risk sensitivities) are often more important than option prices, as they provide critical information for hedging and risk management.
- Often, the delta of an instrument, which measures sensitivity to changes in the underlying asset price, is approximated by finite difference:
$$\delta = \frac{\partial PV}{\partial S} \approx \frac{PV(S + \Delta S) - PV(S - \Delta S)}{2\Delta S}.$$
- The variance of delta is
$$\frac{1}{4\Delta S^2}\left(\mathrm{Var}(PV_{S+\Delta S}) + \mathrm{Var}(PV_{S-\Delta S}) - 2\mathrm{Cov}(PV_{S+\Delta S}, PV_{S-\Delta S})\right).$$
- Reduced Variance with variates recycling
  - Instead of using independent random samples for $PV_{S+\Delta S}$ and $PV_{S-\Delta S}$, we can reuse the same random samples in both calculations.
  - This maximizes the correlation between the two estimates, increasing
$$\mathrm{Cov}(PV_{S+\Delta S}, PV_{S-\Delta S}),$$
which reduces the overall variance.
  - This is also a reason why we would like our "random" numbers to be replicable.

Control Variates
- Our goal is to estimate the expectation of a function $h(X)$.
- If the expectation of a function $g(X)$ is known analytically, we can construct the following estimator:
$$\mathbb{E}[h(X)] \approx \frac{1}{n}\sum_{i=1}^{n}\big(h(X_i) + \beta(g^* - g(X_i))\big),$$
where g^* is the known expectation of $g(X)$ and β is a parameter.
- The variance of this estimator is given by
$$\mathrm{Var}[h] + \beta^2\mathrm{Var}[g] - 2\beta\mathrm{Cov}[h, g].$$
- To minimize the variance, we take the derivative with respect to β and set it to zero, yielding the optimal choice
$$\beta = \frac{\mathrm{Cov}[h, g]}{\mathrm{Var}[g]}.$$
- This optimal β can be estimated using simulation samples.

Control Variates
- The minimized variance of the estimator is given by
$$\mathrm{Var}[h] - \frac{\mathrm{Cov}[h, g]^2}{\mathrm{Var}[g]} = \mathrm{Var}[h](1 - \rho^2),$$
where ρ is the correlation coefficient between h and g.
- The effectiveness of the control variate technique increases with the correlation between g and h, a higher ρ leads to greater variance reduction.
- Examples of control variates:
  - Use a forward contract as a control variate for a call option:
    - Forward contract: an over-the-counter agreement with $S_T - F$ (long), $F - S_T$ (short).
  - Use geometric Asian option as a control variate for an arithmetic Asian option
$$A_{\text{arith}} = \frac{1}{n}\sum_{i=1}^{n} S(t_i) \quad \text{vs.} \quad A_{\text{geom}} = \left(\prod_{i=1}^{n} S(t_i)\right)^{1/n}.$$

- This technique is not universally applicable; it requires careful selection and fine-tuning based on the specific problem.

Stratified Sampling
- The idea is to partition the domain into k disjoint subsets and compute the expectation as:
$$\mathbb{E}[h(X)] = \sum_{j=1}^{k}\mathbb{E}[h(X \in A_j)]\, p(X \in A_j).$$
- Depending on the behavior of h, we can allocate more samples to regions A_j whose h value exhibits higher variability and few samples to regions where h is relatively constant.
- The inverse transformation method is useful in this context, as it enables the generation of uniform random samples corresponding to each stratum, leveraging monotonicity.
- If X is one-dimensional with cumulative distribution function F, then inverse transform sampling allows us to generate X from a uniform variable $U \sim \mathrm{Unif}(0, 1)$ via
$$X = F^{-1}(U).$$
- Now suppose we want samples from a particular stratum, for example
$$X \in A_j = [a_j, a_{j+1}].$$
- This corresponds to
$$U \in [F(a_j), F(a_{j+1})].$$

Stratified sampling
- Then, we can generate a uniform sample
$$U \sim \mathrm{Unif}(F(a_j), F(a_{j+1}))$$
and then set
$$X = F^{-1}(U).$$
- In this way, inverse transform sampling gives samples **restricted to a chosen stratum**. This works well because F is monotone.
- This technique is challenging to generalize.
  - First, the choice of strata is highly problem-dependent. To get good variance reduction, the strata should reflect where h changes rapidly or where the distribution places important mass. There is usually no universal rule for choosing them.
  - Second, in high dimension, stratification becomes much harder. If you divide each coordinate into k intervals in d dimensions, then the total number of strata becomes k^d, which grows exponentially with d.

Importance Sampling
- The key idea is to perform a change of measure to concentrate probability mass in the most important regions:
$$\mathbb{E}_p[h(X)] = \int h(X)q(X)\,dX = \int h(X)\frac{q(X)}{p(X)}p(X)\,dX.$$
- By rewriting the integral, we obtain:
$$= \mathbb{E}_p\left[h(X)\frac{q(X)}{p(X)}\right].$$
- The probability density $p(X)$ can be designed to concentrate more samples in regions where $h(X)$ is more important, reducing variance by allocating fewer samples to less important regions.
- Importance sampling shares similarities with stratified sampling, with similar challenges:
  - Highly problem-dependent, requiring careful selection of an appropriate importance distribution.
  - Difficult to extend to high-dimensional problems due to the complexity of defining an effective sampling distribution.

Example of importance sampling
- Problem: estimating the integral:
$$I = \int_0^\infty e^{-x}h(x)\,dx,$$
where $h(x) = \frac{x^2}{1+x^2}$. Then, this integral is the expectation of $h(x)$ under the exponential distribution e^{-rx} with rate $r = 1$.
- The function $h(x)$ grows slowly for large x, but the exponential distribution decays quickly, so we rarely sample large x, where $h(x)$ has large values, close to 1.
- To apply importance sampling, we choose a proposal distribution q that places relatively more mass on larger values of x. For example, we may choose an exponential distribution with a smaller rate $1/2$:
$$q(x) = \frac{1}{2}e^{-x/2}, \quad x \geq 0.$$
- Then,
$$I = \int_0^\infty h(x)\frac{e^{-x}}{q(x)}q(x)\,dx = \mathbb{E}_{Y\sim q}\left[2e^{-Y/2}\frac{Y^2}{1+Y^2}\right].$$

A better candidate
- Using $q(x) = e^{-x/2}$ is a fine choice, but there are issues
  - $e^{-x/2}$'s very heavy tail can result in large and unstable importance weights, especially as the sample size grows. This can cause increased variance in the estimator, even though you're better covering the region.

- A better candidate should have two properties
  - Good coverage of the tail like $e^{-x/2}$
  - but not overly heavy tails (which cause high-variance)
- Let's switch to a Gamma distribution with shape parameter $\alpha > 1$ and a modest β. For example:

$$q(x) = \text{Gamma}(\alpha = 3, \beta = 2) = 4x^2 e^{-2x}.$$

Summary of Variance Reduction Techniques

- We have briefly discussed the following variance reduction techniques
  - Antithetic sampling
  - Variates recycling
  - Control variates
  - Stratified sampling
  - Importance sampling
- Simple techniques are easy to implement and generalize, making them preferable in many practical scenarios.
- Advanced techniques tend to be harder to implement and generalize but can achieve significant variance reduction when applied correctly.
- Note that however, all these techniques improve convergence only by reducing the constant factor of $O(n^{-1/2})$.
- Is there a method that can fundamentally improve the convergence rate beyond this order of magnitude?

Monte Carlo vs. Quadrature

- In the previous discussion, we saw that variance reduction techniques can improve the efficiency of MC methods but do not change the convergence rate of $O(n^{-1/2})$.
- In contrast, deterministic **Quadrature method** can achieve faster convergence.
- For example, the trapezoidal rule approximates the integral of a $C^2_{[0,1]}$ function

$$\alpha \approx \frac{f(0) + f(1)}{2n} + \frac{1}{n} \sum_{i=1}^{n-1} f(i/n).$$

Trapezoid scheme

- The error of trapezoidal method is $O(n^{-2})$, which is significantly better than Monte Carlo's convergence rate.
- Let $x_i = \frac{i}{n},\ i = 0, 1, \ldots, n$ and $h = \frac{1}{n}$. Then the trapezoidal method reads

$$T = \sum_{i=1}^{n} T_i = h \sum_{i=1}^{n} \frac{f(x_{i-1}) + f(x_i)}{2}.$$

- Next, we derive its error.
  - We have $\int_{x_0}^{x_n} f(x)dx = \sum_{i=0}^{n-1} \int_{x_{i-1}}^{x_i} f(x)dx$.
  - By Taylor expansion of $f(x)$ at x_{i-1}:

$$f(x) = f(x_{i-1}) + (x - x_{i-1})f'(x_{i-1}) + O(h^2).$$

- Then, we have

$$I_i = \int_{x_{i-1}}^{x_i} f(x)dx = hf(x_{i-1}) + \frac{h^2}{2} f'(x_{i-1}) + O(h^3).$$

Monte Carlo vs. Quadrature

- Expanding $f(x_i)$ at x_{i-1} gives

$$f(x_i) = f(x_{i-1}) + hf'(x_{i-1}) + O(h^2) \Rightarrow f'(x_{i-1}) = \frac{f(x_i) - f(x_{i-1})}{h} + O(h).$$

- Substitute into the trapezoidal rule formula:

$$I_i = hf(x_{i-1}) + \frac{h^2}{2}\left(\frac{f(x_i) - f(x_{i-1})}{h} + O(h)\right) + O(h^3) = \frac{h}{2}(f(x_i) + f(x_{i-1})) + O(h^3).$$

- The error on each interval $[x_{i-1}, x_i]$ is then

$$E_i = I_i - T_i = O(h^3),$$

  which gives

$$E = \sum_{i=1}^{n} E_i = O(nh^3) = O(h^2).$$

- This raises an important question: Can we leverage deterministic approaches or hybrid methods to improve upon Monte Carlo sampling?

Monte Carlo vs. Quadrature in Higher Dimensions

- From previous discussions, we observed that Monte Carlo integration is generally not competitive for one-dimensional problems due to its slow convergence rate of $O(n^{-1/2})$.
- However, for a **two-dimensional integral**, a natural approach is to discretize each dimension into m points, leading to a total of $n = m^2$ sample points. A quadrature method, such as the trapezoidal rule, approximates the integral as

$$\int_0^1 \int_0^1 f(x, y)\, dx\, dy \approx \frac{1}{m^2} \sum_i \sum_j f(x_i, y_j) = \frac{1}{n} \sum_i \sum_j f(x_i, y_j).$$

- This is essentially a grid-based numerical integration, where function values are evaluated at a structured set of points.
- Since each dimension contributes an error of $O(m^{-2})$, the overall error in 2D integration is $O(m^{-1})$.
- In a d-dimensional problem, if we discretize each dimension using m points, the quadrature error remains dictated by the lowest-order term across all dimensions, resulting in a general error rate of $O(n^{-2/d})$.
- The error rate of $O(n^{-2/d})$ means the convergence rate of the quadrature method decreases when d increases.
- However, Monte Carlo method remains $O(n^{-1/2})$ due to the Central Limit Theorem. Therefore, Monte Carlo method is attractive for evaluating high-dimensional integrals.
- Let us have a look again at the Monte Carlo estimator:

$$\int_A f(X)\, dX \approx \frac{1}{n} \sum_{i=1}^{n} f(X_i),$$

  and compare it with the d-dimensional quadrature with equal spacing:

$$\int_A f(X)\, dX = \frac{1}{n} \sum_{i=1}^{n} f(X_i).$$

- It is the same formula: taking the average of sample evaluations of the function $f(X)$. So what is the difference?

Monte Carlo vs. Quadrature: Sampling strategy

- The difference lies in the sample points chosen: quadrature chooses regular grid points while Monte Carlo chooses random points.
- Why does this matter?
  - When integrating a function over multiple dimensions, you effectively project the points onto each dimension.
  - In quadrature, because the points are on a grid, when projected onto each dimension, you might see repetition of certain values.
  - In Monte Carlo, since points are randomly chosen, their projections onto each dimension remain independent and varied, ensuring that you still have n distinct values in each dimension.

Monte Carlo vs. Quadrature in higher dimensions

- Quadrature methods are highly efficient in low dimensions, but struggle with high-dimensions due to exponential growth in sample points.
- Monte Carlo integration, despite its slow convergence rate, is competitive in high-dimensions, as its convergence rate does not depend on dimension d.
- This phenomenon is known as the **curse of dimensionality**.

Quasi Monte Carlo

- Monte Carlo methods perform well in high dimensions, but can we improve the convergence rate by sampling more regularly, similar to quadrature in one dimension?
- Quasi-Monte Carlo (QMC) enhances Monte Carlo integration by using quasi-random numbers, which are designed to be low-discrepancy sequences – uniformly distributed without clustering.
- The discrepancy of a sequence is measured as:

$$D_N^{(d)} = \sup_{y \in [0,1]^d} \left| \frac{n_{S(y)}}{N} - \prod_{k=1}^{d} y_k \right|,$$

  where $n_{S(y)}$ is the number of sample points within the axis-aligned hyperrectangle

$$S(y) = [0, y_1] \times [0, y_2] \times \cdots \times [0, y_d]$$

  in the unit cube.

Quasi Monte Carlo

- Understanding the discrepancy formula:
  - Empirical proportion $\frac{n_{S(y)}}{N}$: The fraction of sampled points that fall inside a rectangular region.
  - Expected proportion $\prod_{k=1}^{d} y_k$: The fraction of volume that $S(y)$ occupies under perfect uniform distribution.
  - If the two values are close for all $S(y)$, the sample is well-distributed (low discrepancy); otherwise, uneven coverage indicates deviation from uniformity (high discrepancy).
- QMC achieves a convergence rate of $O\left(\frac{(\log n)^d}{n}\right)$, which simplifies to approximately $O(n^{-1})$ when $d \ll n$.

Halton Sequence

- Each dimension is assigned a unique prime number base p_i.
- Given an index n, it is first expressed in the base p_i as

$$\gamma(n) = \sum_{k=1}^{m_i} a_k p_i^{k-1},$$

  where a_k are the digits of n in base p_i.
- The quasi-random number for the n-th draw in dimension i is computed as

$$u(n) = \sum_{k=1}^{m_i} a_k p_i^{-k}.$$

- Example: for the first dimension, set $\gamma(n) = n$ and $p_1 = 2$, the sequence is

$$1 \to 1_2 \to 0.1_2 \to 0.5, \quad 2 \to 10_2 \to 0.01_2 \to 0.25,$$

$$3 \to 11_2 \to 0.11_2 \to 0.75, \quad 4 \to 100_2 \to 0.001_2 \to 0.125.$$

MT19937 vs. Halton Sequence

- Comparison between pseudo-random number generators (PRNG) and quasi-Monte Carlo (QMC) sequences:
  - **Type:**
    - MT19937: PRNG
    - Halton: QMC
  - **Purpose:**
    - MT19937: General-purpose randomness
    - Halton: Low-discrepancy sampling
  - **Uniformity:**
    - MT19937: Approaches uniformity in the long run but exhibits clustering
    - Halton: More evenly spread (low discrepancy)

Quasi Random Numbers in Monte Carlo

- When incorporating quasi-random numbers in Monte Carlo simulations, careful handling of random number generation is essential.
- Each Brownian increment corresponds to one dimension. For 100 time steps, we need to sample 100-dimensional random vectors.
- A key limitation of the Halton sequence becomes evident: as the base number increases, the discrepancy grows, reducing the effectiveness of quasi-random sampling.

QMC – Dimensionality

- Quasi-random sequences (low-discrepancy sequences) have limited effective dimensions. As the base increases, the quality of higher dimensions degrades.
- It is crucial to allocate high-quality dimensions to the most influential aspects of the simulation, such as option expiries. This can be optimized using:
  - **Brownian Bridge Construction:** Assign the first dimension to the expiry date, the second to the midpoint, and so on, ensuring better variance reduction.
  - **PCA Construction:** Utilize lower-dimensional quasi-random numbers to construct the principal components of Brownian motion, forming its fundamental structure.

Lecture 10

Importance of PDE Formulation in Quantitative Finance

- It is essential to derive the PDEs governing financial instruments, which provide the basis for pricing derivatives and understanding the evolution of risk-neutral probability densities.
- **Fokker-Planck Equation (Forward PDE):** Describes the evolution of the probability density function of an asset price under a given stochastic process.
- **Black-Scholes Equation (Backward PDE):** Governs the price of derivatives based on the underlying asset.
- Understanding these equations allows us to determine how different financial models affect asset price distributions and option pricing.
- Connection Between Fokker-Planck and Black-Scholes PDE:
  - FPE governs probability density evolution (forward in time).
  - BS PDE governs option price evolution (backward in time).
  - Both are linear parabolic PDEs, second order in space and first order in time.
  - The BS equation can be transformed into the heat equation, which has well-known analytical and numerical solutions.

Black-Scholes equation vs Fokker-Planck Equation

- Two PDEs

$$\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{\sigma^2 S^2}{2}\frac{\partial^2 V}{\partial S^2} - rV = 0 \quad \text{(BS)}$$

$$\frac{\partial p}{\partial t} + (r-q)\frac{\partial p}{\partial s} - \frac{\sigma^2 s^2}{2}\frac{\partial^2 p}{\partial s^2} + (r-q-\sigma^2)p = 0 \quad \text{(FPE)}$$

- where
  - $V(S,t)$: the price of a derivative (e.g. European call)
  - $p(s,t)$: the probability density function of underlying asset price
- The parameters
  - r: risk-free interest rate
  - q: continuous dividend yield
  - σ: volatility of the underlying asset (constant or state-dependent)

Fokker Planck equation

- We have derived the general Fokker Planck PDE that governs the risk neutral probability density function $p(x,t)$ for diffusion model:

$$dX = A(X,t)dt + B(X,t)dW,$$

$$\frac{\partial p(x,t)}{\partial t} = -\frac{\partial[p(x,t)a(x,t)]}{\partial x} + \frac{1}{2}\frac{\partial^2[p(x,t)b^2(x,t)]}{\partial x^2}.$$

- For the local volatility model, the underlying asset price follows the risk-neutral SDE:

$$dS_t = \mu(S,t)Sdt + \sigma(S,t)SdW_t,$$

  where μ denotes the risk-neutral drift and σ is the local volatility function.
- The corresponding Fokker Planck equation is

$$\frac{\partial p(s,t)}{\partial t} = -\frac{\partial[p(s,t)\mu(s,t)]}{\partial s} + \frac{1}{2}\frac{\partial^2[s^2\sigma^2(s,t)p(s,t)]}{\partial s^2}.$$

- This equation plays a fundamental role in determining the local volatility surface $\sigma(s,t)$ from observed option prices through calibration techniques.

Two approaches to deriving Black-Scholes Equation

- Replicating Portfolio Approach (Hedging Argument)
  - Construct a portfolio using basic financial instruments, e.g., stocks and bonds.
  - Derive a PDE that guarantees the portfolio replicates the derivative's payoff (e.g., a European call option). If replication is possible, the derivative's price must equal the portfolio's cost to avoid arbitrage.
- Risk-Neutral Measure Approach (Martingale Property)
  - Transform the real-world probability measure to a risk-neutral measure.
  - Use the fact that discounted option prices must be martingales.
  - Leads to the same PDE as the hedging argument but with a probabilistic interpretation.
- Both derivations lead to the same Black-Scholes PDE, but with different perspectives:
  - The hedging argument highlights portfolio replication and arbitrage-free pricing using tools from stochastic calculus.
  - The risk-neutral approach offers a probabilistic view, interpreting option prices as expected discounted payoffs under a risk-neutral measure.

Pricing Derivatives under the Local Volatility Model

- For pricing derivative instruments under the local volatility model, an analytical solution generally does not exist due to the complex dependence, consequently, pricing must rely on numerical methods.
- One way to price is through the partial differential equation that governs the price of the derivative instrument, denoted as $V(S,t)$, where S is spot price and t is time.
- The local volatility model differs from Black-Scholes model only in that the parameters μ and σ are time and state dependent.
- This difference does not affect the derivation of the partial differential equation because the two parameters are both deterministic.
- We will derive Black-Scholes PDE first, and local volatility model's PDE will be straightforward.
- When we say Black-Scholes PDE, what we mean is the PDE that governs the derivative instrument's price $V(S,t)$ under Black-Scholes model.

Black-Scholes PDE Derivation - Replicating Portfolio I

- Setting up the model:
  - Suppose that the price of a stock S_t follows a stochastic process governed by a geometric Brownian motion:

$$\frac{dS}{S} = \bar{\mu}dt + \sigma dW,$$

    where
    - $\bar{\mu}$ is the drift of the stock
    - σ is the volatility
    - W_t: standard Brownian motion
- The risk-free asset (money market account): The value of risk-free asset B_t grows deterministically at a constant rate

$$\frac{dB}{B} = rdt,$$

  where r is the continuously compounded risk-free interest rate.

Black-Scholes PDE Derivation - Replicating Portfolio I

- Constructing the Replicating Portfolio: A replicating portfolio consists of:
  - δ shares of the stock S
  - β amount of cash invested in the risk-free bond B
- The value of the portfolio is given by:

$$\Pi = \delta \cdot S + \beta \cdot B.$$

- For the portfolio to perfectly replicate the derivative price $V(S,t)$, we must ensure that, for all S and t

$$\Pi - V = 0.$$

- This means that the portfolio is constructed such that its value always matches the derivative price, eliminating arbitrage opportunities.

Black-Scholes PDE Derivation - Replicating Portfolio I

- Solving β:

$$V = \delta S + \beta B \Rightarrow \beta = \frac{V - \delta S}{B}.$$

- Computing the Increment of the Replicating Portfolio
  - Compute the differential of Π

$$d\Pi = \delta dS + \beta dB.$$

  - Assume the stock pays continuous dividend yield q:

$$d\Pi = \delta(dS + qSdt) + \beta dB.$$

  - Substituting the dynamics of S and B, we have

$$d\Pi = \delta(dS + qSdt) + \beta rBdt = \delta dS + q\delta Sdt + r(V - \delta S)dt = \delta dS + (\delta qS + rV - r\delta S)dt.$$

Black-Scholes PDE Derivation - Replicating Portfolio I

- Meanwhile, by Ito's Lemma, the increment of $V(S,t)$ is

$$dV = \frac{\partial V}{\partial S}dS + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2}dt + \frac{\partial V}{\partial t}dt.$$

- As $\Pi = V$ for avoiding arbitrage, equating $d\Pi$ and dV:

$$d\Pi = \delta dS + (\delta qS + rV - r\delta S)dt,$$

  we have

$$\begin{cases} \delta = \frac{\partial V}{\partial S} \\ \frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} - rV = 0 \end{cases}$$

- When extended to the local volatility model, only σ changes.
- Note that we didn't mention probabilistic expectations or martingale arguments, but solely through hedging and arbitrage-free pricing.

Black-Scholes PDE Derivation - Through Risk Neutrality I

- When pricing derivatives, we do not care about the expected return $\bar{\mu}$ of the asset. Instead, we seek the arbitrage-free price.

- To ensure an arbitrage-free framework, we work under the risk-neutral measure Q, where asset prices evolve such that all tradable assets have an expected return equal to the risk-free rate.
- Under risk neutral measure, the asset and bond evolve as:

$$\frac{dS}{S} = (r-q)dt + \sigma dW, \qquad \frac{dB}{B} = rdt,$$

where r is the risk-free rate, and q is the continuous dividend yield.
- Martingale property of the discounted derivative price $\frac{V(t)}{B(t)}$ under Q: Its expected future value is equal to its current value when discounted at the risk-free rate:

$$\mathbb{E}\left[\frac{V(T)}{B(T)}\middle|\mathcal{F}_t\right] = \frac{V(t)}{B(t)}.$$

- where $\mathcal{F}_t$ represents the investor's knowledge at time t.

Black-Scholes PDE Derivation - Through Risk Neutrality II

- Recall we have

$$dV = \frac{\partial V}{\partial S}dS + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}dt + \frac{\partial V}{\partial t}dt.$$

- Replacing $\frac{dS}{S} = (r-q)dt + \sigma dW$, we have

$$dV = \frac{\partial V}{\partial t}dt + \frac{\partial V}{\partial S}S(r-q)dt + \frac{\partial V}{\partial S}S\sigma dW + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}dt.$$

- Using the Ito product rule for $\frac{V}{B}$:

$$d\left(\frac{V}{B}\right) = \frac{dV}{B} + Vd\left(\frac{1}{B}\right) + dVd\left(\frac{1}{B}\right).$$

- Since B is deterministic, we have $dVd\left(\frac{1}{B}\right) = 0$. Then,

$$d\left(\frac{V}{B}\right) = \frac{dV}{B} + Vd\left(\frac{1}{B}\right), \quad \text{where} \quad d\left(\frac{1}{B}\right) = d(e^{-rt}) = -re^{-rt}dt = -\frac{r}{B}dt.$$

Black-Scholes PDE Derivation - Through Risk Neutrality II

- Then,

$$d\left(\frac{V}{B}\right) = V\left(-\frac{r}{B}\right)dt + \frac{dV}{B}.$$

- Substituting dV:

$$d\left(\frac{V}{B}\right) = -\frac{rV}{B}dt + \frac{1}{B}\left(\frac{\partial V}{\partial t}dt + \frac{\partial V}{\partial S}S(r-q)dt + \frac{\partial V}{\partial S}S\sigma dW + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}dt\right).$$

- Rearranging:

$$d\left(\frac{V}{B}\right) = \frac{1}{B}\left(\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rV\right)dt + \frac{1}{B}\frac{\partial V}{\partial S}S\sigma dW.$$

- Under the risk-neutral measure, the discounted price of any tradable asset must be a martingale $\Rightarrow$ no drift term.

Verifying Black-Scholes formula

- Defining:

$$d_\pm = \frac{\ln\frac{F}{K} \pm \frac{1}{2}\sigma^2(T-t)}{\sigma\sqrt{T-t}}, \quad F = Se^{(r-q)(T-t)}.$$

- Then, for

$$V = Se^{-q(T-t)}N(d_+) - Ke^{-r(T-t)}N(d_-),$$

we have

$$\frac{\partial V}{\partial S} = e^{-q(T-t)}N(d_+),$$

$$\frac{\partial^2 V}{\partial S^2} = \frac{e^{-q(T-t)}\phi(d_+)}{\sigma S\sqrt{T-t}},$$

$$\frac{\partial V}{\partial t} = \frac{Se^{-q(T-t)}\phi(d_+)\sigma}{2\sqrt{T-t}} - rKe^{-r(T-t)}N(d_-) + qSe^{-q(T-t)}N(d_+).$$

- It is easy to verify that the Black-Scholes formula is a solution to the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rV = 0.$$

Black-Scholes PDE and FPE (Fokker Planck Equation)

- Understanding the connection between the BS PDE and the FPE:

$$\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{\sigma^2 S^2}{2}\frac{\partial^2 V}{\partial S^2} - rV = 0,$$

$$\frac{\partial p}{\partial t} + (r-q)s\frac{\partial p}{\partial s} - \frac{\sigma^2 s^2}{2}\frac{\partial^2 p}{\partial s^2} + (r-q-\sigma^2)p = 0.$$

- Very similar structure: Both equations contain drift term $(r-q)S$ and diffusion term $\frac{1}{2}\sigma^2 S^2$.
- Main difference in sign of the second derivatives: It reflects that BS PDE propagates option prices backward in time, whereas FPE propagates probability densities forward in time.
- If $r = q = 0$, both equations simplify to heat equation. FPE is standard heat equation and the BS PDE is the inverse of heat equation.
- Interpretation:
  - The BS PDE (backward equation) describes how the value of an option evolves as we move backward in time from expiration.
  - The FPE (forward equation) describes how the probability density function of the underlying asset price evolves forward in time.

Finite Difference Method

- Motivation for finite difference methods
  - The Black-Scholes PDE governs the price evolution of derivative instruments but does not have closed-form solutions for most options (except for simple European options with constant σ).
  - To compute option prices numerically, we discretize the PDE using finite difference methods.
  - The idea behind FDM is to approximate continuous derivatives with discrete differences, transforming the PDE into a system of algebraic equations that can be solved computationally.
- How Finite Differences Work
  - The main idea is to replace partial derivatives in the Black-Scholes equation with finite difference approximations.
  - This involves defining a grid in time and asset price space, where values of the option price $V(S,t)$ are computed at discrete points.

Finite Difference - Order Of Convergence

- The definition of derivative is

$$\frac{df(x_0)}{dx} = \lim_{h\to 0}\frac{f(x_0+h) - f(x_0)}{h}.$$

- It is natural to approximate the derivative using small h:

$$\frac{df(x_0)}{dx} \approx \frac{f(x_0+h) - f(x_0)}{h}.$$

- The order of convergence can be analyzed through Taylor expansion:

$$f(x_0+h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + O(h^3),$$

$$f'(x_0) = \frac{f(x_0+h) - f(x_0)}{h} + O(h).$$

- The term $O(h)$ represents the truncation error, which scales proportionally to h.
- The convergence of the approximation is thus $O(h)$.
- The smaller h is, the more accurate the approximation is, but how small can h go?

Type Of Numerical Errors (again)

- In general, for finite difference approximations, there are two types of numerical errors:
  - Round-off error
    - Computer represents numbers at limited precision,
    - In double precision, machine precision is approximately 10^{-16} to 10^{-15}.
  - Truncation error
    - This is the error introduced when we approximate derivatives using finite differences.
    - For the forward difference formula:

$$\frac{f(x_0+h) - f(x_0)}{h},$$

the truncation error is $O(h)$.
- Trade-off between round-off and truncation errors
  - Reducing h decreases truncation error, but increases round-off error.
  - For forward difference, the rounding-off error is amplified by $1/h$. So for very small h, the round-off error dominates.
  - $h \approx 10^{-8}$ is a good choice for double precision floating-point arithmetic.

Central Difference

- h cannot be used to achieve higher accuracy of forward difference.
- Forward difference using one-sided difference: using points $f(x_0)$ and $f(x_0+h)$.
- If we want to achieve higher order convergence, we can list the Taylor expansion at both x_0+h and x_0-h:

$$f(x_0+h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + O(h^3),$$

$$f(x_0-h) = f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + O(h^3).$$

- Subtracting gives:

$$f'(x_0) = \frac{f(x_0+h) - f(x_0-h)}{2h} + O(h^2).$$

- This is called **central difference**, whose order of convergence is second order in h.

Higher Order Finite Difference by More Points

- In general, the more points we have, the more accurate we can achieve by listing the Taylor expansion and eliminating low order terms.
- Considering we use four points at $x_0 + h_1,\ x_0 + h_2,\ x_0 + h_3,\ x_0 + h_4$, we have expansions:

$$c_1 f(x_0 + h_1), \quad c_2 f(x_0 + h_2), \quad c_3 f(x_0 + h_3), \quad c_4 f(x_0 + h_4).$$

- Suppose we will approximate $f'(x_0)$ by the linear combination:

$$f'(x_0) \approx \sum_{i=1}^{4} c_i f(x_0 + h_i).$$

- The next question is how to set the values $\{c_i\}_{i=1}^4$ to have the highest-order truncation error.
- It boils down to solving the linear system

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ h_1 & h_2 & h_3 & h_4 \\ \frac{h_1^2}{2} & \frac{h_2^2}{2} & \frac{h_3^2}{2} & \frac{h_4^2}{2} \\ \frac{h_1^3}{6} & \frac{h_2^3}{6} & \frac{h_3^3}{6} & \frac{h_4^3}{6} \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

- The solution of the system above gives the approximation with order $O(h^3)$:

$$f'(x_0) = \sum_{i=1}^{4} c_i f(x_0 + h_i) + O\left(\sum_{i=1}^{4} c_i h_i^4\right).$$

- The truncation error is of order $O(h^3)$ if all h_i are of the same order and $\sum c_i h_i = 1$.
- In general with n points we can achieve truncation error at $O(h^{n-1})$.
- Note that central difference is a special case: $O(h^2)$ with only two symmetric points.

Richardson Extrapolation: Improving Accuracy of Finite Difference

- Richardson Extrapolation is built on a similar principal
  - Finite difference approximations introduce truncation errors of order $O(h^n)$
  - Richardson Extrapolation is to eliminate lower-order error terms by combining approximations at different step sizes.
- Formulation: Suppose the function $g(h)$ approximates the derivative $f'(x_0)$ with order $O(h^n)$:

$$g(h) = f'(x_0) + Ch^n + O(h^{n+1}),$$

  where C is a constant.
- To construct a higher-order estimator, we evaluate $g(h)$ at two step sizes:
  - $g(h)$ with step size h
  - $g(kh)$ with step size kh, where k is a scale

Richardson Extrapolation: Improving Accuracy of Finite Difference

- Now we define a new estimator $r(h, k)$ by eliminating the term Ch^n:

$$r(h, k) = \frac{k^n g(h) - g(kh)}{k^n - 1}.$$

- Substituting expansions:

$$r(h, k) = \frac{k^n (f'(x_0) + Ch^n + O(h^{n+1})) - (f'(x_0) + Ck^n h^n + O(h^{n+1}))}{k^n - 1} = f'(x_0) + O(h^{n+1}).$$

- By evaluating twice the estimator with two step sizes, we have a higher order estimator.
- Example: considering central difference operator with order $O(h^2)$

$$g(h) = \frac{f(x + h) - f(x - h)}{2h}.$$

- By setting $k = 2$, we get

$$r(h, 2) = \frac{4g(h) - g(2h)}{3} + O(h^3).$$

- It uses 4 points: $\pm 2h, \pm h$ and the order is $O(h^3)$.

Finite Difference Method — Considerations

- Key Applications of Finite Difference Methods in Finance
  - Numerical Option Pricing
    - Used to solve the Black-Scholes PDE and its extensions.
    - Commonly applied in pricing options where closed-form solutions do not exist.
    - Alternative to Monte Carlo methods for solving high-dimensional stochastic models.
  - Model Calibration & Local Volatility Construction
    - Used to construct the local volatility surface by solving inverse problems.
    - Helps fit market-implied volatilities to PDE-based models.
  - Estimating the greeks: when there is no analytic formula available (almost the case)
- The choice of step size h in finite difference computations is critical:
  - Too large $\rightarrow$ Leads to higher truncation error, reducing accuracy.
  - Too small $\rightarrow$ Causes instability and amplification of round-off errors, especially in floating-point computations.

Overview of Our Solution Framework

- Discretize the domain (S, t)
- From the final condition at $t = T$, solve backward the option price $V(S, t)$ at the grids
- For any point not on the grid, we can use interpolation

Outline of the algorithm

- Discretize V in both price and time domain:

$$V_{i,j} = V(S_{\min} + i\Delta S, j\Delta t),$$

  where

$$\Delta S = \frac{S_{\max} - S_{\min}}{N_S - 1}, \quad \Delta t = \frac{T}{N_t - 1}.$$

- Denote the vector of the function at each time step j by

$$V^j = [V_{0,j}, V_{1,j}, \ldots, V_{N_S-1,j}]^T.$$

- The algorithm
  - Calculating $V^j,\ j = N_t - 1$
  - For $j = N_t - 1, \ldots, 0$,
    - Calculate boundary condition
    - Calculate V^{j-1} from V^j by finite difference scheme
  - Output V.

Discretizing Black-Scholes Equation

- Recall the Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + (r - q)S\frac{\partial V}{\partial S} + \frac{\sigma^2 S^2}{2}\frac{\partial^2 V}{\partial S^2} - rV = 0.$$

- For discussion simplicity and without loss of generality
  - We omit the index of time
  - We divide the spot axis in equal space (from $S_{\min}$ to $S_{\max}$) by N_S points
  - We discretize the partial derivatives with respect to S using finite difference:
  - It is easy to check that

$$f''(x_0) = \frac{f(x_0 + h) + f(x_0 - h) - 2f(x_0)}{h^2} + O(h^2).$$

  - Then, we have

$$\frac{\partial V_i}{\partial S} = \frac{V_{i+1} - V_{i-1}}{S_{i+1} - S_{i-1}},$$

$$\frac{\partial^2 V_i}{\partial S^2} = \frac{V_{i+1} + V_{i-1} - 2V_i}{(S_{i+1} - S_i)^2}.$$

Discretizing Black-Scholes Equation

- Substituting the finite differences back to the Black-Scholes PDE we have

$$\frac{\partial V_i}{\partial t} + (r - q)S\frac{V_{i+1} - V_{i-1}}{2\Delta S} + \frac{\sigma^2 S^2}{2}\frac{V_{i+1} + V_{i-1} - 2V_i}{\Delta S^2} - rV_i = 0.$$

- Reorganize it we get

$$\frac{\partial V_i}{\partial t} + \left(\frac{(r - q)S}{2\Delta S} + \frac{\sigma^2 S^2}{2\Delta S^2}\right)V_{i+1} + \left(-\frac{\sigma^2 S^2}{\Delta S^2} - r\right)V_i + \left(-\frac{(r - q)S}{2\Delta S} + \frac{\sigma^2 S^2}{2\Delta S^2}\right)V_{i-1} = 0.$$

Discretizing Black-Scholes Equation

- Define

$$A_i = -\frac{(r - q)S_i}{2\Delta S} + \frac{\sigma^2 S_i^2}{2\Delta S^2}, \quad B_i = r + \frac{\sigma^2 S_i^2}{\Delta S^2}, \quad C_i = \frac{(r - q)S_i}{2\Delta S} + \frac{\sigma^2 S_i^2}{2\Delta S^2}.$$

- We have

$$\frac{\partial V_i}{\partial t} = (A_i, B_i, C_i)\begin{pmatrix} V_{i-1} \\ V_i \\ V_{i+1} \end{pmatrix},$$

  for $i \in \{1, 2, \ldots, N_S - 2\}$.
- Now, we have an ordinary differential equation of V with respect to t, for $i \in \{1, 2, \ldots, N_S - 2\}$.

Discretizing Black-Scholes Equation

- We have a system of $N_S - 2$ ODE's:

$$\begin{pmatrix} \partial V_1/\partial t \\ \partial V_2/\partial t \\ \vdots \\ \partial V_{N_S-2}/\partial t \end{pmatrix} = \begin{pmatrix} A_1 & B_1 & C_1 & 0 & \cdots & 0 \\ 0 & A_2 & B_2 & C_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 0 & A_{N_S-2} & B_{N_S-2} & C_{N_S-2} \end{pmatrix} \begin{pmatrix} V_0 \\ V_1 \\ V_2 \\ \vdots \\ V_{N_S-2} \\ V_{N_S-1} \end{pmatrix}.$$

- In total there are $N_S - 2$ equations, but N_S variables. V_0 and V_{N_S-1} require **boundary conditions**.
- Differential equations can have infinite amount of solutions, boundary conditions impose constraints such that the solution is unique.
- Note boundary condition could violate the differential equation itself so does not guarantee the existence of the solution.

Boundary Conditions

- When solving PDE, we need to specify boundary conditions to ensure a well-posed numerical solution. The choice of boundary condition depends on the type of option being priced and the nature of the discretization grid.
- There are three types of boundary conditions that are popular in finance
  - Dirichlet boundary condition (Constraint on V)
  - Neumann boundary condition (Constraint on V')
  - Linear boundary condition (Constraint on V'')
- Which one to choose depends on what kind of option we are trying to price and what is the state variable of our discretization grid.
- We can also use a mixture of any of them
  - For example, European options: Dirichlet conditions at $S = 0$ and Neumann at $S = S_{\max}$.

Dirichlet Boundary Condition

- Specifies V at the boundary, e.g.,

$$V(b) = e^{-r(T-t)} \text{ payoff}(b).$$

- **Rationale:** when the spot is at the extreme wings, the intrinsic values determine the payoff, the volatility has little impact, or the payoff is really known at the boundary, e.g., knock-out options.
- In matrix form, we have

$$\begin{pmatrix} e^{-r(T-t)}\text{payoff}(S_0) \\ \partial V_1/\partial t \\ \partial V_2/\partial t \\ \vdots \\ \partial V_{N_S-2}/\partial t \\ e^{-r(T-t)}\text{payoff}(S_{N_S-1}) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ A_1 & B_1 & C_1 & 0 & \cdots & 0 \\ 0 & A_2 & B_2 & C_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 0 & A_{N_S-2} & B_{N_S-2} & C_{N_S-2} \\ 0 & 0 & 0 & \cdots & 0 & 1 \end{pmatrix} \begin{pmatrix} V_0 \\ V_1 \\ V_2 \\ \vdots \\ V_{N_S-2} \\ V_{N_S-1} \end{pmatrix}.$$

- Now our matrix M is square. And the solution will be unique if exists.

Neumann Boundary Condition

- Specifies the first derivative of V at the boundary:

$$\frac{\partial V}{\partial S} = c.$$

- **Rationale:** the volatility has little impact at the extreme cases and the payoff becomes linear at the wings and we know directly its derivative.
- With finite difference we have

$$V_1 - V_0 = c_0 \Delta S \Rightarrow V_0 = V_1 - c_0 \Delta S,$$

$$V_{N_S-1} - V_{N_S-2} = c_N \Delta S \Rightarrow V_{N_S-1} = V_{N_S-2} + c_N \Delta S.$$

- In matrix form, we have

$$\begin{pmatrix} c_0 \Delta S \\ \partial V_1/\partial t \\ \partial V_2/\partial t \\ \vdots \\ \partial V_{N_S-2}/\partial t \\ c_N \Delta S \end{pmatrix} = \begin{pmatrix} -1 & 1 & 0 & 0 & \cdots & 0 \\ A_1 & B_1 & C_1 & 0 & \cdots & 0 \\ 0 & A_2 & B_2 & C_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 0 & A_{N_S-2} & B_{N_S-2} & C_{N_S-2} \\ 0 & 0 & 0 & \cdots & -1 & 1 \end{pmatrix} \begin{pmatrix} V_0 \\ V_1 \\ V_2 \\ \vdots \\ V_{N_S-2} \\ V_{N_S-1} \end{pmatrix}.$$

- Again we have M in square form and the solution is unique if exists.

Linear Boundary Condition (Zero Curvature)

- The second derivative is 0:

$$\frac{\partial^2 V}{\partial S^2} = 0.$$

- **Rationale:** payoff becomes linear at the wings. But linear boundary condition leaves the calculation of the first derivative to the system, and allows the first derivative to be changing with respect to time.
- Note that we cannot use central difference for V_0 since it's at the boundary, so we say

$$\frac{\partial^2 V_0}{\partial S^2} \approx \frac{\frac{\partial V_1}{\partial S} - \frac{\partial V_0}{\partial S}}{\Delta S} \approx \frac{\frac{V_2 - V_1}{\Delta S} - \frac{V_1 - V_0}{\Delta S}}{\Delta S} = \frac{V_2 + V_0 - 2V_1}{\Delta S^2}.$$

- Essentially, it is the central difference for V_1.
- Similarly for V_{N_S}:

$$\frac{\partial^2 V_{N_S}}{\partial S^2} \approx \frac{\frac{V_{N_S-1} - V_{N_S-2}}{\Delta S} - \frac{V_{N_S-2} - V_{N_S-3}}{\Delta S}}{\Delta S} = \frac{V_{N_S-1} + V_{N_S-3} - 2V_{N_S-2}}{\Delta S^2}.$$

Linear Boundary Condition

- The system of ODE can be expressed as

$$\begin{pmatrix} 0 \\ \partial V_1/\partial t \\ \partial V_2/\partial t \\ \vdots \\ \partial V_{N_S-2}/\partial t \\ 0 \end{pmatrix} = \begin{pmatrix} 1 & -2 & 1 & 0 & \cdots & 0 \\ A_1 & B_1 & C_1 & 0 & \cdots & 0 \\ 0 & A_2 & B_2 & C_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 0 & A_{N_S-2} & B_{N_S-2} & C_{N_S-2} \\ 0 & \cdots & 1 & -2 & 1 & \end{pmatrix} \begin{pmatrix} V_0 \\ V_1 \\ V_2 \\ \vdots \\ V_{N_S-2} \\ V_{N_S-1} \end{pmatrix}.$$

- Again we have M in square form and the solution is unique if exists.

Lecture 11

Ordinary Differential Equations (ODEs)

- An ordinary differential equation involves derivatives with respect to a single variable.
- Some ODEs admit closed-form solutions e.g., the following equation, while others require numerical methods for approximation:

$$y' = y \Rightarrow \frac{dy}{dx} = y \Rightarrow \frac{dy}{y} = dx \Rightarrow \int \frac{dy}{y} = \int dx \Rightarrow \ln y = x + C$$

$$y = Ke^x, \quad \text{where } K = e^C.$$

- A summary of ODEs with closed-form solutions is provided in the Appendix (though not directly relevant for our PDE pricer, it is useful for advanced modeling).
- Consider the initial value problem of the first-order ODE

$$y'(x) = f(x, y), \quad y(x_0) = y_0.$$

- Our focus is on numerical methods, i.e., approximate $y(x)$ at discrete points

$$x_j = x_0 + j \cdot h$$

with step size h, starting at x_0.

Euler Method

- The simplest yet widely used numerical method for solving ODEs.
- It is a first-order numerical integration method, meaning its error scales as $O(h)$.
- The derivation:

$$y' = \frac{dy}{dx} = \frac{y(x+h) - y(x)}{h} + O(h)$$

$$y' = f(x, y)$$

$$\frac{y(x+h) - y(x)}{h} = f(x, y) + O(h)$$

$$y(x+h) = y(x) + hf(x, y) + O(h^2)$$

- The scheme: for $x_j = x_0 + jh$:

$$y_{j+1} = y_j + hf(x_j, y_j), \quad j = 0, 1, \ldots$$

- The local truncation error is $O(h^2)$. The global error accumulates as $O(h)$, making it first-order method.

Euler Method – Stability

- Stability of the Euler method
  - Euler's method can be unstable for stiff ODEs, requiring more advanced techniques such as implicit methods or adaptive step-size control.
- To analyze the stability of Euler's method, consider the simple linear ODE:

$$y' = -ky, \quad k \geq 0.$$

- Applying Euler's method, the numerical solution is computed iteratively as $\hat{y}_i$:

$$\hat{y}_{i+1} = \hat{y}_i + h(-k\hat{y}_i) = (1 - hk)\hat{y}_i.$$

- The condition above ensures the error decreases at each iteration.
- Euler's method is conditionally stable, meaning it requires a sufficiently small time step h to ensure stability.
- For stiff ODEs (where k is large), implicit methods such as Backward Euler or Runge-Kutta methods are preferred for better stability.

Implicit Euler method for unconditional stability

- Instead of using the forward (right) difference, we now use the backward (left) difference to approximate the derivative:

$$\frac{y(x+h) - y(x)}{h} \approx f(x+h, y(x+h)).$$

- The order of implicit Euler method is $O(h)$.
- By Taylor expansion, we have

$$y(x+h) = y(x) + hy'(x) + \frac{h^2}{2} y''(x) + O(h^3).$$

- Since $y' = f(x, y)$, we substitute $y'(x) = f(x, y)$ and $y''(x) = f_x(x, y)$:

$$y(x+h) = y(x) + hf(x, y) + \frac{h^2}{2} f_x(x, y) + O(h^3).$$

- Similarly, by Taylor expansion,

$$f(x+h, y(x+h)) = f(x, y) + hf_x(x, y(x+h)) + O(h^2).$$

Implicit method and its stability

- Putting two equations together

$$y(x+h) - y(x) = hf(x, y) + \frac{h^2}{2} f_x(x, y) + O(h^3),$$

$$f(x+h, y(x+h)) = f(x, y) + hf_x(x, y(x+h)) + O(h^2).$$

- We have

$$y(x+h) - y(x) = hf(x+h, y(x+h)) + O(h^2).$$

- Applying this to the same test equation $y' = -ky$, we obtain the implicit Euler update

$$\hat{y}_{i+1} - \hat{y}_i = -hk\hat{y}_{i+1} \Rightarrow \hat{y}_{i+1} = \frac{1}{1+hk}\hat{y}_i.$$

- Stability analysis:
  - Since $\left|\frac{1}{1+hk}\right| < 1$ is always true, this scheme is unconditionally stable, meaning it remains stable for any choice of step size h.
- We will revisit implicit methods later when discussing their computational trade-offs.

Mid-point method

- Recall we have

$$y(x+h) - y(x) = hf(x, y) + \frac{h^2}{2} f_x(x, y) + O(h^3),$$

$$f(x+h, y(x+h)) = f(x, y) + hf_x(x, y(x+h)) + O(h^2).$$

- Observed that

$$f\left(x + \frac{h}{2}, y\left(x + \frac{h}{2}\right)\right) = f(x, y) + \frac{h}{2} f_x(x, y(x)) + O(h^2).$$

- We have

$$y(x+h) - y(x) = hf\left(x + \frac{h}{2}, y\left(x + \frac{h}{2}\right)\right) + O(h^3).$$

- The mid-point method is a second-order method with global error $O(h^2)$.

Heun's Method (Runge Kutta 2nd order)

- In mid-point method, $y(x + h/2)$ is not on the grid. We modify it further:

$$y\left(x + \frac{h}{2}\right) = y(x) + \frac{h}{2} y'(x) + O(h^2) = y(x) + \frac{h}{2} f(x, y) + O(h^2).$$

- Then, we can modify the mid-point method by

$$y(x+h) - y(x) = hf\left(x + \frac{h}{2}, y\left(x + \frac{h}{2}\right)\right) + O(h^3)$$

$$= hf\left(x + \frac{h}{2}, y(x) + \frac{h}{2} f(x, y) + O(h^2)\right) + O(h^3)$$

$$= h\left(f\left(x + \frac{h}{2}, y(x)\right) + \frac{h}{2} f(x, y)\right) + O(h^3).$$

- The method is Runge-Kutta with 2nd order:

$$y_{j+1} = y_j + hf\left(x_j + \frac{h}{2}, y_j + \frac{h}{2} f(x_j, y_j)\right).$$

Generalization to higher-order RK methods

- In second-order RK method, the function values are evaluated at the middle point for introducing additional information.
- To derive 4th-order RK method, we seek a formula of the form:

$$y_{j+1} = y_j + a_1 K_1 + a_2 K_2 + a_3 K_3 + a_4 K_4,$$

where $\{K_i\}_{i=1}^4$ are function values at various points.

- Define intermediate steps:

$$K_1 = f(x_j, y_j),$$

$$K_2 = f\left(x_j + \frac{h}{2}, y_j + \frac{h}{2} K_1\right),$$

$$K_3 = f\left(x_j + \frac{h}{2}, y_j + \frac{h}{2} K_2\right),$$

$$K_4 = f(x_j + h, y_j + hK_3).$$

- Then, the formula of 4th-order RK method is

$$y_{j+1} = y_j + \frac{h}{6}(K_1 + 2K_2 + 2K_3 + K_4).$$

The Adams methods

- The Adams methods use multiple previous time steps to compute the next value, improving accuracy while reducing function evaluations.
- For an ODE, integrating from x_j to x_{j+1}:

$$y_{j+1} = y_j + \int_{x_j}^{x_{j+1}} f(x, y)\, dx.$$

- Adams-Bashforth Formula: Using linear interpolation for $f(x, y)$ using two previous values:

$$f(x) \approx f_j + \frac{x - x_j}{h}(f_j - f_{j-1}), \quad \text{where } f_j = f(x_j, y(x_j)).$$

- Integrating over $[x_j, x_{j+1}]$:

$$\int_{x_j}^{x_{j+1}} f(x)\, dx = hf_j + \frac{h}{2}(f_j - f_{j-1}) = \frac{3}{2}hf_j - \frac{1}{2}hf_{j-1}.$$

- Thus, the two-step Adams-Bashforth method is given by:

$$y_{j+1} = y_j + \frac{3}{2}hf_j - \frac{1}{2}hf_{j-1}.$$

The implicit Adams method

- Adams-Moulton Formula: Using linear interpolation for $f(x, y)$ using the following values f_j, f_{j+1}, which gives

$$f(x) \approx f_j + \frac{x - x_j}{h}(f_{j+1} - f_j).$$

- Then, the integral

$$\int_{x_j}^{x_{j+1}} f(x, y(x))\, dx = hf_j + \frac{h}{2}(f_{j+1} - f_j) = \frac{h}{2}(f_j + f_{j+1}).$$

- The one-step Adams-Moulton is

$$y_{j+1} = y_j + \frac{h}{2}(f_j + f_{j+1}) = y_j + \frac{h}{2}(f(x_j, y_j) + f(x_{j+1}, y_{j+1})).$$

- The adams-Moulton method is an implicit method, which requires solve an equation.

More Discretization Schemes

- **Adaptive RK method**
  - Uses variable step sizes to maintain accuracy while reducing computation time.
  - Automatically adjusts step size based on error estimates.
- **Predictor-Corrector Methods**
  - Combine an explicit predictor step with an implicit corrector step.
  - Improve stability and accuracy over explicit methods.
  - Frequently used in stiff problems where standard explicit methods struggle.
- **Choice of Scheme**
  - Lower-order schemes are generally more robust. For example, Euler's Method is often used due to its simplicity.
  - For Monte Carlo simulations, we typically use Euler's method or at most second-order schemes, as higher-order methods do not provide significant benefits given the randomness in stochastic processes.

Solving our ODEs

- Recall that we need to solve a system of ODEs

$$\begin{pmatrix} b(0) \\ \partial V_1/\partial t \\ \partial V_2/\partial t \\ \vdots \\ \partial V_{N_S-2}/\partial t \\ b(N_S-1) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ A_1 & B_1 & C_1 & 0 & \cdots & 0 \\ 0 & A_2 & B_2 & C_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & 0 & A_{N_S-2} & B_{N_S-2} & C_{N_S-2} \\ 0 & 0 & 0 & \cdots & 0 & 1 \end{pmatrix} \begin{pmatrix} V_0 \\ V_1 \\ V_2 \\ \vdots \\ V_{N_S-2} \\ V_{N_S-1} \end{pmatrix}.$$

- There are totally $N_S - 2$ ODEs, and b_0, b_{N_S-1} are boundary condition's coefficients.
- Discretizing the time dimension
- First we divide the temporal dimension into subdivisions (time steps):

$$t_0 = 0,\ t_1, \dots, t_{N_T-1} = T.$$

- For simplicity, we take equal division so our time interval is Δt.

Discretizing the time dimension

- Let $V_{i,j}$ denote the derivative price function $V(S,t)$ at (S_i, t_j). Then, we have

$$\begin{pmatrix} b_{(0,j)} \\ (\partial V/\partial t)_{1,j} \\ (\partial V/\partial t)_{2,j} \\ \vdots \\ (\partial V/\partial t)_{N_S-2,j} \\ b_{(N_S-1,j)} \end{pmatrix} = M \begin{pmatrix} V_{(0,j)} \\ V_{(1,j)} \\ V_{(2,j)} \\ \vdots \\ V_{(N_S-2,j)} \\ V_{(N_S-1,j)} \end{pmatrix}.$$

- To discretize the time derivatives, the simplest way is to use Euler scheme:

$$\text{Left difference: } \frac{\partial V_{i,j}}{\partial t} \approx \frac{V_{i,j} - V_{i,j-1}}{\Delta t},$$

$$\text{Right difference: } \frac{\partial V_{i,j}}{\partial t} \approx \frac{V_{i,j+1} - V_{i,j}}{\Delta t}.$$

Explicit Euler Scheme

- Now considering using the left difference to approximate derivatives in time domain, we get the explicit Euler scheme:

$$\frac{1}{\Delta t} \begin{pmatrix} \Delta t \cdot b_{(0,j)} \\ V_{(1,j)} - V_{(1,j-1)} \\ V_{(2,j)} - V_{(2,j-1)} \\ \vdots \\ V_{(N_S-2,j)} - V_{(N_S-2,j-1)} \\ \Delta t \cdot b_{(N_S-1,j)} \end{pmatrix} = M \begin{pmatrix} V_{(0,j)} \\ V_{(1,j)} \\ V_{(2,j)} \\ \vdots \\ V_{(N_S-2,j)} \\ V_{(N_S-1,j)} \end{pmatrix}.$$

- The reason it is explicit is that we know our V from the back.
- The linear system is an explicit function from V_j to V_{j-1}.
- The derivative instrument's payoff value at time T is the value of the derivative instrument, similar to the tree model. We call that our final value.

Backward calculation in time

- Let the vector V_j denote the solution at time slice t_j, i.e.,

$$V_j = (V_{(0,j)}, V_{(1,j)}, V_{(2,j)}, \dots, V_{(N_S-2,j)}, V_{(N_S-1,j)})^T.$$

- The explicit Euler scheme reads

$$\begin{pmatrix} -\Delta t \cdot b_{(0,j)} \\ V_{(1,j-1)} \\ V_{(2,j-1)} \\ \vdots \\ V_{(N_S-2,j-1)} \\ -\Delta t \cdot b_{(N_S-1,j)} \end{pmatrix} = (D - \Delta t M) V_j,$$

where

$$D = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ 0 & 0 & 0 & \cdots & 0 \end{pmatrix}.$$

- Note that (63) only for $V_{(1,j-1)}$ to $V_{(N_S-2,j-1)}$ in V_{j-1}, the two boundary points $V_{(0,j-1)}$ and $V_{(N_S-1,j-1)}$ still rely on the boundary conditions.

Explicit Euler Scheme

- Stability condition of explicit Euler scheme is

$$\begin{cases} \Delta t \le \left(\frac{\Delta S}{\sigma S_{\max}}\right)^2 \\ \Delta S \le \frac{\sigma^2 S_{\min}}{|r-q|} \end{cases}$$

- The first condition on Δt is derived by Von Neumann Stability Analysis, which comes from the diffusion term $\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}$, ensuring that numerical errors does not blow up due to the second derivatives.
- The second condition on ΔS comes from the Courant-Friedrichs-Lewy (CFL) condition for convection-dominated PDEs, which results from the convection term $(r-q)S\frac{\partial V}{\partial S}$, ensuring that the scheme can resolve the drift term without instability.
- Error of explicit scheme is $O(\Delta t, \Delta S^2)$
  - First-order in time and second-order in spot dimension.
- The time slice V_0 gives us the option price for each spot at time 0. For any spot that does not lie on the points, we can use linear interpolation.

Implicit Euler Scheme for absolute stability

- Substituting the right difference

$$\frac{\partial V_{i,j}}{\partial t} \approx \frac{V_{i,j+1} - V_{i,j}}{\Delta t}$$

to (58), we get the implicit Euler scheme:

$$\frac{1}{\Delta t} \begin{pmatrix} \Delta t \cdot b_{(0,j)} \\ V_{(1,j+1)} - V_{(1,j)} \\ V_{(2,j+1)} - V_{(2,j)} \\ \vdots \\ V_{(N_S-2,j+1)} - V_{(N_S-2,j)} \\ \Delta t \cdot b_{(N_S-1,j)} \end{pmatrix} = M \begin{pmatrix} V_{(0,j)} \\ V_{(1,j)} \\ V_{(2,j)} \\ \vdots \\ V_{(N_S-2,j)} \\ V_{(N_S-1,j)} \end{pmatrix}.$$

Implicit Euler scheme

- Now since we are calculating backward, we know V_{j+1} and want to solve for V_j so:

$$V_{j+1} = (D + \Delta t M) V_j,$$

where D is defined as before.
- This is an implicit function of V_j, therefore

$$V_j = (D + \Delta t M)^{-1} V_{j+1}.$$

- Implicit Euler scheme is unconditionally stable.
- Same error of implicit scheme is $O(\Delta t, \Delta S^2)$ — same as explicit Euler.
- The implementation involves an inverse of the tri-diagonal matrix; we can use an LU decomposition.

Solving Tri-Diagonal Systems Using the Thomas Algorithm

- Algorithm efficiently solves a tri-diagonal system of linear equations:

$$\begin{pmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \cdots & 0 \\ 0 & a_3 & b_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & b_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_n \end{pmatrix}.$$

- LU Decomposition for Tri-Diagonal Matrices $(LU)x = d$,

$$L = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ \ell_2 & 1 & 0 & \cdots & 0 \\ 0 & \ell_3 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ell_n & \cdots & 1 \end{pmatrix}, \quad U = \begin{pmatrix} b_1' & c_1 & 0 & \cdots & 0 \\ 0 & b_2' & c_2 & \cdots & 0 \\ 0 & 0 & b_3' & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & c_{n-1} \\ 0 & 0 & 0 & \cdots & b_n' \end{pmatrix}.$$

Thomas Algorithm

- The equations for L and U:

$$\ell_i = \frac{a_i}{b_{i-1}'}, \quad b_i' = b_i - \ell_i c_{i-1}, \quad d_i' = d_i - \ell_i d_{i-1}'.$$

- Forward elimination (Computing LU and solving $L^{-1}d$):
  - Start with $b_1' = b_1$ and $d_1' = d_1$.
  - Compute $\ell_2, b_2', d_2', \dots, \ell_n, b_n', d_n'$.
- Backward substitution: Solving $Ux = d'$:
  - Solve for x_n first: $x_n = \frac{d_n'}{b_n'}$.
  - Then iteratively solve for the previous values:

$$x_i = \frac{d_i' - c_i x_{i+1}}{b_i'}, \quad i = n-1, n-2, \dots, 1.$$

- The Thomas Algorithm efficiently solves tri-diagonal systems in $O(n)$ time.
- It uses LU decomposition without explicitly storing L and U.

Douglas Scheme

- The Douglas scheme combines the Explicit Euler and Implicit Euler methods to form a weighted numerical scheme for solving the PDE efficiently.
- Given a time step $j \to j-1$, we write the Explicit Euler and Implicit Euler update equations as:

$$V_{j-1} = (D - \Delta t M)V_j,$$

$$(D + \Delta t M)V_{j-1} = V_j.$$

- Douglas scheme is to do a weighted sum of the two schemes:

$$wV_{j-1} + (1-w)(D + \Delta t M)V_{j-1} = [w(D - \Delta t M) + (1-w)I]V_j.$$

- Rearranging:

$$[I_w + (1-w)\Delta t M]V_{j-1} = [I_{1-w} - w\Delta t M]V_j,$$

where I_w and I_{1-w} differ from the identity matrix only at the first and last rows (replaced by w and $1-w$).

Douglas Scheme

- The solution of Douglas scheme:

$$V_{j-1} = [I_w + (1-w)\Delta t M]^{-1}[I_{1-w} - w\Delta t M]V_j.$$

- The parameter $0 \le w \le 1$ balances stability and accuracy.
  - $w = 1$: the scheme reduces to Explicit Euler.
  - $w = 0$: the scheme reduces to Implicit Euler.
- Why Douglas scheme?
  - The truncation error for Douglas scheme is

$$O\left(\frac{1}{2}\Delta t - \frac{1}{12}\Delta S^2 - w\Delta t,\ \Delta S^4,\ \Delta t^2\right).$$

  - If we choose $w = \frac{1}{2} - \frac{\Delta S^2}{12\Delta t}$, we can achieve $O(\Delta t^2, \Delta S^4)$.

Crank-Nicholson Scheme

- When the weight $w = \frac{1}{2}$, it is the famous Crank-Nicholson scheme widely used in PDE:

$$[I_{1/2} + \frac{1}{2}\Delta t M]V_{j-1} = [I_{1/2} - \frac{1}{2}\Delta t M]V_j.$$

- Crank-Nicholson is unconditionally stable.
- Intuitive explanation on stability property of different methods:
  - Consider the PDE $y' = -ky$:

$$\hat{y}_{i+1} = (1 - hk)\hat{y}_i \quad \text{(explicit)},$$

$$\hat{y}_{i+1} = \frac{1}{1+hk}\hat{y}_i \quad \text{(implicit)},$$

$$\hat{y}_{i+1} = \frac{2-hk}{2+hk}\hat{y}_i \quad \text{(Crank-Nicholson)}.$$

Crank-Nicholson Scheme

- Unconditionally stable: Can take large time steps without instability.
- Second-order accuracy in both time, $O(\Delta t^2, \Delta S^2)$.
- Popular in finance for solving the Black-Scholes equation.
- Drawbacks of Crank-Nicholson
  - Oscillations when the payoff is discontinuous (e.g., options with sharp payoff jumps).
  - When the stability condition for explicit scheme does not meet, the oscillations may appear

$$\Delta S \le \frac{\sigma^2 S_{\min}}{|r-q|}.$$

  - Issue when volatility is low and drift is high $\to$ Requires implicit smoothing or alternative techniques.

Richardson Extrapolation

- Richardson Extrapolation is a numerical technique that improves the accuracy of finite difference schemes by combining solutions from two different grids.
- Similar to Richardson Extrapolation applied to finite difference, if we have two solutions V_1 and V_2 whose grid satisfies

$$\frac{\Delta t_1}{\Delta S_1^2} = \frac{\Delta t_2}{\Delta S_2^2}.$$

- Extrapolation Formula: Given V_1 on ΔS_1 and V_2 on ΔS_2, we obtain a new solution:

$$V = \frac{\Delta S_2^2 V_1 - \Delta S_1^2 V_2}{\Delta S_2^2 - \Delta S_1^2}.$$

- ΔS_1 is a coarser grid and ΔS_2 is a finer grid.
- This improves the accuracy to $O(\Delta t^2, \Delta S^3)$ from the original $O(\Delta t, \Delta S^2)$.

Proof of Richardson Extrapolation for PDEs

- The discretization error for a second-order accurate method (like Crank-Nicholson) is for two constants C_1, C_2:

$$V_1 = V + C_1\Delta S_1^2 + C_2\delta S_1^3 + O(\Delta S_1^4),$$

$$V_2 = V + C_1\Delta S_2^2 + C_2\delta S_2^3 + O(\Delta S_2^4).$$

- To eliminate the leading error term $O(\Delta S^2)$, we take a linear combination

$$V_{\text{ext}} = \alpha V_1 + \beta V_2.$$

- Substituting the error expansion, we have

$$(\alpha + \beta)V + C_1(\alpha\Delta S_1^2 + \beta\Delta S_2^2) + C_2(\alpha\delta S_1^3 + \beta\delta S_2^3) + O(\Delta S^4).$$

- Choosing α and β such that

$$\alpha + \beta = 1, \quad C_1(\alpha\Delta S_1^2 + \beta\Delta S_2^2) = 0,$$

the solution is

$$\alpha = \frac{\Delta S_2^2}{\Delta S_2^2 - \Delta S_1^2}, \quad \beta = -\frac{\Delta S_1^2}{\Delta S_2^2 - \Delta S_1^2}.$$

Change Of Variable

- So far we have been using equally spaced discretization in time t and spot S. Our derivation can be generalized to non-regular discretization.
- One general guideline to construct the PDE grid is to place more points at the regions where V is sensitive to the spot: e.g., near the current spot or option strike.
- The choice of the spot as state variable is not ideal when we use equal spacing.
- Basically we need a lot more points at the center than at the wings, but with equal spacing discretization, we need to add a lot of points at the wings which is unnecessary.
- A more natural choice of state variable is the log spot

$$X = \log S.$$

- Linear spacing in X means exponential spacing in S.
- This naturally densifies the grid around S_0 while keeping fewer points in unnecessary regions.

Rewriting the Black-Scholes PDE in Terms of X

- We now express our PDE in terms of X instead of S.

$$\frac{\partial V}{\partial S} = \frac{\partial V}{\partial X}\frac{\partial X}{\partial S} = \frac{1}{S}\frac{\partial V}{\partial X},$$

$$\frac{\partial^2 V}{\partial S^2} = \frac{\partial}{\partial S}\left(\frac{1}{S}\frac{\partial V}{\partial X}\right) = \frac{1}{S^2}\frac{\partial^2 V}{\partial X^2} - \frac{1}{S^2}\frac{\partial V}{\partial X}.$$

- Then, the Black-Scholes PDE becomes:

$$\frac{\partial V}{\partial t} + \left(r - q - \frac{\sigma^2}{2}\right)\frac{\partial V}{\partial X} + \frac{\sigma^2}{2}\frac{\partial^2 V}{\partial X^2} - rV = 0.$$

- Note that the coefficients do not depend on S now. The rest is the same as in the previous discussion, only coefficients changes:

$$A = \frac{r - q - \sigma^2/2}{2\Delta X} - \frac{\sigma^2}{2\Delta X^2}, \quad B = r + \frac{\sigma^2}{\Delta X^2}, \quad C = -\frac{r - q - \sigma^2/2}{2\Delta X} - \frac{\sigma^2}{2\Delta X^2}.$$

Appendix: ODE's with Closed Form Solution

- **Separable ODE**
  - A separable ODE can be written as $\frac{dy}{dx} = g(x)h(y)$. The solution is therefore very simple: integrating both sides of

$$\frac{dy}{h(y)} = g(x)dx \Rightarrow \int \frac{dy}{h(y)} = \int g(x)dx.$$

- There are cases that an ODE appears non-separable, but is in fact separable after a change of variable.
- For example,

$$y' = \frac{x-y}{x+y}.$$

Apply change of variable $z = x + y$, the right hand side becomes $\frac{2x-z}{z}$. The left hand side is

$$\frac{dy}{dx} = \frac{d(z-x)}{dx} = \frac{dz}{dx} - 1.$$

- So we have
$$\frac{dz}{dx} = \frac{2x}{z}.$$
$$\int z\,dz = \int 2x\,dx + c \Rightarrow \frac{1}{2}z^2 = x^2 + c.$$
- The solution is trivial:
$$y^2 + 2xy - x^2 = c.$$

First-order linear ODE

- A first order linear ODE has the form
$$\frac{dy}{dx} + P(x)y = Q(x).$$
- The standard approach to solve it is to find a suitable integrating factor $I(x)$, such that
$$I(x)\left(y' + P(x)y\right) = (I(x)y)'.$$
- If holds, the original ODE becomes $(I(x)y)' = I(x)Q(x)$ and we have
$$I(x)y = \int I(x)Q(x)dx \Rightarrow y(x) = \frac{\int I(x)Q(x)dx}{I(x)}.$$
- For this to hold, we need to have
$$I'(x) = I(x)P(x) \Rightarrow \frac{dI(x)}{I(x)} = P(x)dx \Rightarrow I(x) = e^{\int P(x)dx}.$$
- Applying the above technique we can derive easily that the solution of
$$y' + \frac{y}{x} = \frac{1}{x^2}$$
is
$$y = \frac{\ln x + c}{x}.$$

Homogeneous linear ODE

- A homogeneous linear ODE is a second order linear ODE of the form
$$a(x)y'' + b(x)y' + c(x)y = 0.$$
- It is easy to show that if y_1 and y_2 are linearly independent solutions to the ODE, then any
$$y(x) = c_1y_1(x) + c_2y_2(x)$$
for arbitrary constants c_1, c_2 is also a solution to the ODE.
- When a, b, and c ($a \neq 0$) are constants, the homogeneous linear ODE
$$ay'' + by' + cy = 0$$
has closed form solutions. Let r_1 and r_2 be the roots of the quadratic equation
$$ar^2 + br + c = 0, \quad r_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$
- The closed form solution is
$$y(x) = \begin{cases} c_1e^{r_1x} + c_2e^{r_2x}, & \text{if } b^2 - 4ac > 0, \\ c_1e^{rx} + c_2xe^{rx}, & \text{if } b^2 - 4ac = 0,\ r_1 = r_2 = r = -\frac{b}{2a}, \\ e^{\alpha x}(c_1\cos\beta x + c_2\sin\beta x), & \text{if } b^2 - 4ac < 0,\ r_{1,2} = \alpha \pm i\beta. \end{cases}$$

Non-homogeneous linear ODE

- A non-homogeneous linear ODE is a second order linear ODE of the form
$$a(x)y'' + b(x)y' + c(x)y = d(x).$$
- It does not have closed form solution even if a, b, c are constants. However, for constant coefficients we can find a particular solution $y_p(x)$ for
$$ay'' + by' + cy = d(x).$$
- It can be shown that
$$y(x) = y_g(x) + y_p(x)$$
is a general solution for (88), where $y_g(x)$ is a general solution for the homogeneous linear ODE (87). When $d(x)$ is a simple polynomial, the particular solution is often a polynomial of the same degree.

Lecture 12

Optimization Methods in Finance

- **Mathematical Definition:**
  Optimization involves finding the maximum or minimum of a function that depends on one or more variables.
- **Role in Quantitative Finance:**
  - Enables systematic decision-making by identifying optimal solutions within constraints.
  - Helps practitioners make data-driven choices to enhance financial performance.
- **Key Applications:**
  - Model Calibration: adjusting parameters to align with observed market data.
  - Derivative Pricing and Hedging: computing optimal hedge ratios using stochastic optimization.
  - Portfolio Optimization: constructing portfolios to maximize return or minimize risk.

Problem Formulation

- General form:
$$\min_x f(x), \quad \text{subject to } x \in A$$
- $f : \mathbb{R}^n \to \mathbb{R}$ is the objective function.
- A is the feasible region.
- Maximization is equivalent to minimizing $-f(x)$.
- Categories:
  - Unconstrained optimization:
$$A = \mathbb{R}^n$$
  - Constrained optimization:
    A is a proper subset of $\mathbb{R}^n$
- Types of constraints:
  - Equality: $g_i(x) = 0,\ 1 \leq i \leq m$
  - Inequality: $h_j(x) \leq 0,\ 1 \leq j \leq p$
  - Set constraints: e.g. $x \in \{0,1\}^n$

Special Classes of Optimization Problems

- Linear Optimization (LP)
  - Objective and constraints are linear
$$\min c^\top x, \quad \text{subject to } Ax \leq b$$
  - Example: supply chain optimization
- Nonlinear Optimization
  - Objective or constraints are nonlinear
  - Example: training neural networks
- Convex Optimization
  - Objective is convex, feasible region is convex
  - Example: Lasso regression
- Integer / Mixed-Integer Programming
  - Some variables must be integers
  - Example: scheduling and logistics

Definition of a Minimizer

- Consider:
$$\min_{x \in A} f(x)$$
- Global minimizer x^*:
$$f(x^*) \leq f(x), \quad \forall x \in A$$
- Local minimizer:
  There exists $\epsilon > 0$ such that
$$f(x^*) \leq f(x), \quad \forall x \in A \cap B_\epsilon(x^*)$$
- Stationary point:
$$f'(x^*) = 0$$
- Classification:
  - Local minimum if $f''(x^*) > 0$
  - Local maximum if $f''(x^*) < 0$
  - Saddle point if $f''(x^*) = 0$

Introduction to Convexity

- Why convexity matters:
  - Convex problems are easier to solve
  - Strong theoretical guarantees
  - Widely used in finance and economics
- Convex function:
$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda)f(x_2)$$
- Strict convexity:
$$<$$
- Convex set:
$$\lambda x_1 + (1-\lambda)x_2 \in \Omega$$

Recognition of Convex Differentiable Functions

- Linear combinations preserve convexity
- Composition $f(Mx + b)$ is convex
- Gradient:
$$\nabla f(x)$$
- Hessian:
$$H(x) = \nabla^2 f(x)$$
- First-order condition:
$$f(y) \geq f(x) + \nabla f(x)^\top (y - x)$$
- Second-order condition:
$$H(x) \succeq 0$$

Global vs Local Minima in Convex Problems

- For convex f: Local minimum $\Rightarrow$ global minimum
- If strictly convex: Global minimum is unique

Stationary Points in Convex Optimization

- If $\nabla f(x^*) = 0$ and f is convex
- Then x^* is a global minimizer

Descent Direction

- If there exists d such that
$$\nabla f(x)^\top d < 0$$
- Then for small λ:
$$f(x + \lambda d) < f(x)$$
- Taylor expansion:
$$f(x + \lambda d) = f(x) + \lambda \nabla f(x)^\top d + O(\lambda^2)$$
- Gradient method:
  - Find direction d_k
  - Choose step size λ^k
  - Update:
$$x^{k+1} = x^k + \lambda^k d^k$$

Examples of Descent Directions

- Steepest descent:
$$x^{k+1} = x^k - \lambda^k \nabla f(x^k)$$
- Newton's method:
$$x^{k+1} = x^k - \lambda^k (H(x^k))^{-1} \nabla f(x^k)$$
- Diagonally scaled steepest descent:
$$x^{k+1} = x^k - \lambda^k D^k \nabla f(x^k)$$
where D^k approximates $(H(x^k))^{-1}$.
- Modified Newton's method:
$$x^{k+1} = x^k - \lambda^k (H(x^0))^{-1} \nabla f(x^k)$$
where x^0 is the initial point.

Steepest Descent

- Algorithm:
  - Initialize x^0, $k = 0$
  - $d^k = -\nabla f(x^k)$
  - If $\|d^k\| \le \epsilon$, stop
  - Compute step-size:
$$\lambda^k = \arg\min_\lambda f(x^k + \lambda d^k)$$
  - Update:
$$x^{k+1} = x^k + \lambda^k d^k$$
  - $k \leftarrow k + 1$

Properties of Steepest Descent

- Monotonic decrease:
$$f(x^{k+1}) < f(x^k) < \cdots < f(x^0)$$
- Sequence $\{x^k\}$ may not converge, but has cluster points.
- Cluster point definition:
There exists a subsequence converging to x^*.
- Example:
$$x^k = (-1)^k + \frac{1}{k+1}$$
- If f is convex, any cluster point is a global minimum.
- Cost per iteration:
  - Compute $\nabla f(x^k)$
  - Solve line search problem

Proof of L-Smoothness

- Define:
$$\phi(t) = f(y - t(y - x)), \quad t \in (0, 1)$$
- Then:
$$\phi'(t) = \nabla f(y - t(y - x))^\top (y - x)$$
- Hence:
$$f(y) - f(x) = \int_0^1 \nabla f(y - t(y - x))^\top (y - x)\, dt$$
- Split:
$$= \nabla f(x)^\top (y - x) + \int_0^1 (\nabla f(y - t(y - x)) - \nabla f(x))^\top (y - x)\, dt$$
- Using Lipschitz condition:
$$f(y) \le f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|^2$$

Convergence Rate

- Convergence rate definition:
$$\lim_{k\to\infty} \frac{z_{k+1} - z^*}{(z_k - z^*)^p} = \beta$$
- L-smoothness:
$$\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\|$$
- Implies:
$$f(y) \le f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|^2$$
- μ-strong convexity:
$$f(y) \ge f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|^2$$

Important Inequalities

- Smoothness step:
$$f\left(x - \frac{1}{L}\nabla f(x)\right) \le f(x) - \frac{1}{2L}\|\nabla f(x)\|^2$$
- Therefore:
$$f(x) - f(x^*) \ge \frac{1}{2L}\|\nabla f(x)\|^2$$
- Strong convexity gives:
$$f(x) - f(x^*) \le \frac{1}{2\mu}\|\nabla f(x)\|^2$$

Linear Convergence

- If f is L-smooth and μ-strongly convex:
$$\|x^{k+1} - x^*\|^2 \le (1 - \lambda\mu)\|x^k - x^*\|^2$$
- Convergence is geometric (linear rate).

Step-size λ

- Exact line search:
$$\min_\lambda f(x^k + \lambda d^k)$$
- Restricted search:
$$\lambda \in (0, \lambda_0]$$
- Diminishing step-size:
$$\lambda^k \to 0, \quad \sum_k \lambda^k = \infty$$

Bisection Line Search

- Solve:
$$\lambda^* = \arg\min_\lambda f(x + \lambda d)$$
- Algorithm:
  - Initialize $\lambda_L = 0$, $\lambda_U = \lambda_0$
  - $\lambda = (\lambda_L + \lambda_U)/2$
  - If $h'(\lambda) > 0$, set $\lambda_U = \lambda$
  - If $h'(\lambda) < 0$, set $\lambda_L = \lambda$
  - Repeat until convergence

Armijo Rule

- Inexact line search (efficient)
- Step-size must satisfy:
$$f(x^k - \lambda\nabla f(x^k)) \le f(x^k) - c_1\lambda\|\nabla f(x^k)\|^2$$
$$f(x^k - \lambda\nabla f(x^k)) \ge f(x^k) - c_2\lambda\|\nabla f(x^k)\|^2$$
- Typically $0 < c_1 < 0.5 < c_2 < 1$
- Backtracking:
  - Start with $\lambda = \lambda_0$
  - Reduce $\lambda \leftarrow \beta\lambda$, $\beta \in (0, 1)$
  - Repeat until condition holds

Newton's Method

- Second-order Taylor approximation:
$$f(x) \approx f(x^k) + \nabla f(x^k)^\top (x - x^k) + \frac{1}{2}(x - x^k)^\top H(x^k)(x - x^k)$$
- Solve:
$$\nabla f(x^k) + H(x^k)(x - x^k) = 0$$
- Newton direction:
$$d^k = -(H(x^k))^{-1}\nabla f(x^k)$$
- Algorithm:
  - Initialize x^0, $k = 0$
  - Compute d^k
  - If $\|d^k\| \le \epsilon$, stop
  - Update:
$$x^{k+1} = x^k + d^k$$

Discussion (Newton's Method)

- Assumes Hessian $H(x^k)$ is non-singular
- No guarantee that
$$f(x^{k+1}) \le f(x^k)$$
- Can improve with line search:
$$\min_\lambda f(x^k + \lambda d^k)$$

- If $H(x^k)$ is positive definite:
  - d^k is a descent direction:
$$\nabla f(x^k)^\top d^k < 0$$
  - Proof:
$$\nabla f(x^k)^\top d^k = -\nabla f(x^k)^\top H^{-1} \nabla f(x^k)$$
  - Since H^{-1} is PD:
$$y^\top H^{-1} y > 0, \quad y \neq 0$$
  - Let $y = \nabla f(x^k)$:
$$-\nabla f(x^k)^\top H^{-1} \nabla f(x^k) < 0$$

Convergence of Newton's Method
- Assume $f \in C^3$ and:
$$\nabla f(x^*) = 0, \quad H(x^*) \text{ non-singular}$$
- If initialized close to x^*:
  Convergence is quadratic
- More precise conditions:
  - $\|H(x^*)\| \geq h$
  - $\|H(x) - H(x^*)\| \leq L\|x - x^*\|$
- Then:
$$\|x^k - x^*\| \leq C \left(\|x^0 - x^*\|^2\right)^k$$
- If Hessian is singular:
$$d^k = -(H(x^k) + D^k)^{-1} \nabla f(x^k)$$
  where D^k ensures positive definiteness.

Conjugate Gradient Method
- Quadratic problem:
$$\min f(x) = \frac{1}{2} x^\top Q x + c^\top x$$
- Directions $\{d_i\}$ are Q-conjugate if:
$$d_i^\top Q d_j = 0, \quad i \neq j$$
- Q-conjugate directions are linearly independent
- Update:
$$x^{k+1} = x^k + \alpha_k d_k$$
- Step size:
$$\alpha_k = -\frac{\nabla f(x^k)^\top d_k}{d_k^\top Q d_k}$$

Conjugate Gradient Algorithm
- Define gradient:
$$g_k = \nabla f(x_k) = Q x_k - c$$
- Initialize:
  - x^0, $k = 1$
  - $d_1 = -\nabla f(x^0)$
- Iteration:
  - If $\|\nabla f(x^k)\| \leq \epsilon$, stop
  - Compute:
$$\alpha_k = -\frac{\nabla f(x^k)^\top d_k}{d_k^\top Q d_k}$$
  - Update:
$$x^{k+1} = x^k + \alpha_k d_k$$
  - Direction update:
$$d_{k+1} = -\nabla f(x^{k+1}) + \lambda_k d_k$$
$$\lambda_k = \frac{\|\nabla f(x^{k+1})\|}{\|\nabla f(x^k)\|}$$
  - $k \leftarrow k + 1$

Duality in Optimization
- General problem:
$$\min f(x)$$
$$\text{s.t. } h_i(x) \leq 0, \ \ell_j(x) = 0$$
- Lagrangian:
$$L(x, u, v) = f(x) + \sum_i u_i h_i(x) + \sum_j v_j \ell_j(x)$$
- Dual variables:
$$u \geq 0$$
- Lower bound property:
$$f(x) \geq L(x, u, v)$$

Lagrange Dual Function
- Dual function:
$$g(u, v) = \min_x L(x, u, v)$$
- Provides lower bound on optimal value f^*

Quadratic Programming Example
- Problem:
$$\min \frac{1}{2} x^\top Q x + c^\top x$$
$$\text{s.t. } Ax = b, \ x \geq 0$$
- Lagrangian:
$$L(x, u, v) = \frac{1}{2} x^\top Q x + c^\top x - u^\top x + v^\top (Ax - b)$$
- Dual:
$$g(u, v) = -\frac{1}{2}(c - u + A^\top v)^\top Q^{-1}(c - u + A^\top v) - b^\top v$$

Lagrange Dual Problem
- Dual optimization:
$$\min g(u, v) \quad \text{s.t. } u \geq 0$$
- Weak duality:
$$f^* \geq g^*$$
- Dual is always convex

Strong Duality
- Strong duality:
$$f^* = g^*$$
- Slater's condition:
  There exists strictly feasible x
- Duality gap:
$$f(x) - g(u, v)$$
- Optimality condition:
  If gap $= 0$, solution is optimal
- Stopping criterion:
$$|f(x) - g(u, v)| \leq \epsilon$$

Karush-Kuhn-Tucker (KKT) Conditions
- General problem:
$$\min f(x)$$
$$\text{s.t. } h_i(x) \leq 0, \ \ell_j(x) = 0$$
- KKT conditions:
  - Stationarity:
$$0 \in \partial \left(f(x) + \sum_i u_i h_i(x) + \sum_j v_j \ell_j(x)\right)$$
  - Complementary slackness:
$$u_i h_i(x) = 0$$
  - Primal feasibility:
$$h_i(x) \leq 0, \quad \ell_j(x) = 0$$
  - Dual feasibility:
$$u_i \geq 0$$
- $\partial f(x)$ denotes subgradient (equals $\nabla f(x)$ if differentiable)

Necessity of KKT Conditions
- If strong duality holds:
$$f(x^*) = g(u^*, v^*)$$
- Then:
$$f(x^*) = \min_x \left(f(x) + \sum_i u_i^* h_i(x) + \sum_j v_j^* \ell_j(x)\right)$$
- Using feasibility:
$$f(x^*) \leq f(x^*) + \sum_i u_i^* h_i(x^*) + \sum_j v_j^* \ell_j(x^*) \leq f(x^*)$$
- Hence all inequalities are equalities
- Therefore KKT conditions hold

Sufficiency of KKT Conditions
- If (x^*, u^*, v^*) satisfy KKT:
$$g(u^*, v^*) = f(x^*) + \sum_i u_i^* h_i(x^*) + \sum_j v_j^* \ell_j(x^*)$$
$$= f(x^*)$$
- (by complementary slackness)
- Duality gap is zero
- Hence (x^*, u^*, v^*) are optimal

Quadratic Example with Equality Constraints

- Problem:

$$\min \frac{1}{2} x^\top Q x + c^\top x \quad \text{s.t. } Ax = b$$

- Lagrangian:

$$L(x, u) = \frac{1}{2} x^\top Q x + c^\top x + u^\top (Ax - b)$$

- KKT system:

$$\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix} = \begin{bmatrix} -c \\ 0 \end{bmatrix}$$

Markowitz Mean-Variance Portfolio

- Returns:
  - $R = \frac{x_1}{x_0}, \quad r = R - 1$
  - $x_1 = R x_0$
- Short selling:
  - Borrow asset
  - Sell at current price
  - Buy back later
  - Return asset
- Profit = sell price - buy price

Portfolio Construction

- Budget:

$$x_i = w_i x_0$$

- Weights:

$$\sum_i w_i = 1$$

- Portfolio return:

$$R = \sum_i R_i w_i$$

$$r = \sum_i r_i w_i$$

Markowitz Model

- Expected return:

$$\mathbb{E}(r) = m^\top w$$

- Variance:

$$\mathrm{Var}(r) = w^\top \Sigma w$$

- Optimization:

$$\min \frac{1}{2} w^\top \Sigma w \quad \text{s.t. } m^\top w \geq \mu_b,\ 1^\top w = 1$$

Solution via KKT

- KKT conditions:

$$\Sigma w - \lambda m - \gamma 1 = 0$$

$$m^\top w \geq \mu_b, \quad 1^\top w = 1, \quad \lambda \geq 0$$

$$\lambda(m^\top w - \mu_b) = 0$$

- Case 1: $\lambda = 0$
  - $\Sigma w = \gamma 1$
  - $w^* = (1^\top \Sigma^{-1} 1)^{-1} \Sigma^{-1} 1$
- Case 2: $m^\top w = \mu_b$
  - $w = \lambda \Sigma^{-1} m + \gamma \Sigma^{-1} 1$
  - Solve linear system for (λ, γ)

Option Pricing and Arbitrage

- Investment:
  - Initial cost:

$$p^\top x$$

  - Payoff:

$$Vx$$

- Arbitrage exists if:
  - $p^\top x < 0$
  - $Vx \geq 0$
- No-arbitrage assumption:
  No such x exists

Fundamental Theorem of Asset Pricing

- By Farkas' lemma:
  No arbitrage

$$\iff \exists\ \pi \geq 0 \text{ such that } V^\top \pi = p$$

- Since first column of V is 1:

$$1^\top \pi = 1$$

- π is interpreted as a **risk-neutral probability**
- Expected payoff:

$$V^\top \pi$$

- Pricing relation:

$$V^\top \pi = p$$

- **Conclusion:** No arbitrage $\iff$ there exists a risk-neutral probability under which asset prices equal expected payoffs

Option Price Bounds

- Suppose $p_1, \ldots, p_{n-1}$ known, p_n unknown
- Arbitrage-free bounds:

$$\min / \max_{p_n, \pi} \ p_n$$

  subject to

$$V^\top \pi = p, \quad \pi \geq 0, \quad 1^\top \pi = 1$$

- If min = max:
  Market is complete
- Completeness:
  Every contingent claim can be replicated and uniquely priced

Currency Exchange Problem

- Initial holdings:

$$c^{init} = (c_1^{init}, \ldots, c_n^{init})$$

- Target holdings:

$$c^{req} = (c_1^{req}, \ldots, c_n^{req})$$

- Exchange matrix:

$$X \in \mathbb{R}^{n \times n}, \quad X_{ij} \geq 0$$

- Cost:

$$\Delta_{ij} \text{ per unit exchange}$$

- Total cost:

$$\sum_{i,j} X_{ij} \Delta_{ij}$$

- Optimization:

$$\min_X \sum_{i,j} X_{ij} \Delta_{ij}$$

  subject to
  - $X_{ij} \geq 0$
  - $\mathrm{diag}(X) = 0$
  - Balance constraints:

$$c_i^{init} + \sum_j X_{ij} - \sum_j X_{ji} \geq c_i^{req}, \quad \forall i$$